Serie Banco ZK

3 Volúmenes · 1000+ páginas · 20 Capítulos

Volumen II

Aplicaciones y Privacidad

Capítulos 6–13: Infraestructura, DeFi, Anonimización

Contenido del Volumen 2:

Parte I: Compromisos y Estructuras Avanzadas
Pedersen • KZG • Polynomial IOPs • Lookups • Folding

Parte II: Infraestructura y Aplicaciones
Relayers • Rollups • DeFi Privado • Compliance

Parte III: Privacidad Avanzada
Ring Signatures • Stealth Addresses • Mixers

Parte IV: Protocolos de Anonimización
Privacy Coins • Unlinkability • Metadata Protection

Walter Milstein

2025

Banco ZK: Criptografía y Privacidad en Blockchain
Volumen 2: Aplicaciones y Privacidad

© 2025 Walter Milstein
Todos los derechos reservados.

Primera Edición: 2025

Serie completa:
Volumen 1: Fundamentos Teóricos (Caps. 1–5)
Volumen 2: Aplicaciones y Privacidad (Caps. 6–13)
Volumen 3: Herramientas Avanzadas y Ecosistema (Caps. 14–20)

El código incluido es de dominio público y puede ser utilizado libremente.
Este libro es para uso personal y educativo.

In memoriam Hackeris Euleri, Iacobi Rossini,

ingenii cogitantis architecti,
veritatis technicae indefessi inquisitoris,
qui codicem non solum ut seriem praeceptorum intellexit,
sed ut formam ethicae, rigoris atque libertatis.

Aspectus tuus criticus, fortitudo intellectualis
et amor scientiae
vivunt adhuc
in omni systemate recte excogitato
et in omni errore tempestive detecto.

Prerrequisitos

Este volumen asume que el lector ha completado o tiene conocimiento equivalente al contenido del **Volumen 1: Fundamentos Criptográficos y Circuitos ZK**.

Conocimientos Requeridos del Volumen 1

Cap. 1 Fundamentos Criptográficos: Funciones hash (SHA-256, Keccak, Poseidon), curvas elípticas (secp256k1, BN254, BLS12-381), firmas digitales, Merkle trees, commitment schemes básicos.

Cap. 2 Blockchain Base (L1): Estructura de transacciones, consenso, modelo UTXO vs Account, gas y fees.

Cap. 3 Escalabilidad (L2/L3): State channels, Plasma, Rollups (Optimistic y ZK), data availability.

Cap. 4 Zero-Knowledge Proofs (Core): Definición formal, propiedades (completeness, soundness, zero-knowledge), SNARKs, STARKs, sistemas de prueba.

Cap. 5 Circuitos ZK: R1CS, Plonkish arithmetization, Circom, snarkjs, Halo2, Arkworks, Noir.

Herramientas Recomendadas

Para seguir los ejemplos de código de este volumen:

Herramienta	Versión Mínima
Node.js	18+
Rust	1.70+
Circom	2.1+
snarkjs	0.7+
Foundry	Última estable
Python	3.10+

Referencias Cruzadas

A lo largo de este volumen, las referencias al Volumen 1 se indican como:

Ver Vol. 1, Cap. X, Sec. Y

Contents

Prerrequisitos **5**

Estructura del Volumen 2 **13**

1 Compromisos y Estructuras **15**
1.1 Commitment Schemes: Qué son, Para qué sirven . . . 15
1.1.1 Intuición y Analogías . . . 15
1.1.2 Propiedades Formales . . . 16
1.1.3 Aplicaciones . . . 16
1.2 Pedersen Commitments . . . 17
1.2.1 Construcción Detallada . . . 17
1.2.2 Homomorfismo . . . 18
1.2.3 Vector Pedersen Commitments . . . 19
1.3 KZG Commitments . . . 19
1.3.1 Construcción Detallada . . . 19
1.3.2 Propiedades de KZG . . . 20
1.3.3 Batch Opening . . . 21
1.3.4 KZG en la Práctica . . . 21
1.4 IPA: Inner Product Arguments . . . 22
1.4.1 Motivación: Alternativa a KZG . . . 22
1.4.2 Construcción de IPA . . . 22
1.4.3 De IPA a Polynomial Commitment . . . 23
1.4.4 Bulletproofs . . . 23
1.4.5 Comparación: KZG vs IPA vs FRI . . . 24
1.5 Polynomial IOPs . . . 25
1.5.1 Entendiendo Polynomial IOPs . . . 25
1.5.2 Estructura de un SNARK Moderno . . . 26
1.5.3 Ejemplo: PLONK como Polynomial IOP . . . 27
1.6 Lookup Arguments (Plookup) . . . 27
1.6.1 Motivación . . . 28
1.6.2 Protocolo Plookup . . . 28
1.6.3 Aplicaciones de Lookups . . . 29
1.6.4 Variantes de Lookup Arguments . . . 30
1.7 Accumulation Schemes . . . 31
1.7.1 Motivación . . . 31
1.7.2 Cómo Funciona la Acumulación . . . 32
1.8 Recursión de Pruebas . . . 33
1.8.1 Desafíos . . . 33
1.8.2 Aplicaciones de Recursión . . . 34
1.9 Folding Schemes (Nova, Supernova) . . . 35
1.9.1 La Innovación de Folding . . . 35

1.9.2 Nova . . . 36
1.9.3 Supernova . . . 37
1.9.4 Comparación de Esquemas . . . 38
1.10 Resumen del Capítulo . . . 39

2 Infraestructura ZK **41**
2.1 Relayers: Qué son, Rol en Privacidad . . . 41
2.1.1 Por qué son necesarios . . . 41
2.1.2 Modelo Económico . . . 42
2.1.3 Implementación Técnica . . . 43
2.2 Mixers: Tornado Cash, Cómo Funcionan . . . 44
2.2.1 Historia y Contexto . . . 44
2.2.2 Arquitectura Tornado Cash . . . 44
2.2.3 Componentes Técnicos . . . 45
2.3 Nullifiers . . . 46
2.3.1 Por qué Necesitamos Nullifiers . . . 46
2.4 Anonymity Set . . . 47
2.4.1 Factores que Afectan el Anonymity Set . . . 48
2.4.2 Mejorando el Anonymity Set . . . 49
2.5 Deposit/Withdraw Flow . . . 49
2.5.1 Deposit Flow Detallado . . . 49
2.5.2 Withdraw Flow Detallado . . . 50
2.6 Compliance y Proof-of-Innocence . . . 52
2.7 zkEVM . . . 53
2.7.1 El Desafío de zkEVM . . . 53
2.7.2 Tipos de zkEVM . . . 54
2.7.3 Arquitectura de un zkEVM . . . 55
2.7.4 Comparación de zkEVMs . . . 56
2.8 zkVM . . . 56
2.8.1 zkVM vs zkEVM . . . 57
2.8.2 RISC Zero . . . 57
2.8.3 SP1 . . . 59
2.8.4 Otros zkVMs . . . 60
2.8.5 Aplicaciones de zkVM . . . 61
2.9 Resumen del Capítulo . . . 63

3 Privacidad Avanzada **65**
3.1 Ring Signatures . . . 65
3.1.1 Historia y Contexto . . . 65
3.1.2 Intuición . . . 65
3.1.3 Construcción Matemática . . . 66
3.1.4 Propiedades de Seguridad . . . 67
3.1.5 Tamaño y Eficiencia . . . 67
3.1.6 Linkable Ring Signatures . . . 68
3.1.7 Ring Signatures vs Otras Técnicas . . . 70
3.2 Stealth Addresses . . . 70
3.2.1 Motivación . . . 71
3.2.2 Protocolo DKSAP . . . 71
3.2.3 Proyectos e Implementaciones . . . 73
3.2.4 Variantes de Stealth Addresses . . . 73
3.3 Viewing Keys . . . 74
3.3.1 Tipos de Viewing Keys . . . 74

3.3.2 Implementaciones . . . 75
3.3.3 Casos de Uso . . . 75
3.3.4 Viewing Keys vs ZK Disclosure . . . 76
3.4 Mixnets . . . 77
3.4.1 Historia y Contexto . . . 78
3.4.2 Arquitectura . . . 78
3.4.3 Cover Traffic . . . 79
3.4.4 Nym Network . . . 80
3.4.5 Mixnets vs Tor . . . 81
3.5 Tor e I2P . . . 81
3.5.1 Tor (The Onion Router) . . . 81
3.5.2 I2P (Invisible Internet Project) . . . 83
3.5.3 Comparación de Tecnologías de Anonimato de Red . . . 84
3.6 MPC (Multi-Party Computation) . . . 85
3.6.1 Historia y Motivación . . . 86
3.6.2 Técnicas Fundamentales . . . 86
3.6.3 Aplicaciones en Crypto . . . 89
3.6.4 MPC vs ZK: Cuándo Usar Cada Uno . . . 92
3.7 TEE (Trusted Execution Environments) . . . 93
3.7.1 Concepto y Motivación . . . 93
3.7.2 Intel SGX . . . 93
3.7.3 ARM TrustZone . . . 96
3.7.4 Otros TEEs . . . 97
3.7.5 TEE vs ZK . . . 97
3.7.6 TEE en Blockchain . . . 98
3.7.7 TEE + ZK: Combinación . . . 99
3.8 Resumen del Capítulo . . . 101

4 DeFi y Aplicaciones 103
4.1 AMMs (Automated Market Makers) . . . 103
4.1.1 Historia y Contexto . . . 103
4.1.2 Constant Product Formula (Uniswap v2) . . . 104
4.1.3 Impermanent Loss . . . 109
4.1.4 Código Solidity: AMM Simplificado . . . 110
4.1.5 Concentrated Liquidity (Uniswap v3) . . . 112
4.1.6 StableSwap (Curve Finance) . . . 114
4.1.7 Comparativa de AMMs . . . 115
4.2 Lending Protocols . . . 115
4.2.1 Historia y Contexto . . . 115
4.2.2 Modelo de Tasas de Interés . . . 116
4.2.3 Colateralización y Liquidaciones . . . 117
4.2.4 Flash Loans . . . 118
4.3 Oráculos . . . 120
4.3.1 El Problema del Oráculo . . . 120
4.3.2 Tipos de Oráculos . . . 120
4.3.3 Chainlink . . . 121
4.3.4 TWAP (Time-Weighted Average Price) . . . 121
4.3.5 Manipulación de Oráculos . . . 122
4.3.6 Chainlink VRF (Verifiable Random Function) . . . 123
4.4 Bridges ZK . . . 124
4.4.1 El Problema de los Bridges . . . 124
4.4.2 ZK Bridges: Verificación Criptográfica . . . 124

4.4.3 Proyectos de ZK Bridges . . . 126
4.4.4 Desafíos y Limitaciones . . . 126
4.5 Atomic Swaps . . . 127
4.5.1 Fundamentos . . . 127
4.5.2 HTLC (Hash Time-Locked Contracts) . . . 127
4.5.3 Limitaciones de HTLC . . . 129
4.6 Cross-chain Interoperability . . . 129
4.6.1 Protocolos de Messaging . . . 129
4.6.2 Comparativa de Protocolos Cross-chain . . . 130
4.7 Privacy Pools . . . 131
4.7.1 El Dilema Privacidad vs Compliance . . . 131
4.7.2 Arquitectura de Privacy Pools . . . 131
4.7.3 Tipos de Association Sets . . . 131
4.8 zkML (Zero-Knowledge Machine Learning) . . . 132
4.8.1 Motivación . . . 132
4.8.2 Casos de Uso de zkML . . . 132
4.8.3 Desafíos Técnicos . . . 133
4.8.4 Proyectos de zkML . . . 133
4.9 Resumen del Capítulo . . . 134

5 Avanzado e Investigación 137
5.1 zkML Avanzado . . . 137
5.1.1 Fundamentos y Arquitectura . . . 137
5.1.2 Desafíos Técnicos de zkML . . . 138
5.1.3 Frameworks de zkML . . . 140
5.1.4 Aplicaciones Prácticas de zkML . . . 140
5.1.5 Limitaciones Actuales . . . 141
5.2 Verificación Formal de Circuitos . . . 142
5.2.1 Motivación: El Problema de Soundness . . . 142
5.2.2 Tipos de Bugs en Circuitos . . . 142
5.2.3 Verificación Formal: Conceptos . . . 143
5.2.4 Herramientas de Verificación Formal . . . 144
5.2.5 Best Practices para Circuitos Seguros . . . 144
5.3 Optimización de Circuitos . . . 145
5.3.1 Métricas de Eficiencia . . . 145
5.3.2 Técnicas de Optimización . . . 145
5.3.3 Hardware Acceleration . . . 148
5.4 Proof Aggregation . . . 148
5.4.1 Motivación . . . 148
5.4.2 Métodos de Agregación . . . 149
5.5 Sistemas Recursivos . . . 150
5.5.1 Incrementally Verifiable Computation (IVC) . . . 150
5.5.2 Proof-Carrying Data (PCD) . . . 150
5.5.3 Aplicaciones de IVC/PCD . . . 150
5.5.4 Folding Schemes . . . 151
5.6 Post-Quantum ZK . . . 152
5.6.1 La Amenaza Cuántica . . . 152
5.6.2 Análisis de Seguridad por Sistema . . . 153
5.6.3 STARKs: La Opción Post-Quantum Actual . . . 153
5.6.4 Lattice-Based SNARKs . . . 154
5.6.5 Estrategias de Migración . . . 154
5.7 Resumen del Capítulo . . . 155

6 Protocolos de Anonimización **159**
6.1 CoinJoin (Bitcoin Mixing) . . . 160
6.1.1 El Problema de Privacidad en Bitcoin . . . 160
6.1.2 CoinJoin: Concepto Básico . . . 160
6.1.3 Análisis de Anonimato . . . 161
6.1.4 Requisitos para CoinJoin Efectivo . . . 161
6.1.5 Implementaciones de CoinJoin . . . 162
6.1.6 Ataques contra CoinJoin . . . 163
6.1.7 WabiSabi: CoinJoin Avanzado . . . 163
6.2 CoinShuffle / CoinShuffle++ . . . 164
6.2.1 El Problema del Coordinador . . . 164
6.2.2 Protocolo CoinShuffle . . . 164
6.2.3 CoinShuffle++ . . . 165
6.3 Chaumian e-cash . . . 166
6.3.1 Historia y Concepto . . . 166
6.3.2 Blind Signatures . . . 166
6.3.3 Protocolo e-cash Completo . . . 167
6.3.4 Prevención de Doble Gasto . . . 167
6.3.5 Resurgimiento: Fedimint y Cashu . . . 167
6.4 Dandelion / Dandelion++ . . . 168
6.4.1 El Problema de la Capa de Red . . . 168
6.4.2 Protocolo Dandelion . . . 169
6.4.3 Dandelion++ . . . 169
6.5 Dining Cryptographers Problem (DC-nets) . . . 170
6.5.1 El Problema Original . . . 170
6.5.2 Protocolo DC-net . . . 170
6.6 Mix Zones . . . 171
6.6.1 Contexto: Privacidad de Ubicación . . . 171
6.6.2 Concepto de Mix Zone . . . 171
6.6.3 Aplicaciones . . . 172
6.7 Onion Routing . . . 172
6.7.1 Concepto Fundamental . . . 172
6.7.2 Arquitectura de Tor . . . 173
6.7.3 Ataques contra Tor . . . 173
6.8 Garlic Routing (I2P) . . . 174
6.8.1 Diferencias con Onion Routing . . . 174
6.8.2 Comparación Tor vs I2P . . . 174
6.9 Comparativa de Protocolos . . . 174
6.10 Resumen del Capítulo . . . 175

7 Monedas Privadas (Privacy Coins) **179**
7.1 Zcash . . . 180
7.1.1 Historia y Fundamentos . . . 180
7.1.2 Arquitectura de Transacciones . . . 180
7.1.3 Sapling (2018) . . . 181
7.1.4 Orchard (2022) . . . 182
7.1.5 Value Balance y Binding Signature . . . 183
7.1.6 Claves en Zcash . . . 183
7.2 Monero . . . 184
7.2.1 Filosofía y Diseño . . . 184
7.2.2 Ring Signatures . . . 185
7.2.3 RingCT (Ring Confidential Transactions) . . . 185

7.2.4 Bulletproofs . . . 186
7.2.5 Stealth Addresses . . . 187
7.3 Aztec Protocol . . . 187
7.3.1 Visión y Arquitectura . . . 187
7.3.2 Noir: Lenguaje para Circuitos Privados . . . 188
7.4 Railgun . . . 188
7.4.1 Concepto: Privacidad para Tokens Existentes . . . 188
7.5 Tornado Cash . . . 188
7.5.1 Historia y Diseño . . . 188
7.5.2 Arquitectura Técnica . . . 189
7.5.3 Relayer Network . . . 189
7.6 Umbra (Stealth Payments) . . . 189
7.6.1 Concepto: Privacidad sin ZK . . . 189
7.6.2 Protocolo de Stealth Addresses . . . 190
7.7 Penumbra . . . 190
7.8 Comparativa de Privacy Coins . . . 191
7.9 Resumen del Capítulo . . . 192

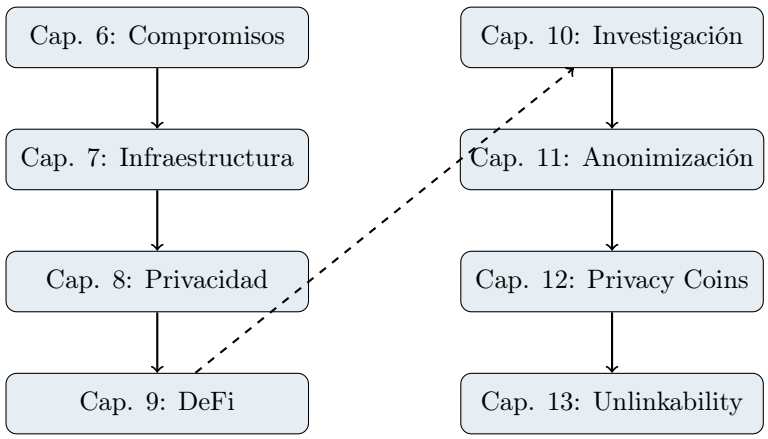
8 Técnicas de Unlinkability 195
8.1 Input/Output Unlinkability . . . 196
8.1.1 Definición Formal . . . 196
8.1.2 Métricas de Unlinkability . . . 196
8.2 Transaction Graph Analysis . . . 197
8.2.1 El Grafo de Transacciones . . . 197
8.2.2 Heurísticas de Clustering . . . 197
8.2.3 Ataques de Análisis de Grafos . . . 198
8.3 Timing Attacks y Mitigaciones . . . 199
8.3.1 Anatomía de los Timing Attacks . . . 199
8.3.2 Tipos de Timing Attacks . . . 199
8.3.3 Mitigaciones . . . 200
8.4 Amount Hiding . . . 200
8.4.1 Por Qué Ocultar Montos . . . 200
8.4.2 Técnicas de Amount Hiding . . . 200
8.5 Decoy Selection . . . 201
8.5.1 El Problema de los Decoys . . . 201
8.5.2 Distribución Gamma para Decoys . . . 202
8.5.3 Ataques contra Decoy Selection . . . 202
8.6 Modelos de Datos: Note vs UTXO vs Account . . . 202
8.6.1 Comparación de Modelos . . . 202
8.7 Shielded Pools . . . 203
8.7.1 Arquitectura de Shielded Pool . . . 203
8.7.2 Operaciones del Pool . . . 203
8.8 Splitting y Merging de Notas . . . 204
8.8.1 Motivación . . . 204
8.8.2 Riesgos de Privacidad . . . 205
8.8.3 Estrategias para Preservar Privacidad . . . 205
8.9 Resumen del Capítulo . . . 205

Conclusión del Volumen 2 209

Estructura del Volumen 2

Capítulos 6–13

Cap.	Título	Páginas Est.
6	Compromisos y Estructuras	∼50
7	Infraestructura ZK	∼45
8	Privacidad Avanzada	∼45
9	DeFi y Aplicaciones	∼40
10	Avanzado e Investigación	∼40
11	Protocolos de Anonimización	∼45
12	Monedas Privadas (Privacy Coins)	∼40
13	Técnicas de Unlinkability	∼42
	Total Volumen 2	∼347



Chapter 1

Compromisos y Estructuras

Capítulo 6 de la serie · Capítulo 1 del Volumen 2

1.1 Commitment Schemes: Qué son, Para qué sirven

Definición: Esquema de Compromiso

Protocolo criptográfico con dos fases:

1. **Commit:** Sellar un valor
2. **Open:** Revelar y verificar

1.1.1 Intuición y Analogías

Intuición

Un commitment es como un sobre sellado:

- Escribes algo en un papel y lo metes en un sobre opaco
- Sellas el sobre y lo entregas
- Nadie puede ver qué hay dentro (hiding)
- No puedes cambiar el contenido sin que se note (binding)
- Más tarde, abres el sobre para revelar el contenido

Ejemplos cotidianos:

- **Subasta sellada:** Cada participante "compromete" su oferta en un sobre. Después de cerrar, todos revelan.
- **Juego piedra-papel-tijera:** Ambos jugadores eligen y "comprometen" (mano cerrada), luego revelan simultáneamente.
- **Hash como commitment:** "Mi predicción está en el hash X" → luego revelas el texto y verifican que hashea a X.

1.1.2 Propiedades Formales

Definición: Propiedades de un Commitment Scheme

Un esquema de compromiso (C, O) sobre espacio de mensajes $\mathcal{M}$ satisface:
1. Hiding (Ocultamiento):

$$\forall m_0, m_1 \in \mathcal{M} : \text{Commit}(m_0) \approx \text{Commit}(m_1)$$

El compromiso no revela información sobre el mensaje.
2. Binding (Vinculación):

$$\Pr[\text{Open}(c, m_1, r_1) = 1 \land \text{Open}(c, m_2, r_2) = 1 \land m_1 \neq m_2] \leq \mathsf{negl}(\lambda)$$

No es posible abrir el mismo compromiso a dos mensajes diferentes.

Nota Matemática

Tipos de seguridad:

Esquema	Hiding	Binding
Pedersen	Perfect (info-theoretic)	Computational
Hash-based	Computational	Perfect (info-theoretic)
ElGamal	Computational	Computational

Teorema (Imposibilidad): No puede existir un esquema con hiding perfecto Y binding perfecto simultáneamente.
Demostración (intuición):

- Si hiding es perfecto, para cada c existen múltiples (m, r) que lo producen
- Esto significa que binding no puede ser perfecto (hay colisiones)
- Y viceversa

1.1.3 Aplicaciones

Las aplicaciones principales de los commitment schemes incluyen:

- Ocultar valores en transacciones (Confidential Tx)
- Comprometer a polinomios (SNARKs)
- Voting schemes
- Auctions selladas

Ejemplo: Aplicaciones detalladas

1. Transacciones Confidenciales:

```
# En lugar de: Transfer(Alice, Bob, 100)
# Publicamos: Transfer(Alice, Bob, Commit(100, r))

# El commitment oculta el monto
# Pero permite verificar que sum(inputs) = sum(outputs)
```

```
# Gracias al homomorfismo
```

2. Voting:

```
# Fase 1 (Commit):
#     Cada votante publica C_i = Commit(voto_i, r_i)

# Fase 2 (Reveal):
#     Cada votante revela (voto_i, r_i)
#     Se verifica C_i = Commit(voto_i, r_i)

# Fase 3 (Count):
#     Contar votos revelados

# Nadie puede cambiar su voto despues de ver otros
# (binding protege contra esto)
```

3. SNARKs:

```
# Prover compromete a polinomio p(x):
#     C = Commit(p)

# Verifier pide evaluacion en punto aleatorio z:
#     y = p(z)

# Prover genera prueba de apertura:
#     pi = Open(C, z, y)

# Verifier verifica:
#     Verify(C, z, y, pi) = 1
```

1.2 Pedersen Commitments

Propiedades clave de Pedersen (introducido en Vol. 1, Cap. 1):

- $C = g^m h^r$
- Perfectly hiding
- Computationally binding
- Homomórfico aditivo

1.2.1 Construcción Detallada

Definición: Pedersen Commitment

Sea $\mathbb{G}$ un grupo cíclico de orden primo q, y sean $g, h \in \mathbb{G}$ generadores tales que $\log_g(h)$ es desconocido.
Commit: Para mensaje $m \in \mathbb{Z}_q$ y randomness $r \xleftarrow{\$} \mathbb{Z}_q$:

$$C = g^m \cdot h^r$$

Open: Revelar (m, r) y verificar $C = g^m \cdot h^r$.

Nota Matemática

Análisis de seguridad:
Perfect Hiding:
Para cualquier commitment C, y cualquier mensaje m', existe exactamente un r' tal que $C = g^{m'}h^{r'}$:

$$r' = \log_h(C \cdot g^{-m'}) = \log_h(C) - m' \cdot \log_h(g)$$

Por lo tanto, C no da información sobre m.
Computational Binding:
Supón que puedes abrir C a dos mensajes:

$$C = g^{m_1}h^{r_1} = g^{m_2}h^{r_2}$$

Entonces:

$$g^{m_1-m_2} = h^{r_2-r_1}$$

$$\log_g(h) = \frac{m_1 - m_2}{r_2 - r_1}$$

¡Esto calcula el logaritmo discreto de h base g!
Si el DLP es difícil, binding es seguro.

1.2.2 Homomorfismo

Idea Clave

Propiedad homomórfica aditiva:

$$\mathsf{Commit}(m_1, r_1) \cdot \mathsf{Commit}(m_2, r_2) = \mathsf{Commit}(m_1 + m_2, r_1 + r_2)$$

Demostración:

$$\begin{aligned} C_1 \cdot C_2 &= (g^{m_1}h^{r_1}) \cdot (g^{m_2}h^{r_2}) \\ &= g^{m_1+m_2} \cdot h^{r_1+r_2} \\ &= \mathsf{Commit}(m_1 + m_2, r_1 + r_2) \end{aligned}$$

Aplicación: Verificar balance sin revelar montos

```
# Inputs:  C_1 = Commit(100, r1), C_2 = Commit(50, r2)
# Outputs: C_3 = Commit(80, r3), C_4 = Commit(70, r4)

# Verificar: C_1 * C_2 = C_3 * C_4 ?

# g^(100+50) * h^(r1+r2) = g^(80+70) * h^(r3+r4) ?
# g^150 * h^(r1+r2) = g^150 * h^(r3+r4)

# Si r1+r2 = r3+r4, la ecuacion se cumple.
# El verificador no conoce los montos individuales.
```

1.2.3 Vector Pedersen Commitments

Definición: Vector Pedersen Commitment

Para vector $\vec{m} = (m_1, \ldots, m_n)$ y generadores $g_1, \ldots, g_n, h$:

$$C = \prod_{i=1}^{n} g_i^{m_i} \cdot h^r = g_1^{m_1} \cdot g_2^{m_2} \cdots g_n^{m_n} \cdot h^r$$

Permite comprometer a múltiples valores en un solo elemento de grupo.

Ejemplo: Uso en Bulletproofs

```
# Comprometer a vector de bits para range proof
bits = [1, 0, 1, 1, 0, 0, 1, 0]  # 8 bits

# C = g_0^1 * g_1^0 * g_2^1 * g_3^1 * g_4^0 * g_5^0 * g_6^1 * g_7
    ^0 * h^r

# Un solo punto de curva representa 8 bits
# Verificar propiedades de bits sin revelarlos
```

1.3 KZG Commitments

Propiedades clave de KZG (introducido en Vol. 1, Cap. 1):

- Compromiso a polinomio
- Evaluación verificable en $O(1)$
- Requiere trusted setup
- Base de PLONK, Danksharding

1.3.1 Construcción Detallada

Definición: KZG Polynomial Commitment

Setup: Elegir $\tau \xleftarrow{\$} \mathbb{F}$ secreto (toxic waste). Generar:

$$\text{SRS} = (g, g^\tau, g^{\tau^2}, \ldots, g^{\tau^d}, h, h^\tau)$$

donde g genera $\mathbb{G}_1$ y h genera $\mathbb{G}_2$.
Commit: Para polinomio $p(x) = \sum_{i=0}^{d} a_i x^i$:

$$C = g^{p(\tau)} = \prod_{i=0}^{d} (g^{\tau^i})^{a_i}$$

El prover no conoce τ, solo usa los elementos del SRS.
Open (evaluación): Para probar que $p(z) = y$:

1. Calcular polinomio cociente: $q(x) = \frac{p(x) - y}{x - z}$
2. Commitment al cociente: $\pi = g^{q(\tau)}$

Verify: Verificar con pairing:

$$e(C/g^y, h) = e(\pi, h^\tau/h^z)$$

O equivalentemente:

$$e(C - g^y, h) = e(\pi, h^{\tau - z})$$

Nota Matemática

¿Por qué funciona la verificación?
Si $p(z) = y$, entonces $(x - z)$ divide a $p(x) - y$.
Existe $q(x)$ tal que:

$$p(x) - y = q(x) \cdot (x - z)$$

Evaluando en τ:

$$p(\tau) - y = q(\tau) \cdot (\tau - z)$$

En el grupo:

$$g^{p(\tau)-y} = g^{q(\tau)\cdot(\tau - z)}$$

Usando pairings:

$$e(g^{p(\tau)-y}, h) = e(g^{q(\tau)}, h^{\tau - z})$$

$$e(C \cdot g^{-y}, h) = e(\pi, h^\tau \cdot h^{-z})$$

1.3.2 Propiedades de KZG

Idea Clave

Ventajas de KZG:

Propiedad	Valor
Tamaño del commitment	48 bytes (BLS12-381)
Tamaño de prueba de evaluación	48 bytes
Tiempo de verificación	2 pairings (constante)
Batch verification	Muy eficiente
Homomorfismo	Aditivo

Desventajas:

- Requiere trusted setup
- No es post-quantum
- Setup size crece con grado máximo del polinomio

1.3.3 Batch Opening

Ejemplo: Abrir múltiples evaluaciones eficientemente

Problema: Probar $p(z_1) = y_1, p(z_2) = y_2, \ldots, p(z_k) = y_k$
Naive: k pruebas de 48 bytes cada una $= 48k$ bytes
Batch: Una sola prueba agregada

```
# Verifier elige random gamma
gamma = hash(C, z_1, y_1, ..., z_k, y_k)

# Prover construye polinomio agregado
I_x = sum(gamma**i * y_i for i in range(k))  # Interpolacion
Z_x = prod((x - z_i) for i in range(k))      # Polinomio de ceros

# Prueba unica
q_x = (p_x - I_x) / Z_x
pi = g ** q_tau

# Tamano: 48 bytes (independiente de k)
```

1.3.4 KZG en la Práctica

Observación

Usos de KZG en producción:
1. **PLONK y variantes:**
   - Commitment a polinomios del circuito
   - Pruebas de evaluación en puntos aleatorios
   - Base de zkSync, Polygon zkEVM, Scroll
2. **Ethereum Danksharding (EIP-4844):**
   - Blobs son polinomios comprometidos con KZG
   - Permite data availability sampling
   - Ceremonia con 140,000+ participantes
3. **Verkle Trees:**
   - Reemplazo propuesto para Merkle Patricia Tries
   - Pruebas de membresía mucho más pequeñas
   - Usando KZG para commitment a branches

Aplicación en Banco ZK

KZG en Banco ZK:
Usamos KZG para:

- Polynomial commitments en nuestros circuitos PLONK

- SRS de la ceremonia de Ethereum (máxima confianza)
- Pruebas de evaluación para verificación en L1

Consideración: Aunque KZG no es post-quantum, la transición a alternativas (FRI, IPA) está en nuestro roadmap para cuando sea necesario.

1.4 IPA: Inner Product Arguments

Definición: Inner Product Argument

Protocolo para probar que el producto interno de dos vectores $\vec{a}, \vec{b} \in \mathbb{F}^n$ es igual a un valor c:

$$\langle \vec{a}, \vec{b} \rangle = \sum_{i=1}^{n} a_i \cdot b_i = c$$

sin revelar los vectores completos.

1.4.1 Motivación: Alternativa a KZG

Intuición

¿Por qué IPA?
KZG tiene una debilidad: requiere **trusted setup**. Si el "toxic waste" τ se filtra, todo el sistema se rompe.
IPA ofrece una alternativa **transparente**:

- No requiere trusted setup
- Solo necesita generadores aleatorios (verificables públicamente)
- Trade-off: verificación $O(\log n)$ en lugar de $O(1)$

Comparación rápida:

Propiedad	KZG	IPA
Trusted setup	Sí	No
Tamaño prueba	$O(1)$	$O(\log n)$
Verificación	$O(1)$ pairings	$O(n)$ o $O(\log n)$*
Post-quantum	No	No

*Con técnicas de acumulación, verificación amortizada es $O(\log n)$

1.4.2 Construcción de IPA

Nota Matemática

Protocolo IPA (simplificado):
Setup público: Generadores $\vec{G} = (G_1, \ldots, G_n)$, $\vec{H} = (H_1, \ldots, H_n)$, U (todos en $\mathbb{G}$).
Commitment: Para vectores $\vec{a}, \vec{b}$ con producto interno c:

$$P = \langle \vec{a}, \vec{G} \rangle + \langle \vec{b}, \vec{H} \rangle + c \cdot U$$

Protocolo recursivo (log n rondas):
En cada ronda i, con vectores de tamaño $n_i = n/2^i$:
1. Prover calcula:

$$L_i = \langle \vec{a}_{lo}, \vec{G}_{hi} \rangle + \langle \vec{b}_{hi}, \vec{H}_{lo} \rangle + (a_{lo} \cdot b_{hi}) \cdot U$$
$$R_i = \langle \vec{a}_{hi}, \vec{G}_{lo} \rangle + \langle \vec{b}_{lo}, \vec{H}_{hi} \rangle + (a_{hi} \cdot b_{lo}) \cdot U$$

2. Verifier envía challenge $x_i \xleftarrow{\$} \mathbb{F}$
3. Prover reduce vectores:

$$\vec{a}' = \vec{a}_{lo} + x_i \cdot \vec{a}_{hi}$$
$$\vec{b}' = \vec{b}_{lo} + x_i^{-1} \cdot \vec{b}_{hi}$$
$$\vec{G}' = \vec{G}_{lo} + x_i^{-1} \cdot \vec{G}_{hi}$$
$$\vec{H}' = \vec{H}_{lo} + x_i \cdot \vec{H}_{hi}$$

4. Repetir hasta que vectores tengan tamaño 1.

Prueba final: $(L_1, R_1, \ldots, L_{\log n}, R_{\log n}, a, b)$
Tamaño: $2 \log n$ puntos de curva + 2 escalares $\approx O(\log n)$

1.4.3 De IPA a Polynomial Commitment

Idea Clave

Polynomial commitment con IPA:
Un polinomio $p(x) = \sum_{i=0}^{d} a_i x^i$ se puede ver como vector de coeficientes $\vec{a} = (a_0, \ldots, a_d)$.
Para probar $p(z) = y$:

1. Sea $\vec{b} = (1, z, z^2, \ldots, z^d)$
2. Notar que $\langle \vec{a}, \vec{b} \rangle = p(z) = y$
3. Usar IPA para probar el producto interno

Ventaja: El verifier conoce $\vec{b}$ (es público), así que solo necesita commitment a $\vec{a}$.

1.4.4 Bulletproofs

Definición: Bulletproofs

Sistema de pruebas de rango (range proofs) basado en IPA, con tamaño $O(\log n)$ sin trusted setup.

Ejemplo: Range Proof con Bulletproofs

Problema: Probar que $v \in [0, 2^n)$ sin revelar v.
Idea:

```
# v en binario: v = sum(a_i * 2^i) donde a_i in {0,1}

# Constraints:
```

```
# 1. a_i * (a_i - 1) = 0  (cada a_i es bit)
# 2. sum(a_i * 2^i) = v   (reconstruccion)

# Esto se codifica como producto interno:
# <a, 2^n> = v
# <a, a-1> = 0  (vector-wise)

# IPA prueba ambas condiciones en O(log n)
```

Tamaños (64-bit range proof):

Esquema	Tamaño
Naive (64 firmas)	∼2 KB
Borromean (Monero antiguo)	∼6 KB
Bulletproofs	∼700 bytes
Bulletproofs+	∼600 bytes

Observación

Usos de IPA/Bulletproofs en producción:

- **Monero:** Range proofs para transacciones confidenciales
- **Grin/MimbleWimble:** Toda la criptografía de transacciones
- **Halo/Halo2:** Polynomial commitment scheme (recursión sin pairings)
- **Zcash Orchard:** Migración de Groth16 a Halo2

1.4.5 Comparación: KZG vs IPA vs FRI

Propiedad	KZG	IPA	FRI
Trusted setup	Sí	No	No
Tamaño prueba	$O(1)$	$O(\log n)$	$O(\log^2 n)$
Verificación	2 pairings	$O(n)$	$O(\log^2 n)$
Prover time	$O(n)$	$O(n)$	$O(n \log n)$
Post-quantum	No	No	Sí
Assunción	DLP + pairing	DLP	Hash

Aplicación en Banco ZK

Elección de PCS en Banco ZK:
Usamos **KZG** como PCS principal por:

- Verificación $O(1)$ crítica para L1 (gas costs)
- Trusted setup de Ethereum (140K+ participantes)
- Compatibilidad con ecosistema existente

Plan de migración:

- Corto plazo: KZG (producción actual)

- Mediano plazo: Halo2/IPA (eliminar trusted setup)
- Largo plazo: FRI/STARKs (post-quantum)

La arquitectura modular (PIOP + PCS separados) permite cambiar el PCS sin reescribir los circuitos.

1.5 Polynomial IOPs

Definición: Polynomial IOP

Protocolo interactivo donde el prover envía polinomios (comprometidos) y el verifier hace queries.

Estructura típica:

1. Prover compromete a polinomios que codifican el witness
2. Verifier envía challenges aleatorios
3. Prover abre evaluaciones en puntos específicos
4. Verifier chequea relaciones polinomiales

1.5.1 Entendiendo Polynomial IOPs

Intuición

¿Por qué polinomios?
Los polinomios tienen una propiedad mágica: el **Schwartz-Zippel Lemma**.
Lemma: Si $p(x)$ es un polinomio no-cero de grado d sobre campo $\mathbb{F}$, entonces:

$$\Pr_{r \xleftarrow{\$} \mathbb{F}}[p(r) = 0] \leq \frac{d}{|\mathbb{F}|}$$

Para campos grandes ($|\mathbb{F}| \approx 2^{256}$) y grados pequeños ($d \approx 10^6$):

$$\Pr \leq \frac{10^6}{2^{256}} \approx 0$$

Implicación: Si dos polinomios $p(x)$ y $q(x)$ de grado d coinciden en un punto aleatorio, son iguales con probabilidad abrumadora.
Esto permite verificar identidades polinomiales con UN SOLO punto.

Nota Matemática

De constraints a polinomios:
Paso 1: Codificar constraints como polinomio
Un circuito con n constraints:

$$C_1(w), C_2(w), \ldots, C_n(w)$$

Se codifica como polinomio $T(X)$ que es cero en puntos $\omega^0, \omega^1, \ldots, \omega^{n-1}$:

$$T(\omega^i) = 0 \iff C_i(w) \text{ se satisface}$$

Paso 2: Verificar con vanishing polynomial
Si $T(X)$ es cero en todos los ω^i, entonces $Z_H(X) = \prod_{i=0}^{n-1}(X - \omega^i)$ divide a $T(X)$. Existe $Q(X)$ tal que:

$$T(X) = Q(X) \cdot Z_H(X)$$

Paso 3: Verificación en punto aleatorio
Verifier elige ζ aleatorio y verifica:

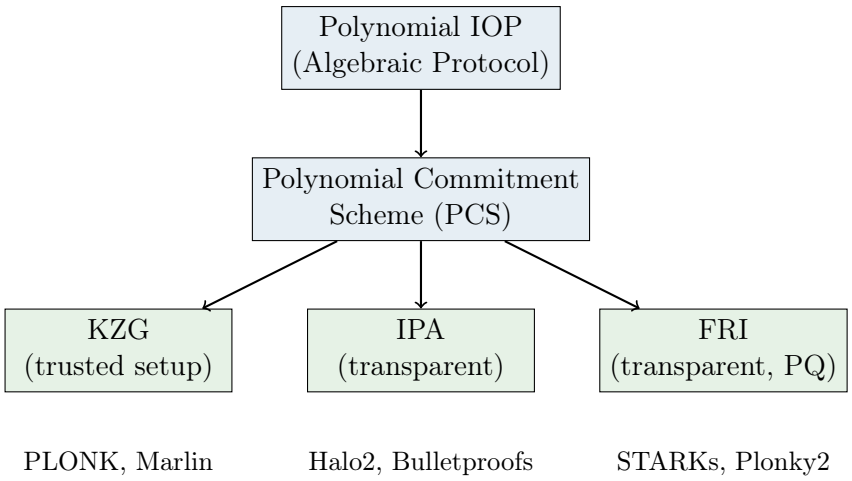
$$T(\zeta) = Q(\zeta) \cdot Z_H(\zeta)$$

Si esto se cumple, con alta probabilidad $T(X)$ es divisible por $Z_H(X)$, lo que significa que todos los constraints se satisfacen.

1.5.2 Estructura de un SNARK Moderno

SNARKs modernos = Polynomial IOP + Polynomial Commitment:

- PLONK = PIOP + KZG
- Halo2 = PIOP + IPA
- STARKs = PIOP + FRI



Idea Clave

Separación de concerns:
Polynomial IOP define:

- Qué polinomios envía el prover
- Qué relaciones deben satisfacer
- Qué queries hace el verifier

Polynomial Commitment Scheme define:

- Cómo comprometer a polinomios
- Cómo probar evaluaciones
- Propiedades de seguridad

Esta separación permite:

- Cambiar el PCS sin cambiar el PIOP
- Optimizar cada parte independientemente
- Elegir trade-offs (setup, tamaño, velocidad)

1.5.3 Ejemplo: PLONK como Polynomial IOP

Ejemplo: PLONK simplificado

Setup:

- Dominio $H = \{\omega^0, \omega^1, \ldots, \omega^{n-1}\}$ donde $\omega^n = 1$
- Polinomios selectores q_L, q_R, q_O, q_M, q_C (fijos)
- Polinomio de permutación σ (fijo)

Prover envía:

1. Commitments a $a(X), b(X), c(X)$ (valores de wires)
2. Commitment a $z(X)$ (acumulador de permutación)
3. Commitments a polinomios cocientes $t(X)$

Verifier queries:

- Punto aleatorio ζ
- Evaluaciones: $a(\zeta), b(\zeta), c(\zeta), z(\zeta), z(\omega\zeta), \ldots$

Verificación:

1. Gate constraint:

$$q_L(\zeta) \cdot a(\zeta) + q_R(\zeta) \cdot b(\zeta) + q_O(\zeta) \cdot c(\zeta) + q_M(\zeta) \cdot a(\zeta) \cdot b(\zeta) + q_C(\zeta) = t(\zeta) \cdot Z_H(\zeta)$$

2. Permutation constraint: (más complejo, involucra $z(X)$)

1.6 Lookup Arguments (Plookup)

Definición: Lookup Argument

Prueba de que un conjunto de valores está contenido en una tabla.

1.6.1 Motivación

Intuición

¿Por qué necesitamos lookups?
Algunas operaciones son difíciles de expresar como polinomios de grado bajo:

- XOR de bits: $a \oplus b$ no es algebraico
- Comparaciones: $a < b$ requiere descomposición en bits
- Funciones arbitrarias: S-boxes de AES, SHA

Solución: Si la función tiene dominio pequeño, podemos tabularla.
Ejemplo XOR de 8 bits:

```
# Tabla T con 256 x 256 = 65536 entradas:
# | a   | b   | a XOR b |
# |-----|-----|---------|
# | 0   | 0   |    0    |
# | 0   | 1   |    1    |
# | ...
# | 255 | 255 |    0    |

# Probar: (5, 3, 6) in T  (porque 5 XOR 3 = 6)
```

En lugar de constraints complicados, simplemente probamos que (a, b, c) está en la tabla.

1.6.2 Protocolo Plookup

Dado:

- Tabla $t = (t_1, \ldots, t_n)$
- Valores a verificar $f = (f_1, \ldots, f_m)$

Probar: $\forall i : f_i \in \{t_1, \ldots, t_n\}$
Idea:

1. Ordenar f y t juntos
2. Construir polinomio que relaciona listas ordenadas
3. Verificar con permutation argument

Nota Matemática

Plookup en detalle:
Observación clave:
Si f está contenido en t, podemos construir una lista s que es:

- Una permutación de t con elementos de f intercalados
- Ordenada de la misma forma que t

Construcción:
Sea $s = \text{sort}(f \cup t)$ donde cada elemento de f se coloca junto a su correspondiente en t.

Propiedad: Para elementos consecutivos en s:

$$s_i = s_{i+1} \quad \text{o} \quad (s_i, s_{i+1}) \in \{(t_j, t_{j+1})\}$$

Esta propiedad se verifica con un grand product.
Grand Product:

$$\prod_i \frac{(1+\beta)(t_i+\gamma)(t_{i+1}+\gamma)}{(s_i+\gamma)(s_{i+1}+\gamma)} = 1$$

Si esta igualdad se cumple para β, γ aleatorios, con alta probabilidad $f \subseteq t$.

Ejemplo: Plookup ilustrado

```
# Tabla t = [1, 2, 3, 4, 5]
# Valores f = [2, 4, 2]

# Paso 1: Verificar que cada f_i esta en t
# - 2 in t: OK
# - 4 in t: OK
# - 2 in t: OK

# Paso 2: Construir s = sort(f U t)
s = [1, 2, 2, 2, 3, 4, 4, 5]
#    ^  ^  ^  ^  ^  ^  ^  ^
#    t  t  f  f  t  t  f  t

# Paso 3: Verificar propiedad de consecutivos
# - (1,2): diferencia de 1 en t OK
# - (2,2): iguales OK
# - (2,2): iguales OK
# - (2,3): diferencia de 1 en t OK
# - (3,4): diferencia de 1 en t OK
# - (4,4): iguales OK
# - (4,5): diferencia de 1 en t OK

# Paso 4: Grand product con random beta, gamma
# (calculo algebraico que verifica todo a la vez)
```

1.6.3 Aplicaciones de Lookups

Aplicaciones principales:

- Range proofs eficientes
- Operaciones de bits (XOR, AND)
- S-boxes en hashes
- Cualquier función tabulable

Ejemplo: Range proof con lookup

Sin lookup (descomposición en bits):

```
# Probar x in [0, 2^16)
```

```
# Descomponer en 16 bits: 16 constraints
# Verificar cada bit es 0 o 1: 16 constraints
# Reconstruir: 1 constraint
# Total: ~33 constraints
```

Con lookup:

```
# Tabla T = [0, 1, 2, ..., 65535]
# Probar x in T: ~1 constraint (amortizado)
# Total: ~1 constraint
```

Para rangos más grandes:

```
# Probar x in [0, 2^32)
# Descomponer x = x_hi * 2^16 + x_lo
# Lookup x_hi in [0, 2^16)
# Lookup x_lo in [0, 2^16)
# Total: ~2 constraints (vs ~65 sin lookup)
```

Ejemplo: Operaciones de bits

```
# Tabla XOR de 4 bits (256 entradas)
XOR_TABLE = [(a, b, a^b) for a in range(16) for b in range(16)]

# Para calcular x XOR y donde x, y son de 8 bits:
x_lo, x_hi = x % 16, x // 16
y_lo, y_hi = y % 16, y // 16

# 4 lookups:
# lookup(x_lo, y_lo, z_lo) in XOR_TABLE
# lookup(x_hi, y_hi, z_hi) in XOR_TABLE

z = z_lo + 16 * z_hi

# Total: 4 lookups (~4 constraints) vs ~32 constraints naive
```

1.6.4 Variantes de Lookup Arguments

Observación

Evolución de lookup arguments:

Esquema	Año	Costo	Característica
Plookup	2020	$O(n)$	Original
Caulk	2022	$O(m \log m)$	Sublineal en tabla
Caulk+	2022	Mejorado	Optimizaciones
Baloo	2022	$O(m)$	Más simple
Lasso/Jolt	2023	$O(m \log m)$	Para zkVM

Tendencia: Hacia lookups más eficientes para tablas muy grandes (necesarios para zkVMs).

Aplicación en Banco ZK

Lookups en Banco ZK:
Usamos lookups para:

- Range checks de montos (asegurar $0 \leq \text{amount} < 2^{64}$)
- Verificación de índices de Merkle tree
- Optimización de operaciones de Poseidon (S-box)

Tablas predefinidas:

```
RANGE_16 = list(range(65536))            # Para range checks
POSEIDON_SBOX = [(x, x**5) for x in F]   # Para S-box
MERKLE_DEPTH = list(range(32))           # Para indices de
    profundidad
```

Beneficio: Circuitos 3-5x más pequeños que sin lookups.

1.7 Accumulation Schemes

Definición: Accumulation

Combinar múltiples instancias en una sola, difiriendo verificación.

En lugar de verificar n pruebas:

1. Acumular en un "acumulador"
2. Verificar el acumulador una vez al final
3. Costo: verificación de 1 prueba, no n

1.7.1 Motivación

Intuición

El problema de verificar muchas pruebas:
Imagina un rollup que procesa 10,000 transacciones:

- Cada tx tiene una prueba ZK individual
- Verificar cada prueba: ~200K gas
- Total naive: $10,000 \times 200K = 2$ mil millones de gas (¡imposible!)

Solución 1: Proof Aggregation

- Generar una prueba que verifica las 10,000 pruebas
- Problema: verificar pruebas dentro de circuito es MUY caro
- Un pairing en circuito $\approx$ millones de constraints

Solución 2: Accumulation

- No verificar completamente cada prueba

- "Acumular" la verificación pendiente
- Al final, una sola verificación completa
- Costo por prueba: casi nada

1.7.2 Cómo Funciona la Acumulación

Nota Matemática

Esquema de acumulación:
Setup:

- Sistema SNARK con verificación $V(\text{vk}, x, \pi) \to \{0, 1\}$
- Acumulador inicial acc_0

Acumulación:
Para cada nueva prueba π_i:

$$\text{acc}_{i+1} = \text{Accumulate}(\text{acc}_i, x_i, \pi_i)$$

El acumulador "absorbe" la prueba sin verificarla completamente.
Verificación diferida:
Al final:

$$\text{DeciderVerify}(\text{acc}_n) \to \{0, 1\}$$

Si acepta, TODAS las pruebas acumuladas eran válidas.
Propiedad clave:

$$\text{DeciderVerify}(\text{acc}_n) = 1 \iff \forall i : V(\text{vk}, x_i, \pi_i) = 1$$

Ejemplo: Acumulación en Halo

```
# Sistema IPA (Inner Product Argument)
# Verificacion normal: O(n) operaciones de grupo

# Con acumulacion:
acc = initial_accumulator()

for i in range(1000):
    proof_i = receive_proof()

    # Acumular: O(1) operaciones
    acc = accumulate(acc, proof_i)

    # NO verificamos completamente aqui

# Al final: UNA verificacion O(n)
final_check = decider_verify(acc)

# Costo total:
# - 1000 acumulaciones O(1): ~1000 ops
# - 1 verificacion final O(n): ~n ops
# vs naive: 1000 verificaciones O(n) = 1000n ops
```

1.8 Recursión de Pruebas

Definición: Prueba Recursiva

Prueba que verifica otra prueba dentro de un circuito.

1.8.1 Desafíos

- Verificar pairings en circuito es muy costoso
- Campo del circuito $\neq$ campo de la curva verificada
- Solución: cycles de curvas

Nota Matemática

El problema de los campos:
Situación:

- SNARK sobre curva E_1 trabaja en campo $\mathbb{F}_{p_1}$ (campo base)
- El circuito opera en campo $\mathbb{F}_{r_1}$ (campo escalar)
- Para verificar dentro del circuito, necesitamos aritmética de $\mathbb{F}_{p_1}$ en $\mathbb{F}_{r_1}$

Problema:
Si $p_1 \neq r_1$ (lo normal), cada operación de campo requiere:

- Aritmética de enteros grandes
- Modular reduction
- Muchos constraints

Una multiplicación de campo: $\sim$100 constraints
Un pairing: $\sim$millones de constraints
Solución: 2-cycle de curvas
Curvas E_1 y E_2 tales que:

- Campo base de E_1 = Campo escalar de E_2: $p_1 = r_2$
- Campo escalar de E_1 = Campo base de E_2: $r_1 = p_2$

Ahora:

- Prueba sobre E_1 se verifica "nativamente" en circuito sobre E_2
- Prueba sobre E_2 se verifica "nativamente" en circuito sobre E_1
- Alternamos entre las dos curvas

Ejemplo: Pasta Curves (Pallas y Vesta)

```
# Pallas: y^2 = x^3 + 5
#     Base field:   p = 0x40000000...01
#     Scalar field: q = 0x40000000...01 (diferente)

# Vesta: y^2 = x^3 + 5  (misma ecuacion, diferentes campos)
```

```
#     Base field:   q (= scalar de Pallas)
#     Scalar field: p (= base de Pallas)

# Uso en recursion:
# 1. Prueba pi_1 sobre Pallas (circuito en F_q)
# 2. Prueba pi_2 sobre Vesta verifica pi_1 (circuito en F_p)
# 3. Prueba pi_3 sobre Pallas verifica pi_2 (circuito en F_q)
# 4. ...alternando...

# Cada verificacion es ''nativa'' - no hay overhead de
# aritmetica de campo grande.
```

Usados por: Mina Protocol, Halo2

1.8.2 Aplicaciones de Recursión

- **Proof aggregation:** n pruebas $\rightarrow$ 1 prueba
- **IVC:** Computación incremental verificable
- **Rollups:** Miles de txs comprimidas
- **Blockchains sucintas:** Mina (22 KB siempre)

Ejemplo: Aplicaciones detalladas de recursión

1. Proof Aggregation para Rollups:

```
# 10,000 transacciones, cada una con proof individual
proofs = [pi_1, pi_2, ..., pi_10000]

# Arbol de agregacion (log depth)
# Level 0: 10,000 proofs
# Level 1: 5,000 proofs (cada uno verifica 2)
# Level 2: 2,500 proofs
# ...
# Level 13: 1 proof final

# La prueba final:
# - Verifica todas las 10,000 transacciones
# - Tamano: ~200-500 bytes
# - Verificacion en L1: ~200K gas
```

2. Incremental Verifiable Computation (IVC):

```
# Computacion larga: F aplicada 1 millon de veces
# F(F(F(...F(x)...))) = y

# Sin IVC: guardar todo el trace (1M estados)
# Con IVC:

state = x
proof = trivial_proof()

for i in range(1_000_000):
    new_state = F(state)
    proof = prove(
        statement=''new_state = F(state) AND proof_prev is valid'
```

```
            ',
        witness=(state, new_state, proof)
    )
    state = new_state

# proof final prueba toda la computacion
# Tamano constante, independiente de 1M pasos
```

3. Mina Protocol (Blockchain Sucinta):

```
# Blockchain tradicional: verificar = descargar todo
# Bitcoin: ~500 GB
# Ethereum: ~1 TB+

# Mina: siempre 22 KB

# Cada bloque contiene:
# - Transacciones del bloque
# - Proof recursivo que verifica:
#   - Bloque anterior era valido
#   - Nuevas transacciones son validas
#   - Estado se actualizo correctamente

# Verificar toda la blockchain = verificar 1 proof
```

1.9 Folding Schemes (Nova, Supernova)

Definición: Folding

Combinar dos instancias de un circuito en una, sin generar prueba completa.

1.9.1 La Innovación de Folding

Intuición

Folding vs Recursión tradicional:
Recursión tradicional:

- Genera prueba completa en cada paso
- Verifica prueba anterior dentro del circuito
- Costoso: circuito de verificación es grande

Folding:

- NO genera prueba en cada paso
- "Combina" dos instancias en una
- Solo al FINAL genera una prueba SNARK
- Muy barato por paso

Analogía:

- Recursión = hacer examen completo en cada clase
- Folding = tomar notas en cada clase, examen final al terminar el curso

1.9.2 Nova

Características de Nova:

- Folding para R1CS
- Overhead mínimo por paso: 2 multiplicaciones de grupo
- Solo genera prueba SNARK al final
- Ideal para IVC

Proceso:

1. Instancia 1: (u_1, w_1)
2. Instancia 2: (u_2, w_2)
3. Fold: $(u', w') = \text{Fold}((u_1, w_1), (u_2, w_2), r)$
4. u' es "running instance"
5. Al final: SNARK sobre u'

Nota Matemática

Nova en detalle:
Relaxed R1CS:
R1CS normal: $Az \circ Bz = Cz$
Nova usa "relaxed R1CS":

$$Az \circ Bz = u \cdot Cz + E$$

Donde:

- u es un escalar (1 para instancia normal)
- E es un vector de "error"

Para instancia satisfecha normal: $u = 1$, $E = \vec{0}$.
Folding de dos instancias:
Dadas:

- (u_1, E_1, w_1) satisfaciendo $Aw_1 \circ Bw_1 = u_1 Cw_1 + E_1$
- (u_2, E_2, w_2) satisfaciendo $Aw_2 \circ Bw_2 = u_2 Cw_2 + E_2$

Fold con random r:

$$\begin{aligned} u' &= u_1 + r \cdot u_2 \\ w' &= w_1 + r \cdot w_2 \\ E' &= E_1 + r \cdot T + r^2 \cdot E_2 \end{aligned}$$

Donde T es un "cross term" calculado por el prover.
Propiedad mágica:
(u', E', w') satisface relaxed R1CS si y solo si ambas instancias originales lo satisfacían.
Costo del fold:

- Prover: calcular T, 1 commitment
- Verifier: 2 multiplicaciones escalares de grupo
- ¡Sin verificación de SNARK!

Ejemplo: IVC con Nova

```
// Funcion a iterar
fn F(state: State) -> State {
    // Alguna computacion
}

// IVC con Nova
fn nova_ivc(initial_state: State, n: usize) -> (State, Proof) {
    // Inicializar
    let mut running_instance = trivial_instance();
    let mut state = initial_state;

    for i in 0..n {
        // Calcular siguiente estado
        let new_state = F(state);

        // Crear instancia para este paso
        let step_instance = create_instance(state, new_state);

        // Fold (muy barato!)
        running_instance = fold(running_instance, step_instance);

        state = new_state;
    }

    // Solo al final: generar SNARK
    let proof = spartan_prove(running_instance);

    return (state, proof);
}

// Costos:
// - Cada fold: ~10us
// - SNARK final: ~100ms
// - 1 millon de pasos: ~10s (vs dias con recursion tradicional)
```

1.9.3 Supernova

Extensión de Nova para circuitos heterogéneos.

- Permite alternar entre diferentes circuitos
- Selector de cuál circuito ejecutar
- Útil para VMs y computación general

Idea Clave

Nova vs Supernova:
Nova:

- Un circuito F
- Ejecutar: $F, F, F, F, \ldots$ (siempre el mismo)
- Ideal para: hash chains, funciones recursivas simples

Supernova:

- Múltiples circuitos $F_1, F_2, \ldots, F_k$
- Ejecutar: $F_2, F_1, F_3, F_1, F_2, \ldots$ (cualquier orden)
- Selector indica cuál ejecutar en cada paso
- Ideal para: VMs, computación general

Ejemplo VM:

```
# Circuitos:
F_1 = ''ADD instruction''
F_2 = ''MUL instruction''
F_3 = ''LOAD instruction''
F_4 = ''STORE instruction''
F_5 = ''JUMP instruction''

# Ejecucion:
# LOAD, ADD, MUL, STORE, JUMP, LOAD, ...
#   F_3,  F_1,  F_2,   F_4,  F_5,  F_3, ...

# Cada instruccion se ''foldea'' con la running instance
```

1.9.4 Comparación de Esquemas

Aspecto	Recursión	Nova	Supernova
Costo por paso	Alto	Muy bajo	Bajo
Circuitos	Uno	Uno	Múltiples
Prueba intermedia	Sí	No	No
Ideal para	Aggregation	IVC simple	VM
Prover final	Igual	Necesario	Necesario

Observación

Estado del arte (2024-2025):

- **Nova:** Producción (Microsoft, varios proyectos)
- **Supernova:** Desarrollo avanzado
- **HyperNova:** Extensión con custom constraints
- **ProtoStar:** Folding para Plonkish

- **Sangria:** Folding para PLONK

Tendencia: folding se está volviendo el estándar para IVC.

🏛 Aplicación en Banco ZK

Folding en Banco ZK:
Aplicación actual:

- Acumular transacciones incrementalmente
- Cada tx se "foldea" con la running instance
- Batch de 1000 txs: 1000 folds + 1 SNARK final

Beneficios:

- Prover time por tx: ∼10ms (vs ∼1s con recursión)
- Memoria constante: no necesita guardar proofs intermedios
- Latencia reducida: fold es casi instantáneo

Arquitectura:

```
# Transacciones entrantes:
#     tx_1 -> fold -> running_1
#     tx_2 -> fold -> running_2
#     tx_3 -> fold -> running_3
#     ...
#     tx_1000 -> fold -> running_1000

# Cuando batch esta listo:
#     proof = SNARK(running_1000)
#     publish(proof) to L1
```

Futuro: Supernova

- Diferentes tipos de tx (transfer, deposit, withdraw)
- Cada tipo es un circuito diferente
- Supernova permite mezclarlos eficientemente

1.10 Resumen del Capítulo

🔑 Idea Clave

Conceptos clave de este capítulo:

1. **Commitment Schemes:** Sellar valores con hiding + binding
2. **Pedersen:** $C = g^m h^r$, perfectly hiding, homomórfico
3. **KZG:** Polynomial commitment, $O(1)$ verification, trusted setup
4. **Polynomial IOPs:** Base de SNARKs modernos, Schwartz-Zippel

5. **Lookups (Plookup):** Verificar membresía en tablas eficientemente
6. **Accumulation:** Diferir verificación, verificar una vez al final
7. **Recursión:** Prueba que verifica otra prueba, cycles de curvas
8. **Folding (Nova):** Combinar instancias sin generar prueba, IVC eficiente

Chapter 2

Infraestructura ZK

Capítulo 7 de la serie · Capítulo 2 del Volumen 2

2.1 Relayers: Qué son, Rol en Privacidad

Definición: Relayer

Servicio que envía transacciones on-chain en nombre del usuario.

2.1.1 Por qué son necesarios

Sin relayer:

- Usuario necesita ETH para gas
- Pagar gas desde una dirección vincula esa dirección al retiro
- Se rompe la privacidad

Con relayer:

- Usuario envía prueba al relayer (off-chain)
- Relayer envía tx y paga gas
- Fee deducido del retiro
- No hay vínculo on-chain entre usuario y tx

Intuición

El problema del gas:
Imagina que depositaste 1 ETH en un mixer desde la dirección A. Quieres retirarlo a una dirección nueva B (limpia, sin historial).
Sin relayer:

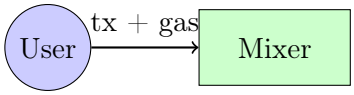
- B necesita ETH para pagar gas
- ¿De dónde viene ese ETH? De alguna dirección C
- C está vinculada a ti (la financiaste de alguna forma)

- Un observador puede conectar: C → B, y sospechar que B es tuya

Con relayer:

- Envías la prueba ZK al relayer (off-chain, encriptado)
- Relayer envía la tx y paga el gas
- El fee (0.1–1%) se resta de tu retiro
- B aparece on-chain sin ninguna conexión previa contigo

Sin Relayer:



¡Dirección del user visible!

Con Relayer:



Solo relayer visible on-chain

2.1.2 Modelo Económico

- Fee típico: 0.1–1% del retiro
- Competencia entre relayers reduce fees
- Puede ser centralizado o red descentralizada
- Incentivos para uptime y servicio

Nota Matemática

Economía de relayers:
Ingresos del relayer:

- Fee por transacción: f (ej: 0.3%)
- Volumen diario: V (ej: 1000 ETH)
- Ingresos diarios: $f \times V = 3$ ETH

Costos del relayer:

- Gas por tx: ∼0.01 ETH
- Infraestructura: servidores, monitoreo
- Capital: ETH para pagar gas antes de recibir fee

Competencia:

- Múltiples relayers compiten por transacciones
- Usuarios eligen por fee más bajo
- Equilibrio de mercado determina fee

Modelos de implementación:

- **Centralizado:** Un operador, riesgo de censura
- **Red P2P:** Múltiples operadores, más resistente
- **DAO-managed:** Gobernanza comunitaria

2.1.3 Implementación Técnica

Ejemplo: Flujo de relayer

```
# 1. Usuario genera prueba localmente
proof = generate_proof(secret, nullifier, merkle_path, recipient)

# 2. Usuario prepara request para relayer
request = {
    ''proof'': proof,
    ''public_inputs'': {
        ''root'': current_merkle_root,
        ''nullifier_hash'': hash(nullifier),
        ''recipient'': recipient_address,
        ''relayer'': relayer_address,
        ''fee'': 0.003 * amount,  # 0.3%
    },
    ''signature'': sign(request, user_key)  # Opcional
}

# 3. Enviar a relayer (off-chain, puede ser HTTPS o Tor)
response = await relayer.submit(request)

# 4. Relayer verifica y envia tx
# En el backend del relayer:
def handle_request(request):
    # Verificar que la prueba es valida
    if not verify_proof(request.proof, request.public_inputs):
        return error(''Invalid proof'')

    # Verificar que el fee es suficiente
    if request.fee < min_fee:
        return error(''Fee too low'')

    # Enviar tx on-chain
    tx = mixer_contract.withdraw(
        request.proof,
        request.public_inputs
    )

    # Relayer paga gas, recibe fee del contrato
    return tx.hash
```

2.2 Mixers: Tornado Cash, Cómo Funcionan

Definición: Mixer

Protocolo que mezcla fondos de múltiples usuarios para romper trazabilidad.

2.2.1 Historia y Contexto

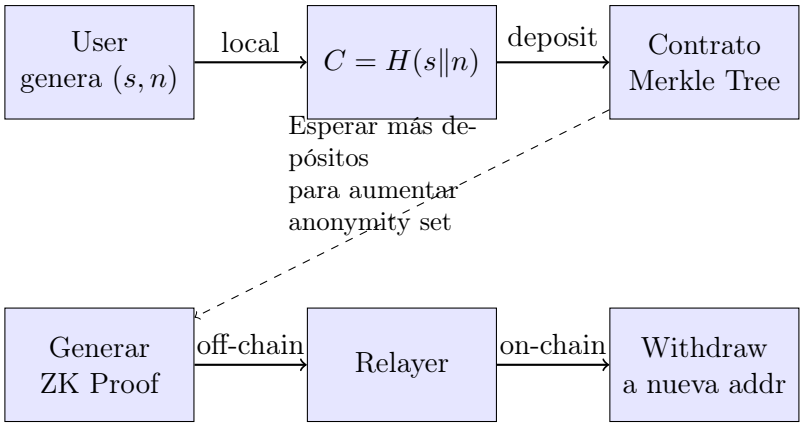
Observación

Evolución de los mixers:

Era	Tecnología	Problemas
2013	Mixers centralizados	Confianza en operador
2014	CoinJoin (Bitcoin)	Coordinación, trazabilidad parcial
2016	Zcash (shielded)	Nueva blockchain requerida
2019	Tornado Cash	ZK en Ethereum, trustless
2022	Tornado sanctioned	Regulación, debates legales
2023+	Privacy pools	Compliance + privacidad

2.2.2 Arquitectura Tornado Cash

1. **Deposit:**
   - Usuario genera secretos (s, n) localmente
   - Calcula commitment $C = H(s\|n)$
   - Envía tx: deposit(C, monto)
   - Contrato inserta C en Merkle tree
2. **Wait:**
   - Más depósitos aumentan anonymity set
   - Usuario guarda (s, n) de forma segura
3. **Withdraw:**
   - Obtener Merkle proof para C
   - Calcular nullifier $N = H(s)$ (nota: diferente de commitment)
   - Generar prueba ZK
   - Enviar vía relayer
   - Contrato verifica y paga



2.2.3 Componentes Técnicos

- Smart contracts (Ethereum)
- Circuitos Circom
- Frontend (descentralizado vía IPFS)
- Red de relayers
- Denominaciones fijas: 0.1, 1, 10, 100 ETH

Ejemplo: Circuito Tornado Cash (simplificado)

```
// withdraw.circom (simplificado)
template Withdraw(levels) {
    // Public inputs
    signal input root;
    signal input nullifierHash;
    signal input recipient;
    signal input relayer;
    signal input fee;

    // Private inputs (witness)
    signal input secret;
    signal input nullifier;
    signal input pathElements[levels];
    signal input pathIndices[levels];

    // 1. Verificar que el commitment esta en el tree
    component hasher = Poseidon(2);
    hasher.inputs[0] <== secret;
    hasher.inputs[1] <== nullifier;
    signal commitment <== hasher.out;

    component merkle = MerkleTreeChecker(levels);
    merkle.leaf <== commitment;
    merkle.root <== root;
    for (var i = 0; i < levels; i++) {
        merkle.pathElements[i] <== pathElements[i];
        merkle.pathIndices[i] <== pathIndices[i];
    }

    // 2. Verificar nullifier hash
```

```
    component nullifierHasher = Poseidon(1);
    nullifierHasher.inputs[0] <== nullifier;
    nullifierHasher.out === nullifierHash;

    // 3. Hashear recipient/relayer/fee (anti-frontrun)
    signal recipientSquare;
    recipientSquare <== recipient * recipient;
}
```

2.3 Nullifiers

Definición: Nullifier

Valor único derivado de un depósito que previene doble gasto.

Propiedades:

- Determinístico: mismo depósito → mismo nullifier
- Único: diferentes depósitos → diferentes nullifiers
- No vinculable: observador no puede vincular N con C

Implementación típica:

$$N = H(\text{secret} \| \text{leaf_index})$$

2.3.1 Por qué Necesitamos Nullifiers

Intuición

El problema del doble gasto en mixers:
Sin nullifiers:

- Usuario deposita 1 ETH, obtiene commitment C
- Retira 1 ETH con prueba de que C está en el tree
- ¡Retira OTRA VEZ con la misma prueba!
- El commitment sigue en el tree
- Puede retirar infinitamente

Solución con nullifiers:

- Al retirar, revela $N = H(\text{secret})$
- Contrato registra N en un conjunto de "usados"
- Si N ya está en el conjunto, rechaza
- Cada depósito solo puede retirarse una vez

Propiedad clave:

- N está determinado por el secreto

- Pero observador no puede vincular N con C
- Porque $C = H(\text{secret}, \text{nullifier_seed})$
- Y $N = H(\text{nullifier_seed})$ usa input diferente

Nota Matemática

Diseño criptográfico de nullifiers:
Requerimientos:

1. **Unicidad:** Diferentes depósitos $\rightarrow$ diferentes N
2. **Determinismo:** Mismo depósito siempre produce mismo N
3. **No-vinculabilidad:** Dado N, no se puede determinar C
4. **Prueba de conocimiento:** Solo el dueño puede producir N válido

Construcción de Tornado Cash:

```
# Deposit:
secret = random(31)  # bytes
nullifier_seed = random(31)  # bytes

commitment = Poseidon(secret, nullifier_seed)
note = (secret, nullifier_seed, leaf_index)

# Withdraw:
nullifier_hash = Poseidon(nullifier_seed)

# ZK proof demuestra:
# 1. Conoce (secret, nullifier_seed) tal que
#    Poseidon(secret, nullifier_seed) esta en el Merkle tree
# 2. nullifier_hash = Poseidon(nullifier_seed)
```

Por qué funciona:

- C depende de ambos: secret Y nullifier_seed
- N solo depende de nullifier_seed
- Dado N, no puedes deducir secret
- Dado N, no puedes calcular C sin secret
- $\rightarrow$ No vinculable

2.4 Anonymity Set

Definición: Anonymity Set

Conjunto de participantes entre los cuales un usuario es indistinguible.

Nota Matemática

Con n depósitos del mismo monto:

- Probabilidad de linking correcto: $1/n$
- $n = 100 \rightarrow 1\%$
- $n = 10,000 \rightarrow 0.01\%$
- $n = 1,000,000 \rightarrow 0.0001\%$

2.4.1 Factores que Afectan el Anonymity Set

Factores que reducen anonymity set:

- Depositar y retirar inmediatamente
- Montos únicos
- Misma dirección para deposit y withdraw
- Patrones de comportamiento

Ejemplo: Ataques al anonymity set

1. Timing attack:

```
# Depositos en pool de 1 ETH:
#     09:00 - Alice deposita
#     09:01 - Bob deposita
#     09:02 - Carol deposita
#     09:03 - Retiro a direccion X

# Si hay pocos depositos, probabilidad alta de vincular.
# Con solo 3 depositos: 33% de adivinar.
```

2. Amount correlation:

```
# Si Alice deposita exactamente 1.337 ETH:
#     Solo ella deposito ese monto
#     Cualquier retiro de 1.337 ETH es de Alice
#     Anonymity set = 1

# Solucion: denominaciones fijas (0.1, 1, 10, 100 ETH)
```

3. Behavioral patterns:

```
# Alice siempre retira a la misma hora
# Alice siempre usa el mismo relayer
# Alice retira a direcciones que luego interactuan entre si

# Analisis de grafos puede reducir anonymity set
```

4. Dust attacks:

```
# Atacante envia pequenas cantidades a muchas direcciones
# Rastreo: ''este retiro envio dust a direccion conocida''
# Puede vincular deposito con retiro
```

2.4.2 Mejorando el Anonymity Set

Idea Clave

Mejores prácticas para máxima privacidad:

1. **Esperar tiempo suficiente:**
   - Más tiempo = más depósitos = mayor anonymity set
   - Mínimo recomendado: semanas a meses
   - Ideal: esperar hasta que anonymity set sea grande
2. **Usar denominaciones populares:**
   - Pools más grandes tienen más usuarios
   - 0.1 ETH: millones de depósitos
   - 100 ETH: miles de depósitos
3. **Evitar patrones:**
   - Variar tiempos de retiro
   - Usar diferentes relayers
   - No consolidar retiros
4. **Nueva dirección para cada retiro:**
   - Nunca reutilizar direcciones de retiro
   - No enviar desde retiro a direcciones conocidas tuyas
5. **Múltiples hops:**
   - Depositar, retirar, depositar otra vez
   - Aumenta costo pero mejora privacidad

2.5 Deposit/Withdraw Flow

2.5.1 Deposit Flow Detallado

1. Usuario genera aleatoriamente:
   - s: secret (31 bytes)
   - n: nullifier seed (31 bytes)
2. Calcula $C = \text{Poseidon}(s, n)$
3. Firma y envía tx con C y monto
4. Contrato verifica monto y añade C al tree
5. Usuario almacena "note": $(s, n, \text{leafIndex})$

Ejemplo: Código de depósito

```
# Frontend (JavaScript/Python)
async def deposit(amount):
    # 1. Generar secretos localmente
    secret = crypto.randomBytes(31)
    nullifier_seed = crypto.randomBytes(31)

    # 2. Calcular commitment
    commitment = poseidon([secret, nullifier_seed])

    # 3. Enviar transaccion
    tx = await tornado_contract.deposit(
        commitment,
        value=amount
    )
    await tx.wait()

    # 4. Guardar note (CRITICO! Sin esto, fondos perdidos)
    leaf_index = await get_leaf_index(commitment)
    note = encode_note({
        ''secret'': secret,
        ''nullifier_seed'': nullifier_seed,
        ''leaf_index'': leaf_index,
        ''amount'': amount,
        ''currency'': ''ETH''
    })

    # 5. Mostrar note al usuario para backup
    print(f''Your note: {note}'')
    # Formato: tornado-eth-1-0x1234...

    return {''tx'': tx, ''note'': note}
```

2.5.2 Withdraw Flow Detallado

1. Usuario recupera note (s, n, idx)
2. Obtiene Merkle path para leaf idx
3. Calcula nullifier $N = \text{Poseidon}(n)$
4. Genera ZK proof que demuestra:
   - Conoce (s, n) tal que $\text{Poseidon}(s, n)$ está en tree
   - Merkle path es válido para root R
   - N es el nullifier correcto
   - Recipient address está hasheado en la prueba
5. Envía $(N, \text{proof}, \text{recipient})$ a relayer
6. Relayer envía tx al contrato
7. Contrato verifica:
   - N no usado antes

- Proof válida
- Root R es o fue válido

8. Contrato registra N y envía fondos

Ejemplo: Código de retiro

```
# Frontend (JavaScript/Python)
async def withdraw(note_string, recipient_address, relayer_url):
    # 1. Decodificar note
    note = decode_note(note_string)
    secret = note[''secret'']
    nullifier_seed = note[''nullifier_seed'']
    leaf_index = note[''leaf_index'']
    amount = note[''amount'']

    # 2. Obtener datos actuales del contrato
    root = await tornado_contract.getLastRoot()
    merkle_proof = await get_merkle_proof(leaf_index)

    # 3. Calcular nullifier hash
    nullifier_hash = poseidon([nullifier_seed])

    # 4. Generar prueba ZK (costoso, segundos)
    input_data = {
        # Public
        ''root'': root,
        ''nullifierHash'': nullifier_hash,
        ''recipient'': recipient_address,
        ''relayer'': relayer_address,
        ''fee'': relayer_fee,

        # Private (witness)
        ''secret'': secret,
        ''nullifier'': nullifier_seed,
        ''pathElements'': merkle_proof.path_elements,
        ''pathIndices'': merkle_proof.path_indices
    }

    proof, public_signals = await snarkjs.groth16.fullProve(
        input_data,
        ''withdraw.wasm'',
        ''withdraw.zkey''
    )

    # 5. Enviar a relayer
    response = await fetch(relayer_url + ''/relay'', {
        ''method'': ''POST'',
        ''body'': json.dumps({
            ''proof'': proof,
            ''args'': public_signals
        })
    })

    # 6. Esperar confirmacion
    tx_hash = (await response.json())[''txHash'']
    return await provider.waitForTransaction(tx_hash)
```

2.6 Compliance y Proof-of-Innocence

Definición: Proof-of-Innocence

Prueba de que fondos no provienen de fuentes ilícitas, sin revelar origen exacto.

Mecanismos:

- Probar depósito antes de fecha X (antes de hack)
- Probar no-inclusión en lista de direcciones sancionadas
- Probar que origen está en "association set" legítimo
- Viewing keys para auditoría selectiva

Idea Clave

Privacy Pools (propuesta 2023):
Respuesta al problema: "¿Cómo tener privacidad Y cumplir regulaciones?"
Idea: Association Sets

```
# Association Set = conjunto de depositos ''buenos''
#     - Depositantes verificados
#     - Antes de eventos ilicitos conocidos
#     - Aprobados por DAO/comunidad

# Al retirar, pruebo (en ZK):
#     1. Mi deposito esta en el Merkle tree del mixer (privacidad)
#     2. Mi deposito esta en el Association Set (compliance)

# No revelo CUAL deposito, solo que:
#     - Es valido (tengo derecho a retirar)
#     - Es ''inocente'' (pertenece al set bueno)
```

Beneficios:

- Privacidad: nadie sabe cuál depósito específico
- Compliance: reguladores saben que no es fondos ilícitos
- Flexible: diferentes association sets para diferentes requisitos

Aplicación en Banco ZK

Compliance en Banco ZK:
Enfoque multi-nivel:
1. Screening de depósitos:

- Verificar que dirección de origen no está sancionada
- Opcional: KYC para depósitos grandes
- Rechazar depósitos de addresses conocidas como ilícitas

2. Proof-of-Innocence al retirar:

- Usuarios pueden generar prueba de association set

- Exchanges/servicios pueden requerir esta prueba
- Sin revelar depósito específico

3. **Viewing keys opcionales:**
   - Usuario puede revelar historial a auditor específico
   - Útil para impuestos, compliance corporativo
   - Solo con consentimiento del usuario

Arquitectura:

```
# Modo default: privacidad total
#     -> Anonymity set completo

# Modo compliance: privacidad + proof-of-innocence
#     -> Prueba de association set

# Modo audit: revelacion selectiva
#     -> Viewing key a auditor especifico
```

2.7 zkEVM

Definición: zkEVM

EVM completa ejecutada dentro de un circuito ZK.

2.7.1 El Desafío de zkEVM

Intuición

¿Por qué es difícil hacer zkEVM?
La EVM fue diseñada para ejecución, NO para ZK proofs:

- **Keccak256:** Hash usado en todo, pero muy costoso en ZK
- **256 bits:** Aritmética en campo "incómodo" para ZK
- **Stack machine:** Acceso dinámico es caro en circuitos
- **Memoria dinámica:** Tamaño variable, difícil de circuitizar
- **Storage:** Merkle Patricia Trie con Keccak (doble problema)

Trade-off fundamental:

- Más compatible con Ethereum → Circuitos más grandes → Proofs más lentos
- Menos compatible → Circuitos más eficientes → Proofs más rápidos

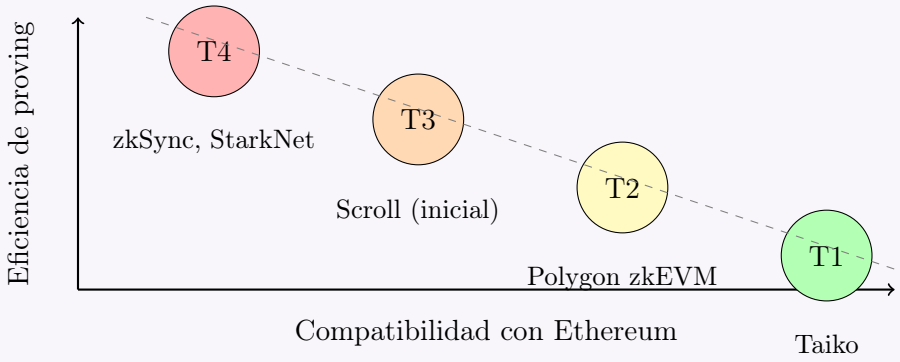
2.7.2 Tipos de zkEVM

Tipo	Nombre	Descripción
1	Ethereum-equivalent	Idéntico a Ethereum, incluye todo
2	EVM-equivalent	Compatible con EVM, pequeñas diferencias
3	EVM-compatible	Algunas modificaciones para eficiencia
4	Language-compatible	Lenguaje propio que compila a EVM-like

- **Tipo 1:** Scroll, Taiko (más lento, totalmente compatible)
- **Tipo 2:** Polygon zkEVM (muy compatible)
- **Tipo 3:** Scroll (en desarrollo hacia tipo 1)
- **Tipo 4:** zkSync Era, StarkNet (lenguajes propios)

Nota Matemática

Espectro de compatibilidad:



Tipo 1 (Ethereum-equivalent):

- Puede verificar bloques de Ethereum directamente
- Mismo state tree (MPT), mismo hash (Keccak)
- Contratos de Ethereum funcionan sin cambios
- Proving: muy lento (horas por bloque)

Tipo 2 (EVM-equivalent):

- Mismo bytecode EVM
- Pequeñas diferencias en precompilados, gas costs
- Contratos funcionan casi siempre sin cambios
- Proving: más rápido (minutos)

Tipo 3 (EVM-compatible):

- Opcodes modificados para eficiencia ZK

- Algunos precompilados diferentes
- Mayoría de contratos funcionan
- Proving: relativamente rápido

Tipo 4 (Language-compatible):

- Lenguaje nuevo (Cairo, Zinc)
- Compila a bytecode optimizado para ZK
- Contratos deben reescribirse
- Proving: muy rápido

2.7.3 Arquitectura de un zkEVM

Ejemplo: Componentes de Polygon zkEVM

```
# Arquitectura de Polygon zkEVM:
#
# +------------------+
# |   Transactions   |
# +--------+---------+
#          |
#          v
# +------------------+
# |    Sequencer     | <- Ordena y ejecuta txs
# +--------+---------+
#          |
#          v
# +------------------+
# |   Executor +     | <- Genera trace de ejecucion
# |   State Manager  |
# +--------+---------+
#          |
#          v
# +------------------+
# |  zkProver        | <- Genera ZK proof del trace
# |  (multiples      |
# |   componentes)   |
# +--------+---------+
#          |
#          v
# +------------------+
# |  Aggregator      | <- Agrega proofs, envia a L1
# +--------+---------+
#          |
#          v
# +------------------+
# |  L1 Contract     | <- Verifica proof, actualiza state
# +------------------+

# Componentes del zkProver:
# - Main SM: Orquesta sub-maquinas
```

```
# - Arithmetic SM: Operaciones aritmeticas
# - Binary SM: Operaciones de bits
# - Memory SM: Accesos a memoria
# - Storage SM: Accesos a storage (MPT)
# - Keccak SM: Hash Keccak256
# - Poseidon SM: Hash Poseidon (interno)
# - ROM SM: Bytecode execution
```

2.7.4 Comparación de zkEVMs

Proyecto	Tipo	Proof System	Estado	TPS aprox
Scroll	2	Halo2	Mainnet	20–50
Polygon zkEVM	2–3	PLONK + STARK	Mainnet	30–50
zkSync Era	4	PLONK	Mainnet	100+
StarkNet	4	STARK	Mainnet	50–100
Taiko	1	Multi-prover	Mainnet	10–20
Linea	2	Custom	Mainnet	20–40

Observación

Evolución de zkEVMs (2024–2025):
Tendencias:

- Todos moviéndose hacia Tipo 1–2 (máxima compatibilidad)
- Proving paralelo y distribuido
- Hardware acceleration (GPU, FPGA)
- Pruebas recursivas para batches más grandes

Desafíos actuales:

- Proving time sigue siendo alto (minutos a horas)
- Costo de proving ($10–100 por bloque)
- Descentralización de secuenciadores
- Bugs en circuitos complejos

2.8 zkVM

Definición: zkVM

Máquina virtual de propósito general con ejecución verificable en ZK.

2.8.1 zkVM vs zkEVM

Idea Clave

Diferencia conceptual:
zkEVM:

- Emula específicamente la EVM de Ethereum
- Objetivo: compatibilidad con ecosistema Ethereum
- Trade-off: eficiencia por compatibilidad

zkVM:

- Máquina virtual de propósito general
- Objetivo: probar cualquier programa
- Típicamente: arquitectura RISC (RISC-V, MIPS)
- Trade-off: flexibilidad por simplicidad de circuito

¿Cuándo usar cada uno?

- zkEVM: cuando necesitas compatibilidad con contratos Ethereum
- zkVM: cuando escribes programa desde cero y quieres eficiencia

2.8.2 RISC Zero

- Ejecuta RISC-V en ZK
- Cualquier programa Rust/C verificable
- Guest programs + host interaction
- Bonsai: proving service en la nube

Nota Matemática

Arquitectura RISC Zero:
Modelo de ejecución:

```
# Host (no-ZK)           Guest (dentro de ZK)
# +-----------+          +-------------------+
# |           |  input   |                   |
# |   Host    +--------->|  Guest Program    |
# |  Program  |          |   (RISC-V)        |
# |           |<---------+                   |
# +-----------+  output  +-------------------+
#      |                         |
#      v                         v
#   Normal                  ZK Proof
#   execution               of execution
```

El guest:

- Programa Rust compilado a RISC-V
- Ejecutado dentro del zkVM

- Toda la ejecución genera prueba

El host:

- Prepara inputs para el guest
- Ejecuta el guest en el zkVM
- Recibe outputs y proof

Ejemplo: Programa RISC Zero

```rust
// guest/src/main.rs (ejecutado en zkVM)
#![no_main]
use risc0_zkvm::guest::env;

risc0_zkvm::guest::entry!(main);

fn main() {
    // Leer input del host
    let n: u32 = env::read();

    // Computacion que queremos probar
    let result = fibonacci(n);

    // Escribir output (sera parte de la prueba)
    env::commit(&result);
}

fn fibonacci(n: u32) -> u64 {
    if n <= 1 { return n as u64; }
    let (mut a, mut b) = (0u64, 1u64);
    for _ in 1..n {
        let temp = a + b;
        a = b;
        b = temp;
    }
    b
}

// host/src/main.rs (ejecutado normalmente)
use risc0_zkvm::{default_prover, ExecutorEnv};
use methods::{FIBONACCI_ELF, FIBONACCI_ID};

fn main() {
    // Preparar input
    let n: u32 = 50;

    // Configurar environment
    let env = ExecutorEnv::builder()
        .write(&n)
        .unwrap()
        .build()
        .unwrap();

    // Ejecutar y generar prueba
```

```
    let prover = default_prover();
    let receipt = prover.prove(env, FIBONACCI_ELF).unwrap();

    // Verificar
    receipt.verify(FIBONACCI_ID).unwrap();

    // Leer resultado
    let result: u64 = receipt.journal.decode().unwrap();
    println!(''fibonacci({}) = {} (verified!)'', n, result);
}
```

2.8.3 SP1

- zkVM de Succinct Labs
- Muy rápido para recursión
- Optimizado para blockchain
- Soporta Rust nativo

Nota Matemática

Innovaciones de SP1:

1. **Precompilados para cripto:**
   - SHA256, Keccak256 optimizados
   - Secp256k1, Ed25519 signatures
   - BN254 pairings
   - Mucho más rápido que implementación naive en RISC-V
2. **Recursión eficiente:**
   - Proofs se pueden verificar dentro de SP1
   - Permite proof aggregation
   - Ideal para rollups y bridges
3. **Compatibilidad Rust:**
   - Compila Rust estándar (con restricciones)
   - Crates de crates.io funcionan
   - Desarrollo similar a programación normal

Performance (2024–2025):

- 10–100x más rápido que alternativas para ciertos programas
- Proving distribuido para programas grandes
- Costo: centavos a dólares por proof

Ejemplo: Programa SP1

```rust
// program/src/main.rs
#![no_main]
sp1_zkvm::entrypoint!(main);

use sha2::{Sha256, Digest};

pub fn main() {
    // Leer input
    let preimage = sp1_zkvm::io::read::<Vec<u8>>();

    // Calcular hash (usando precompilado optimizado)
    let mut hasher = Sha256::new();
    hasher.update(&preimage);
    let hash = hasher.finalize();

    // Commit al resultado
    sp1_zkvm::io::commit(&hash.to_vec());
}

// script/src/main.rs (host)
use sp1_sdk::{ProverClient, SP1Stdin};

fn main() {
    // Setup
    let client = ProverClient::new();
    let mut stdin = SP1Stdin::new();

    // Input
    let preimage = b''hello world''.to_vec();
    stdin.write(&preimage);

    // Probar
    let (pk, vk) = client.setup(ELF);
    let proof = client.prove(&pk, stdin).unwrap();

    // Verificar
    client.verify(&proof, &vk).unwrap();

    // Verificar on-chain (Solidity)
    let solidity_proof = proof.solidity_proof();
}
```

2.8.4 Otros zkVMs

Otros proyectos: Valida, Jolt, Nexus.

Observación

Panorama de zkVMs (2024–2025):

Proyecto	ISA	Proof System	Estado
RISC Zero	RISC-V	STARK	Producción
SP1	RISC-V	STARK + SNARK	Producción
Valida	Propio	STARK	Desarrollo
Jolt	RISC-V	Lasso lookups	Desarrollo
Nexus	RISC-V	Folding	Desarrollo
Powdr	Multi-ISA	STARK	Desarrollo

Tendencias:

- RISC-V dominando como ISA
- STARKs para proving principal
- SNARKs (Groth16) para verificación on-chain
- Precompilados para operaciones costosas

2.8.5 Aplicaciones de zkVM

Ejemplo: Casos de uso de zkVM

1. Bridges verificables:

```
# Probar que bloque de Bitcoin es valido
# (verificar proof-of-work, transacciones, etc.)
bitcoin_block = ...
proof = zkvm.prove(validate_bitcoin_block, bitcoin_block)

# Verificar en Ethereum
ethereum_bridge.submit_block(block_header, proof)
```

2. ZK Coprocessors:

```
# Ejecutar computo costoso off-chain, verificar on-chain
# Ejemplo: calcular resultado de subasta compleja

bids = fetch_bids_from_chain()
winners, proof = zkvm.prove(calculate_auction_result, bids)

auction_contract.finalize(winners, proof)
# Costo: ~200K gas (verificar proof)
# vs millones de gas ejecutando on-chain
```

3. Rollups con lógica compleja:

```
# zkVM permite rollups con logica arbitraria
# No limitado a EVM

txs = batch_transactions()
new_state = apply_transactions(current_state, txs)
proof = zkvm.prove(state_transition, (current_state, txs,
    new_state))

rollup_contract.submit_batch(new_state_root, proof)
```

4. Verificación de ML inference:

```
# Probar que un modelo de ML produjo cierto output
model_weights = ...
input_data = ...
output, proof = zkvm.prove(ml_inference, (model_weights,
    input_data))

# Verificar que output es correcto para el modelo dado
verify(proof, model_hash, input_hash, output)
```

🏛 Aplicación en Banco ZK

zkVM en la arquitectura de Banco ZK:
Rol actual:
Por ahora usamos circuitos Circom específicos porque:

- Circuitos más pequeños y eficientes para nuestro caso
- Mejor control sobre constraints exactos
- Proving más rápido para operaciones específicas

Futuro potencial con zkVM:
1. Compliance logic:

```
# Logica de compliance puede ser compleja
# zkVM permite escribirla en Rust normal
def check_compliance(tx):
    # Verificar contra listas de sanciones
    # Aplicar reglas de AML
    # Verificar limites de transaccion
    # Todo verificable en ZK
    pass
```

2. Bridges con otras chains:

```
# Verificar estado de otras blockchains
# Usar zkVM para validar headers/proofs de otras chains
def verify_external_deposit(chain_proof):
    # Verificar consensus de otra chain
    # Validar transaccion de deposito
    # Retornar resultado verificable
    pass
```

3. Extensibilidad:

```
# Usuarios avanzados podrian definir logica custom
# Ejecutada en zkVM, verificada por el protocolo
def custom_spending_condition(tx):
    # Logica personalizada
    pass
```

2.9 Resumen del Capítulo

Idea Clave

Puntos clave de Infraestructura ZK:

1. **Relayers:**
   - Envían txs en nombre del usuario
   - Esenciales para privacidad (evitan link por gas)
   - Modelo económico basado en fees
2. **Mixers:**
   - Tornado Cash: depósito → espera → retiro con ZK proof
   - Nullifiers previenen doble gasto
   - Anonymity set crece con más usuarios
3. **zkEVM:**
   - Tipos 1–4: trade-off compatibilidad vs eficiencia
   - En producción: Scroll, Polygon, zkSync, StarkNet
   - Proving sigue siendo costoso (minutos–horas)
4. **zkVM:**
   - Propósito general (RISC-V típicamente)
   - RISC Zero, SP1 en producción
   - Permite probar cualquier programa Rust/C
   - Aplicaciones: bridges, coprocessors, ML

Chapter 3

Privacidad Avanzada

3.1 Ring Signatures

Definición: Ring Signature

Firma que prueba que uno de n miembros del "ring" firmó, sin revelar cuál.

Propiedades:

- Anonymity: imposible saber quién firmó
- Unforgeability: solo miembros pueden firmar
- Spontaneous: no requiere setup entre miembros

3.1.1 Historia y Contexto

Observación

Evolución de Ring Signatures:

Año	Desarrollo	Autores
2001	Ring signatures	Rivest, Shamir, Tauman
2004	Linkable ring signatures	Liu, Wei, Wong
2012	CryptoNote (Monero base)	van Saberhagen
2015	Ring Confidential Transactions	Shen Noether
2019	CLSAG (Monero actual)	Goodell et al.
2020	Triptych	Sarang Noether

El nombre "ring signature" viene de la estructura circular del algoritmo original, donde las firmas de los miembros se "encadenan" en un anillo.

3.1.2 Intuición

Intuición

Analogía: La mesa redonda anónima
Imagina una mesa redonda con 10 personas. Una de ellas escribe un mensaje secreto y lo pasa al siguiente. Cada persona puede:

- Ser el autor original (solo 1 persona)

- O simplemente pasar el mensaje modificándolo un poco

Al final, el mensaje vuelve al inicio. Un observador ve que:

- El mensaje es "válido" (la cadena cierra correctamente)
- Alguien en la mesa lo escribió
- Pero no puede saber quién fue el autor original

En términos criptográficos:

- Las 10 personas son las claves públicas del "ring"
- El mensaje es lo que se firma
- El autor real usa su clave privada para "iniciar" el anillo
- Los demás son "señuelos" que añaden plausibilidad

3.1.3 Construcción Matemática

Nota Matemática

Ring Signature basada en Schnorr (simplificada):
Setup:

- Ring de n claves públicas: $P_0, P_1, \ldots, P_{n-1}$
- El firmante conoce la clave privada x_π tal que $P_\pi = x_\pi G$
- Mensaje a firmar: m

Signing (el firmante es el índice π):

1. Elegir $\alpha \xleftarrow{\$} \mathbb{Z}_q$ aleatorio
2. Para $i \neq \pi$, elegir $c_i, s_i \xleftarrow{\$} \mathbb{Z}_q$ aleatorios
3. Calcular:
$$L_\pi = \alpha G$$
$$L_i = s_i G + c_i P_i \quad \text{para } i \neq \pi$$
4. Calcular el challenge global:
$$c = H(m \| L_0 \| L_1 \| \ldots \| L_{n-1})$$
5. Calcular c_π:
$$c_\pi = c - \sum_{i \neq \pi} c_i \mod q$$
6. Calcular s_π:
$$s_\pi = \alpha - c_\pi \cdot x_\pi \mod q$$
7. La firma es: $\sigma = (c, s_0, s_1, \ldots, s_{n-1})$

Verification:

1. Para cada i, calcular:
$$L'_i = s_i G + c_i P_i$$
donde c_i se deriva de la descomposición de c

2. Verificar:
$$c \stackrel{?}{=} H(m \| L'_0 \| L'_1 \| \dots \| L'_{n-1})$$

Por qué funciona:
Para el firmante real π:

$$L_\pi = s_\pi G + c_\pi P_\pi = (\alpha - c_\pi x_\pi)G + c_\pi (x_\pi G) = \alpha G$$

Para los demás: L_i es aleatorio (elegimos c_i, s_i al azar).
La firma "cierra" correctamente solo si alguien en el ring conoce una clave privada.

3.1.4 Propiedades de Seguridad

Idea Clave

Propiedades formales:
1. Anonimato (Anonymity):
Dado un ring $\{P_0, \dots, P_{n-1}\}$ y una firma válida σ, la probabilidad de adivinar cuál P_i es el firmante real es:
$$\Pr[\text{adivinar } \pi] = \frac{1}{n}$$
Esto se cumple incluso si el adversario conoce todas las claves privadas excepto una.
2. Infalsificabilidad (Unforgeability):
Un adversario que no conoce ninguna clave privada del ring no puede producir una firma válida, excepto con probabilidad negligible.
3. Espontaneidad (Spontaneous):
El firmante puede crear un ring incluyendo claves públicas de cualquier persona, sin su conocimiento o cooperación.
Esta propiedad es crucial: no hay "setup" previo entre los miembros del ring.

3.1.5 Tamaño y Eficiencia

Observación

Trade-offs de ring signatures:

Ring Size	Tamaño Firma	Anonimato
2	∼200 bytes	50%
11 (Monero)	∼1.5 KB	91%
100	∼6.4 KB	99%
1000	∼64 KB	99.9%

Problema: El tamaño crece linealmente con el ring size.
Soluciones modernas:

- **CLSAG:** Optimización que reduce tamaño ∼25%

- **Triptych:** Tamaño logarítmico $O(\log n)$
- **Seraphis (futuro Monero):** Mejor eficiencia

3.1.6 Linkable Ring Signatures

Definición: Linkable Ring Signature

Ring signature que permite detectar si la misma clave firmó dos veces.

- Cada firma incluye "key image" $I = x \cdot H_p(P)$
- Key image es único por clave privada
- Permite detectar doble gasto
- Usado en Monero

Por qué Necesitamos Linkability

Intuición

El problema del doble gasto con ring signatures:
Sin linkability:

- Alice tiene una moneda (output) con clave P_A
- Firma con ring $\{P_A, P_B, P_C\}$ para gastar $\rightarrow$ OK
- Firma OTRA VEZ con ring $\{P_A, P_D, P_E\}$ $\rightarrow$ ¿Cómo detectar?
- Las firmas son válidas y no revelan que P_A es el firmante real
- ¡Doble gasto!

Con linkability (key images):

- Cada firma incluye key image $I = x_A \cdot H_p(P_A)$
- I es determinístico para x_A
- Si Alice firma dos veces, ambas firmas tienen el mismo I
- Sistema rechaza la segunda firma
- Pero observador no sabe que I corresponde a P_A

Construcción de Key Images

Nota Matemática

Key Image en Monero:
Para clave privada x y clave pública $P = xG$:

$$I = x \cdot H_p(P)$$

Donde H_p es un hash-to-point (mapea P a un punto de la curva).
Propiedades:
1. Determinístico:
Para la misma clave (x, P), siempre produce el mismo I.

$$I_1 = x \cdot H_p(P) = I_2$$

2. No vinculable:
Dado I, no se puede determinar P sin conocer x.

- $I = x \cdot H_p(P)$
- $H_p(P)$ es un punto "aleatorio" desde la perspectiva del observador
- Sin x, no puede verificar si I corresponde a un P específico

3. Único:
Diferentes claves producen diferentes key images (con probabilidad abrumadora).

- Si $P_1 \neq P_2$, entonces $H_p(P_1) \neq H_p(P_2)$
- Por lo tanto $I_1 = x_1 H_p(P_1) \neq x_2 H_p(P_2) = I_2$

CLSAG (Monero Actual)

Ejemplo: CLSAG - Concise Linkable Spontaneous Anonymous Group Signatures

```
Estructura de firma CLSAG:

Ring: {P_0, P_1, ..., P_{n-1}}  // Claves p\'ublicas
Commitment: {C_0, C_1, ..., C_{n-1}}  // Pedersen commitments

Firma:
    I        // Key image (32 bytes)
    c_0      // Challenge inicial (32 bytes)
    s_0, s_1, ..., s_{n-1}  // Respuestas (32 bytes cada una)
    D        // Commitment auxiliar (32 bytes)

Tama\~no total: 32 * (n + 3) bytes
Para ring size 11: 32 * 14 = 448 bytes (por input)

Mejora vs MLSAG anterior: ~25% m\'as peque\~no
```

Verificación:

1. Reconstruir todos los L_i y R_i a partir de c_0 y los s_i
2. Verificar que el último c_n coincide con c_0 (cierra el anillo)
3. Verificar que I no está en la lista de key images usados

3.1.7 Ring Signatures vs Otras Técnicas

Propiedad	Ring Sig	ZK Proof	Mixer
Anonimato	Ring size	Anonymity set	Anonymity set
Tamaño	$O(n)$	$O(1)$	N/A
Setup	Ninguno	Posible trusted	Depósito previo
Interactividad	No	No	Sí (2 txs)
Doble gasto	Key images	Nullifiers	Nullifiers

🏛 Aplicación en Banco ZK

Ring Signatures en Banco ZK:
¿Por qué NO usamos ring signatures?

- Tamaño lineal en ring size (ineficiente para L2)
- Preferimos ZK proofs con anonymity set completo
- Nullifiers logran lo mismo que key images

Sin embargo, conceptos relacionados que usamos:

- **Key images → Nullifiers:** Mismo concepto, diferente implementación
- **Anonimato de conjunto:** Nuestro anonymity set es TODO el Merkle tree
- **No-linkability:** Nullifiers no revelan qué commitment gastan

Comparación:

```
Monero (Ring Signatures):
    - Ring size: 16 (fijo)
    - Anonimato: 1/16 = 6.25% de probabilidad de link
    - Tama\~no tx: ~2-3 KB

Banco ZK (ZK Proofs):
    - Anonymity set: TODO el tree (millones)
    - Anonimato: 1/n donde n = total de deposits
    - Tama\~no tx: ~200-500 bytes (proof)
```

3.2 Stealth Addresses

Definición: Stealth Address

Dirección de un solo uso generada por el sender para el receiver.

3.2.1 Motivación

Intuición

El problema de las direcciones reutilizadas:
En Ethereum normal:

- Alice publica su dirección: `0xAlice...`
- Bob le envía 1 ETH
- Carol le envía 2 ETH
- David le envía 0.5 ETH

Cualquiera puede ver:

- Todos los pagos a Alice
- El balance total de Alice
- Potencialmente, quiénes le pagan

Con stealth addresses:

- Cada pago va a una dirección DIFERENTE
- Las direcciones no están vinculadas públicamente
- Solo Alice puede identificar cuáles son suyas

3.2.2 Protocolo DKSAP

Protocolo:

1. Receiver publica meta-address: $(A, B) = (aG, bG)$
2. Sender genera r aleatorio
3. Calcula stealth address: $P = H(rA)G + B$
4. Publica $R = rG$ junto con la tx
5. Receiver escanea: $P' = H(aR)G + B$
6. Si $P = P'$, puede gastar con $sk = H(aR) + b$

Nota Matemática

Matemáticas del protocolo DKSAP:
(Dual-Key Stealth Address Protocol)
Setup del Receiver:

- Spending key: $b \xleftarrow{\$} \mathbb{Z}_q$
- Viewing key: $a \xleftarrow{\$} \mathbb{Z}_q$
- Meta-address pública: $(A, B) = (aG, bG)$

Envío por el Sender:

1. Generar efímero: $r \xleftarrow{\$} \mathbb{Z}_q$
2. Shared secret: $s = rA = raG$ (sender calcula)
3. Stealth address: $P = H(s)G + B$
4. Ephemeral pubkey: $R = rG$
5. Enviar fondos a P, publicar R

Escaneo por el Receiver:

1. Para cada tx con ephemeral R:
2. Calcular shared secret: $s' = aR = arG = raG = s$
3. Calcular stealth: $P' = H(s')G + B$
4. Si $P' = P$ (dirección en la tx), es para mí

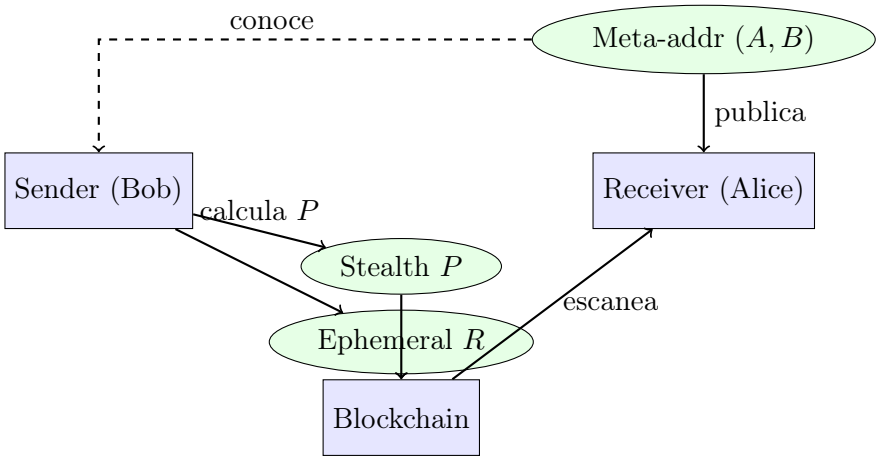
Gasto por el Receiver:

- Clave privada del stealth: $p = H(s) + b$
- Verificación: $pG = H(s)G + bG = P$ ✓

Por qué funciona:

$$\text{Sender calcula: } s = r \cdot A = r \cdot aG$$
$$\text{Receiver calcula: } s' = a \cdot R = a \cdot rG = raG = s$$

Ambos llegan al mismo shared secret sin revelarlo públicamente.



3.2.3 Proyectos e Implementaciones

Observación

Stealth addresses en la práctica:
Umbra Protocol (Ethereum):

- Implementación ERC-5564
- Registro on-chain de meta-addresses
- Soporta ETH y tokens ERC-20
- Fee mínimo para operación

EIP-5564 (Stealth Address Standard):

- Estándar propuesto para Ethereum
- Define formato de meta-addresses
- Interoperabilidad entre wallets

Desafíos actuales:

- **Escaneo:** Receiver debe revisar TODAS las txs
- **Gas para gastar:** Stealth address empieza con 0 ETH
- **Compatibilidad:** Wallets deben soportar el estándar

3.2.4 Variantes de Stealth Addresses

Ejemplo: Tipos de stealth addresses

1. Single-key (básico):

```
Meta-address: B = bG (solo spending key)
Stealth: P = H(rB)G + B
Problema: no hay viewing key separada
```

2. Dual-key (DKSAP):

```
Meta-address: (A, B) = (aG, bG)
Stealth: P = H(rA)G + B
Ventaja: viewing key (a) separada de spending key (b)
```

3. Con viewing key delegable:

```
Compartir 'a' con tercero
$\to$ Puede ver todas las txs entrantes
$\to$ No puede gastar (no tiene 'b')
$\to$ \'Util para contadores, auditores
```

3.3 Viewing Keys

Definición: Viewing Key

Clave que permite ver transacciones sin poder gastar.

Usos:

- Dar a contador para impuestos
- Dar a auditor para compliance
- Dar a exchange para verificar depósitos
- Spending key siempre privada

3.3.1 Tipos de Viewing Keys

Nota Matemática

Granularidad de viewing keys:

1. **Full Viewing Key:**
   - Ver TODAS las transacciones (entrantes y salientes)
   - Conocer balances exactos
   - Ver destinatarios de pagos salientes
2. **Incoming Viewing Key:**
   - Ver solo transacciones ENTRANTES
   - Útil para exchanges: verificar depósitos
   - No revela gastos del usuario
3. **Outgoing Viewing Key:**
   - Ver solo transacciones SALIENTES
   - Menos común
   - Útil para auditoría de gastos
4. **Per-transaction Viewing Key:**
   - Revelar UNA transacción específica
   - Máxima granularidad
   - Probar un pago sin revelar historial

3.3.2 Implementaciones

Ejemplo: Viewing keys en Zcash

```
Jerarqu\'ia de claves en Zcash:

spending_key (sk)
    |
    |---- ask (spend authorizing key)
    |   \---- ak = ask * G (verificaci\'on de gasto)
    |
    |---- nsk (nullifier private key)
    |   \---- nk = nsk * G (generaci\'on de nullifiers)
    |
    \---- ovk (outgoing viewing key)
        \---- Para ver txs salientes

full_viewing_key (fvk) = (ak, nk, ovk)
    $\to$ Puede ver todas las txs
    $\to$ No puede gastar (no tiene ask)

incoming_viewing_key (ivk) = derivado de (ak, nk)
    $\to$ Solo puede ver txs entrantes
    $\to$ No puede ver salientes ni gastar
```

3.3.3 Casos de Uso

Idea Clave

Escenarios prácticos:
1. Impuestos:

```
Usuario genera viewing key
    $\to$ Da al contador
    $\to$ Contador puede ver historial para calcular impuestos
    $\to$ Contador NO puede gastar fondos
    $\to$ Usuario mantiene privacidad de spending key
```

2. Compliance empresarial:

```
Empresa tiene wallet corporativo
    $\to$ CFO tiene spending key
    $\to$ Auditor externo recibe viewing key
    $\to$ Auditor verifica flujos sin riesgo de robo
    $\to$ Cumple con regulaciones financieras
```

3. Exchanges (incoming only):

```
Usuario quiere depositar en exchange
    $\to$ Exchange da direcci\'on de dep\'osito
    $\to$ Exchange tiene incoming viewing key
    $\to$ Puede verificar dep\'ositos autom\'aticamente
    $\to$ No puede ver otros gastos del usuario
```

4. Herencia/Recovery:

```
Usuario crea viewing key para familiar
    $\to$ Familiar puede monitorear actividad
    $\to$ Si usuario fallece, familiar sabe qu\'e hay
    $\to$ Spending key guardada en testamento seguro
```

3.3.4 Viewing Keys vs ZK Disclosure

Observación

Dos enfoques para revelación selectiva:
Viewing Keys:

- Revelan TODO (o categoría completa)
- Una vez compartida, acceso permanente
- Más simple de implementar
- Requiere confiar en el receptor

ZK Proofs de disclosure:

- Revelan EXACTAMENTE lo necesario
- Por transacción o afirmación
- Más complejo pero más preciso
- "Pruebo que pagué, sin revelar cuánto"

Ejemplo comparativo:

```
Escenario: Probar que pagu\'e alquiler de $1000

Con viewing key:
    $\to$ Propietario ve TODAS mis transacciones
    $\to$ Ve mi salario, otros gastos, etc.
    $\to$ Mucha informaci\'on extra revelada

Con ZK proof:
    $\to$ Genero prueba: ''tengo tx de $1000 a address X en fecha Y''
    $\to$ Propietario verifica la prueba
    $\to$ No ve nada m\'as de mi historial
    $\to$ Privacidad preservada
```

Aplicación en Banco ZK

Viewing Keys en Banco ZK:
Diseño de claves:

```
master_key
    |
```

```
    |---- spending_key
    |   $\to$ Autoriza transacciones
    |   $\to$ NUNCA se comparte
    |
    |---- full_viewing_key
    |   $\to$ Ver todas las transacciones
    |   $\to$ Para auditor\'ia completa
    |
    |---- incoming_viewing_key
    |   $\to$ Solo dep\'ositos entrantes
    |   $\to$ Para exchanges/servicios
    |
    \---- proof_generation_key
        $\to$ Generar ZK proofs
        $\to$ Sin capacidad de gasto
```

Casos de uso soportados:

- **Compliance:** Usuario puede probar origen de fondos a regulador
- **Impuestos:** Exportar historial para declaración
- **Auditoría:** Empresas pueden dar viewing keys a auditores
- **Recuperación:** Viewing key en backup, spending key en hardware wallet

Implementación técnica:

```
// Derivaci\'on de viewing keys
spending_key = random()
viewing_key = derive_viewing(spending_key)

// Escaneo de transacciones
for commitment in merkle_tree:
    if can_decrypt(commitment, viewing_key):
        // Es m\'ia, extraer detalles
        (amount, sender, memo) = decrypt(commitment, viewing_key)

// Generar prueba de propiedad
proof = prove_ownership(commitment, viewing_key)
// Prueba: ''puedo ver este commitment'' sin revelar spending key
```

3.4 Mixnets

Definición: Mixnet

Red de servidores que mezclan mensajes en capas para anonimato.

3.4.1 Historia y Contexto

Observación

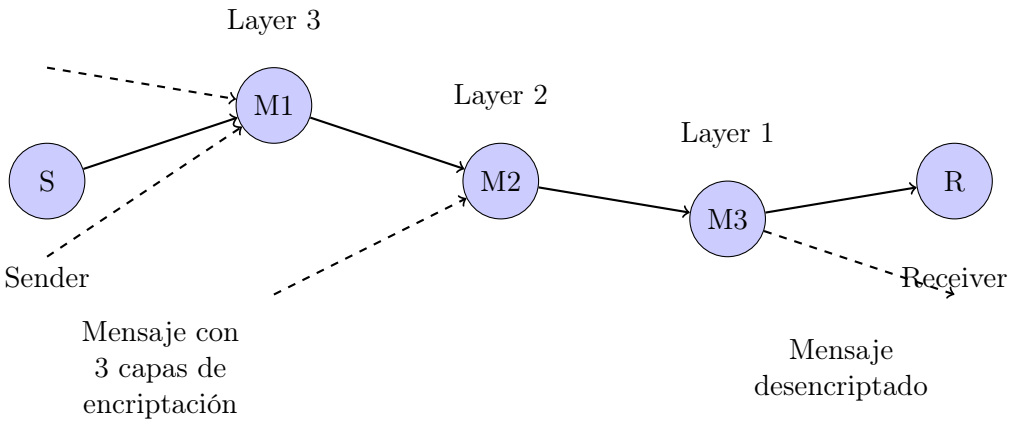
Evolución de las redes de anonimato:

Año	Tecnología	Innovación
1981	Mix networks (Chaum)	Concepto original
1996	Onion routing	Capas de encriptación
2002	Tor	Implementación práctica
2003	I2P	Red interna anónima
2018	Nym	Mixnet con incentivos crypto

David Chaum (1981) inventó las mix networks como solución al problema de comunicación anónima. La idea: si muchos mensajes pasan por un "mezclador", es difícil correlacionar entrada con salida.

3.4.2 Arquitectura

1. Mensaje encriptado en múltiples capas
2. Cada mix node remueve una capa y reordena
3. Delays aleatorios en cada hop
4. Cover traffic para resistir timing analysis



Nota Matemática

Encriptación en capas (Sphinx):
Preparación del mensaje:
Para enviar mensaje m a través de nodos N_1, N_2, N_3:

```
// Claves p\'ublicas de los mix nodes
pk_1, pk_2, pk_3 = public_keys_of_route

// Generar claves de sesi\'on para cada hop
session_1 = ECDH(ephemeral, pk_1)
session_2 = ECDH(ephemeral, pk_2)
session_3 = ECDH(ephemeral, pk_3)
```

```
// Encriptar en capas (de adentro hacia afuera)
layer_3 = Enc(session_3, m || recipient)
layer_2 = Enc(session_2, layer_3 || next_hop_3)
layer_1 = Enc(session_1, layer_2 || next_hop_2)

// Enviar layer_1 al primer nodo
```

Procesamiento en cada nodo:

```
// Nodo N_i recibe paquete encriptado
function process(encrypted_packet):
    // Derivar clave de sesi\'on
    session = ECDH(my_private_key, ephemeral_from_packet)

    // Desencriptar una capa
    (inner_layer, next_hop) = Dec(session, encrypted_packet)

    // A\~nadir delay aleatorio
    delay(random_milliseconds())

    // Reordenar con otros paquetes (batch)
    add_to_batch(inner_layer, next_hop)

    // Enviar cuando batch est\'a listo
    send_batch()
```

3.4.3 Cover Traffic

Idea Clave

¿Por qué cover traffic?
Problema: Timing attacks
Sin cover traffic:

- Adversario observa: mensaje entra a las 10:00:00.000
- Observa: mensaje sale a las 10:00:00.150 (150ms delay)
- Correlaciona por timing, incluso sin ver contenido

Solución: Cover traffic (mensajes dummy)
Con cover traffic:

- Red genera mensajes falsos constantemente
- Todos los mensajes lucen iguales (mismo tamaño)
- Adversario no puede distinguir real de dummy
- Timing analysis mucho más difícil

Trade-off:

- Más cover traffic → Mejor privacidad

- Más cover traffic → Mayor costo (ancho de banda)
- Nym incentiva cover traffic con tokens

3.4.4 Nym Network

- Mixnet moderno con incentivos crypto
- NYM token para pagar y stakear
- Cover traffic automático
- SDK para aplicaciones
- Mejor resistencia a timing que Tor

Ejemplo: Arquitectura Nym

```
Componentes de Nym:

1. Mix Nodes (3 capas)
   |---- Layer 1: Recibe del cliente
   |---- Layer 2: Intermediario
   \---- Layer 3: Env\'ia al gateway de salida

2. Gateways
   |---- Entry Gateway: Conexi\'on de clientes
   \---- Exit Gateway: Conexi\'on a internet/servicios

3. Validators (Cosmos SDK)
   |---- Consensus sobre estado de la red
   |---- Distribuci\'on de recompensas
   \---- Slashing de nodos maliciosos

4. NYM Token
   |---- Staking en mix nodes
   |---- Pago por uso de la red
   \---- Governance

Flujo de un mensaje:
Client $\to$ Entry Gateway $\to$ Mix L1 $\to$ Mix L2 $\to$ Mix L3
      $\to$ Exit Gateway $\to$ Destination

Incentivos:
- Mix nodes reciben NYM por procesar tr\'afico
- Cover traffic cuenta para recompensas
- Usuarios pagan por ancho de banda
```

3.4.5 Mixnets vs Tor

Propiedad	Mixnet (Nym)	Tor
Latencia	Alta (segundos)	Baja (ms)
Resistencia timing	Alta	Media
Cover traffic	Sí	No
Modelo de incentivos	Crypto tokens	Voluntarios
Decentralización	Mayor	Media
Uso típico	Mensajería	Navegación

3.5 Tor e I2P

3.5.1 Tor (The Onion Router)

- 3 nodos: guard, middle, exit
- Baja latencia (segundos)
- Vulnerable a timing attacks
- Onion services (.onion)

Arquitectura de Tor

Nota Matemática

Cómo funciona Tor:
Circuito Tor (3 hops):

```
Usuario $\to$ Guard Node $\to$ Middle Node $\to$ Exit Node $\to$ Destino
          (fijo)       (rotativo)    (rotativo)
```

Guard Node:

- Primer punto de contacto
- Se mantiene fijo por meses
- Razón: evitar ataques de rotación de guards
- Conoce la IP del usuario

Middle Node:

- Intermediario
- Solo sabe: viene del guard, va al exit
- No conoce origen ni destino real

Exit Node:

- Conexión al destino final
- Ve el tráfico (si no es HTTPS)
- No conoce el origen (usuario)
- Punto más "peligroso" para operadores

Onion Services

Ejemplo: Onion Services (.onion)

```
Servicio tradicional:
    Usuario $\to$ Tor $\to$ Exit Node $\to$ servidor.com

    Problema: Exit node ve la conexi\'on al servidor
    Problema: Servidor ve IP del exit node

Onion Service:
    Usuario $\to$ Tor $\to$ Rendezvous Point $\leftarrow$ Tor $\leftarrow$ Servicio .onion

    Ventajas:
    - No hay exit node (todo dentro de Tor)
    - Servidor tambi\'en es an\'onimo
    - Comunicaci\'on end-to-end encriptada

Direcciones .onion:
    - v2 (deprecated): 16 caracteres
    - v3 (actual): 56 caracteres
```

Integración con Crypto

Observación

Usos de Tor en cryptocurrency:
1. Conexión a nodos:

```
# Bitcoin Core con Tor
bitcoind -proxy=127.0.0.1:9050

# Monero con Tor
monerod --tx-proxy tor,127.0.0.1:9050
```

2. Wallets que usan Tor:

- Wasabi Wallet (Bitcoin, Tor por defecto)
- Samourai Wallet (Bitcoin, Tor opcional)
- Electrum (Tor opcional)

3. Relayers como onion services:

```
# Tornado Cash ten\'ia relayers en .onion
# Permite submittir proofs sin revelar IP
relayer.onion/submit_proof
```

4. Block explorers:

- Blockchair tiene versión .onion
- Mempool.space tiene .onion
- Consultar sin revelar qué addresses te interesan

3.5.2 I2P (Invisible Internet Project)

- Garlic routing: múltiples mensajes juntos
- Red interna (no exit nodes por defecto)
- Mayor latencia
- Mejor para servicios dentro de I2P

Nota Matemática

Diferencias clave I2P vs Tor:
Modelo de red:

- Tor: Principalmente para acceder a internet normal
- I2P: Diseñado como red interna (“darknet”)

Garlic routing:

```
Tor (Onion): Cada mensaje se encripta individualmente
    Msg1 $\to$ [Enc(Enc(Enc(Msg1)))]
    Msg2 $\to$ [Enc(Enc(Enc(Msg2)))]
```

```
I2P (Garlic): M\'ultiples mensajes se encriptan juntos
    Msg1 + Msg2 + Msg3 $\to$ [Enc(Enc(Enc(Msg1, Msg2, Msg3)))]

Ventaja: M\'as dif\'icil correlacionar mensajes individuales
```

Túneles unidireccionales:

```
Tor: Circuito bidireccional
    A $\leftarrow$$\to$ Guard $\leftarrow$$\to$ Middle $\leftarrow$$\to$ Exit $\leftarrow$$

I2P: T\'uneles separados para entrada/salida
    A $\to$ Outbound tunnel $\to$ B
    B $\to$ Outbound tunnel $\to$ A

Ventaja: Compromiso de un t\'unel no afecta al otro
```

Ejemplo: I2P en crypto

```
Monero tiene soporte nativo para I2P:

# Configuraci\'on de Monero con I2P
monerod --tx-proxy i2p,127.0.0.1:4447

Kovri (proyecto relacionado):
- Router I2P en C++ para Monero
- Integraci\'on nativa con el daemon
- Objetivo: Monero completamente sobre I2P

Servicios I2P (.i2p):
- Exploradores de bloques
- Pools de miner\'ia
- Nodos seed
```

3.5.3 Comparación de Tecnologías de Anonimato de Red

Aspecto	Tor	I2P	Nym	VPN
Latencia	Baja	Media	Alta	Muy baja
Timing resistance	Baja	Media	Alta	Ninguna
Decentralización	Media	Alta	Alta	Ninguna
Exit nodes	Sí	Opcional	Sí	Sí
Incentivos	Ninguno	Ninguno	Crypto	Pago
Madurez	Alta	Media	Baja	Alta

Aplicación en Banco ZK

Integración de red anónima en Banco ZK:
Capas de privacidad:

```
Capa de aplicaci\'on: ZK proofs (privacidad de datos)
Capa de red: Tor/Nym (privacidad de IP)
```

```
Sin protecci\'on de red:
    $\to$ Observador ve: IP X consult\'o balance de cuenta Y
    $\to$ Aunque balance est\'a encriptado, se filtra metadata

Con protecci\'on de red:
    $\to$ Observador ve: nodo Tor consult\'o algo
    $\to$ No puede vincular IP con cuenta
```

Recomendaciones para usuarios:

- **Mínimo:** Usar Tor para consultas de balance
- **Recomendado:** Tor para todas las interacciones
- **Paranoico:** Nym para resistencia a timing

Infraestructura:

- Relayers disponibles como onion services
- Nodos de la red accesibles vía Tor
- SDK con soporte para SOCKS5 proxy

Código ejemplo:

```
// Configurar cliente con Tor
const client = new BancoZKClient({
    rpcUrl: ''http://localhost:8545'',
    proxy: {
        type: ''socks5'',
        host: ''127.0.0.1'',
        port: 9050  // Tor SOCKS proxy
    }
});

// O usar relayer .onion
const relayer = ''http://bancozk...onion/relay'';
await client.submitViaRelayer(proof, relayer);
```

3.6 MPC (Multi-Party Computation)

Definición: MPC

Protocolo donde múltiples partes computan una función sin revelar sus inputs.

$$f(x_1, x_2, \ldots, x_n) \to (y_1, y_2, \ldots, y_n)$$

Cada parte i conoce solo x_i y aprende solo y_i.

3.6.1 Historia y Motivación

Observación

Historia de MPC:

Año	Desarrollo	Autores
1982	Millionaires' Problem	Andrew Yao
1986	Garbled Circuits	Yao
1987	BGW Protocol	Ben-Or, Goldwasser, Wigderson
1988	GMW Protocol	Goldreich, Micali, Wigderson
2004	Fairplay (primera impl.)	Malkhi et al.
2009	SPDZ (eficiente)	Damg?rd et al.
2017	MPC en crypto ceremonies	Zcash, Ethereum

Intuición

El problema de los millonarios (Yao, 1982):
Alice y Bob quieren saber quién es más rico, sin revelar sus fortunas exactas.
Requisitos:

- Alice no revela su fortuna a Bob
- Bob no revela su fortuna a Alice
- Al final, ambos saben quién tiene más
- Ninguno aprende la cantidad del otro

Sin MPC:

- Opción 1: Confiar en tercero → Pero ese tercero conoce ambas fortunas
- Opción 2: Revelar → Pierde privacidad
- Opción 3: No calcular → No se obtiene resultado

Con MPC:

- Alice y Bob ejecutan protocolo entre ellos
- Al final: "Alice tiene más" o "Bob tiene más"
- Ninguno aprendió el número del otro
- Sin tercero de confianza

3.6.2 Técnicas Fundamentales

- **Garbled Circuits:** Una parte "garbles", otra evalúa
- **Secret Sharing:** Datos divididos entre partes
- **Oblivious Transfer:** Base de muchos protocolos

Secret Sharing

Nota Matemática

Shamir Secret Sharing (SSS):
Objetivo: Dividir secreto s entre n partes, requiriendo t para reconstruir.
Construcción:

1. Elegir polinomio aleatorio de grado $t-1$:

$$p(x) = s + a_1 x + a_2 x^2 + \ldots + a_{t-1} x^{t-1}$$

   donde $p(0) = s$ (el secreto)

2. Dar a cada parte i el punto $(i, p(i))$

3. Con t puntos, se puede interpolar el polinomio (Lagrange)

4. Con menos de t puntos, el secreto es completamente oculto

Ejemplo (2-de-3):

```
Secreto: s = 42
Polinomio: p(x) = 42 + 7x  (grado 1, necesita 2 puntos)

Shares:
    Parte 1: (1, p(1)) = (1, 49)
    Parte 2: (2, p(2)) = (2, 56)
    Parte 3: (3, p(3)) = (3, 63)

Reconstrucci\'on con partes 1 y 2:
    Interpolaci\'on de Lagrange:
    p(0) = 49 * (0-2)/(1-2) + 56 * (0-1)/(2-1)
         = 49 * 2 + 56 * (-1)
         = 98 - 56 = 42 $\checkmark$
```

Propiedad aditiva:
Si $[a]$ y $[b]$ son shares de a y b:

$$[a] + [b] = [a + b]$$

Cada parte puede sumar localmente sus shares, resultado es share de la suma.

Garbled Circuits

Idea Clave

Garbled Circuits (Yao):
Idea: Convertir un circuito booleano en una versión "encriptada" que se puede evaluar sin conocer los valores intermedios.
Protocolo (2 partes):

1. **Garbler (Alice):**
   - Toma el circuito que computa $f(x, y)$

- Asigna 2 claves aleatorias a cada wire: k_w^0, k_w^1
- Para cada gate, crea tabla encriptada (garbled table)
- Envía: circuito garbled + su input encriptado

2. **Evaluator (Bob):**
   - Obtiene claves para su input via Oblivious Transfer
   - Evalúa el circuito gate por gate
   - Solo puede desencriptar UNA entrada de cada tabla
   - Obtiene claves del output, las envía a Alice
3. **Output:**
   - Alice traduce claves de output a bits reales
   - Ambos aprenden $f(x, y)$

Seguridad:

- Bob no aprende valores intermedios (todo son claves aleatorias)
- Alice no aprende input de Bob (Oblivious Transfer)

Ejemplo: Garbled AND gate

```
Gate AND con wires de entrada A, B y salida C

Claves:
    Wire A: k^0_A = ''key_A0'', k^1_A = ''key_A1''
    Wire B: k^0_B = ''key_B0'', k^1_B = ''key_B1''
    Wire C: k^0_C = ''key_C0'', k^1_C = ''key_C1''

Tabla de verdad AND:
    A=0, B=0 $\to$ C=0
    A=0, B=1 $\to$ C=0
    A=1, B=0 $\to$ C=0
    A=1, B=1 $\to$ C=1

Garbled Table (permutada aleatoriamente):
    Enc(k^0_A, k^0_B, k^0_C)  // 0 AND 0 = 0
    Enc(k^0_A, k^1_B, k^0_C)  // 0 AND 1 = 0
    Enc(k^1_A, k^0_B, k^0_C)  // 1 AND 0 = 0
    Enc(k^1_A, k^1_B, k^1_C)  // 1 AND 1 = 1

Evaluaci\'on (Bob tiene k^1_A y k^0_B):
    - Intenta desencriptar cada fila con sus claves
    - Solo la fila 3 desencripta correctamente
    - Obtiene k^0_C (resultado: C=0, correcto!)
    - No aprende que A=1, B=0, solo obtiene k^0_C
```

Oblivious Transfer (OT)

Nota Matemática

1-de-2 Oblivious Transfer:
Setup:

- Sender tiene dos mensajes: m_0, m_1
- Receiver quiere obtener m_b donde $b \in \{0, 1\}$

Requisitos:

- Receiver obtiene exactamente m_b
- Receiver no aprende m_{1-b}
- Sender no aprende b

Protocolo (basado en DH):

```
1. Sender genera (a, A = g^a) y env\'ia A
2. Receiver:
   - Genera k aleatorio
   - Si b=0: B = g^k
   - Si b=1: B = A * g^k = g^(a+k)
   - Env\'ia B

3. Sender:
   - Calcula k_0 = B^a = g^(ka) (si b=0) o g^(a?+ka) (si b=1)
   - Calcula k_1 = (B/A)^a = g^(ka-a?) (si b=0) o g^(ka) (si b=1)
   - Env\'ia: e_0 = Enc(k_0, m_0), e_1 = Enc(k_1, m_1)

4. Receiver:
   - Calcula k_b = A^k = g^(ak)
   - Desencripta e_b para obtener m_b
   - No puede calcular k_{1-b} (no conoce a)
```

Uso en Garbled Circuits:
Bob usa OT para obtener las claves correspondientes a sus bits de input, sin que Alice sepa cuáles eligió.

3.6.3 Aplicaciones en Crypto

- Trusted setup ceremonies (Powers of Tau)
- Threshold signatures
- Private auctions
- Mixing protocols

Trusted Setup Ceremonies

Ejemplo: MPC en Powers of Tau

```
Objetivo: Generar SRS sin que nadie conozca ? (toxic waste)

Protocolo MPC:
    Participante 1: elige ?_1, calcula g^?_1, pasa al siguiente
    Participante 2: elige ?_2, calcula (g^?_1)^?_2 = g^(?_1*?_2)
    ...
    Participante n: calcula g^(?_1*?_2*...*?_n)

? final = ?_1 * ?_2 * ... * ?_n

Seguridad:
    - Solo se necesita 1 participante honesto
    - Si al menos uno destruy\'o su ?_i, nadie conoce ? completo
    - Verificable: cada contribuci\'on se puede verificar

Ceremonias famosas:
    - Zcash Sprout (2016): 6 participantes
    - Zcash Sapling (2018): 90+ participantes
    - Ethereum KZG (2023): 141,000+ participantes
```

Threshold Signatures

Nota Matemática

Threshold ECDSA/Schnorr:
Objetivo: t-de-n firma donde t partes deben cooperar para firmar.
Sin MPC (multisig):

- Cada parte firma independientemente
- Verificador ve t firmas separadas
- Revela quiénes firmaron

Con MPC (threshold):

- Partes ejecutan protocolo MPC
- Producen UNA firma estándar
- Verificador no puede distinguir de firma normal
- No revela estructura interna

Protocolos:

- GG18, GG20 (threshold ECDSA)
- FROST (threshold Schnorr)
- Usados en: Fireblocks, ZenGo, institutional custody

Ventajas para wallets:

```
Escenario: Wallet 2-de-3

Multisig tradicional:
    - Direcci\'on P2SH especial
    - Transacci\'on m\'as grande
    - Revela que es multisig en la blockchain

Threshold MPC:
    - Direcci\'on normal (P2WPKH)
    - Transacci\'on de tama\~no normal
    - Indistinguible de wallet individual
    - Mejor privacidad
```

Private Auctions

Ejemplo: Subasta privada con MPC

```
Escenario: Subasta de primer precio sellada

Participantes: Alice, Bob, Carol quieren pujar
Requisito: Nadie debe conocer las pujas de otros antes del cierre

Protocolo MPC:
1. Cada participante tiene su puja: x_A, x_B, x_C

2. Ejecutan MPC para calcular:
   f(x_A, x_B, x_C) = (ganador, precio)
   donde ganador = argmax(x_A, x_B, x_C)

3. Output: ''Bob gana con $1000''

4. Nadie aprende:
   - Las pujas perdedoras
   - Por cu\'anto gan\'o Bob (margen)

Variantes:
- Subasta de segundo precio (Vickrey)
- Subastas continuas
- NFT auctions privadas
```

3.6.4 MPC vs ZK: Cuándo Usar Cada Uno

Característica	MPC	ZK
Partes	Múltiples (2+)	Una (prover)
Comunicación	Alta (interactivo)	Baja (una prueba)
Privacidad	Entre las partes	Prover oculta al verifier
Output	Todos aprenden resultado	Verifier aprende validez
Cómputo	Distribuido	Local (prover)
Verificación	No transferible	Transferible (prueba)

Observación

Criterios de decisión:
Usar MPC cuando:

- Múltiples partes tienen inputs privados
- Todas las partes necesitan el output
- No hay un "prover" privilegiado
- Ejemplo: voting, auctions, key generation

Usar ZK cuando:

- Una parte quiere probar algo a otra
- El prover tiene toda la información privada
- La prueba debe ser verificable por terceros
- Ejemplo: transacciones privadas, identity proofs

Combinar MPC + ZK:

- Trusted setup ceremonies (MPC genera, ZK usa)
- Collaborative proving (partes generan prueba juntas)
- Privacy-preserving analytics

Aplicación en Banco ZK

MPC en Banco ZK:
1. Trusted Setup:

- Usamos SRS de ceremonia de Ethereum (141K participantes)
- Máxima confianza posible para KZG
- No hacemos nuestra propia ceremonia (menos participantes = menos seguro)

2. Threshold custody (para tesorería):

```
Admin keys del protocolo:
    - 3-de-5 threshold signature
    - Distribuido entre fundadores/advisors
    - Una firma = decisi\'on colectiva
```

```
    - Sin revelar qui\'enes firmaron espec\'ificamente
```

3. Futuro: Collaborative proving

```
Escenario: Usuario quiere probar solvencia
    - Usuario tiene balance privado
    - Exchange necesita verificar >= $X
    - MPC: ambos aportan inputs
    - Output: ''Usuario tiene suficiente'' (s\'i/no)
    - Sin revelar balance exacto
```

3.7 TEE (Trusted Execution Environments)

Definición: TEE

Hardware que ejecuta código en un enclave aislado y seguro.

3.7.1 Concepto y Motivación

Intuición

El problema de ejecutar código con datos sensibles:
Escenario tradicional:

- Tienes datos sensibles (claves, información médica, etc.)
- Quieres ejecutar un programa sobre esos datos
- Pero no confías en la máquina/operador donde se ejecuta

Riesgos:

- Administrador del servidor puede leer memoria
- Malware puede interceptar datos
- Otros procesos pueden espiar
- Incluso el sistema operativo puede ser malicioso

Solución TEE:

- Hardware crea una "burbuja" segura (enclave)
- Código y datos dentro del enclave están protegidos
- Ni el OS, ni el administrador, ni otros procesos pueden ver dentro
- Solo el CPU tiene acceso

3.7.2 Intel SGX

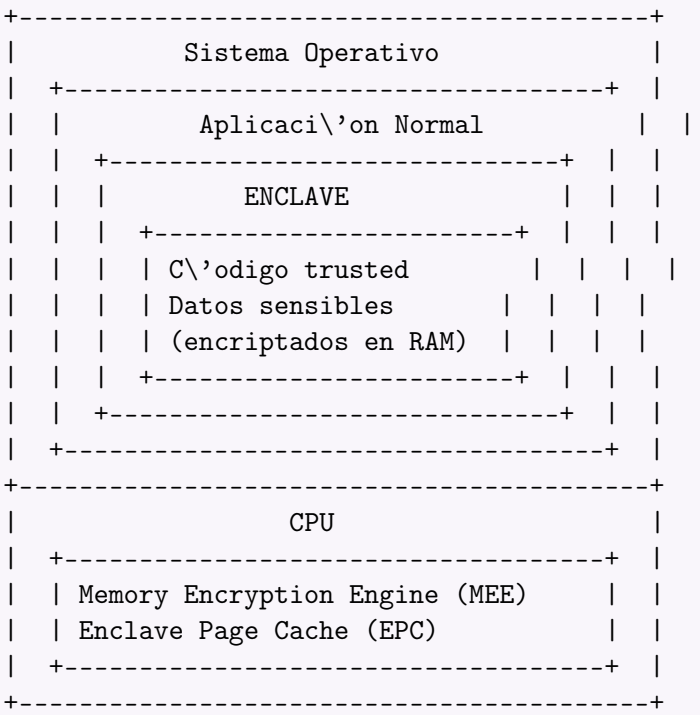
- Enclaves protegidos por CPU
- Memoria encriptada

- Attestation: probar que enclave ejecuta código correcto

- Vulnerabilidades históricas: Spectre, Foreshadow

Nota Matemática

Arquitectura de Intel SGX:
Componentes:



Protecciones:

- **Aislamiento:** Enclave separado del resto del sistema
- **Encriptación:** Datos en RAM están encriptados
- **Integridad:** CPU verifica que código no fue modificado
- **Sealing:** Datos persistentes encriptados con clave del enclave

Flujo de ejecución:

1. Aplicación crea enclave y carga código
2. CPU verifica y mide el código (hash)
3. Aplicación hace "ECALL" para entrar al enclave
4. Código se ejecuta en modo protegido
5. Resultados salen del enclave (sin datos sensibles)

Remote Attestation

Idea Clave

Remote Attestation: Probar qué código se ejecuta
Problema:

- Quiero enviar datos sensibles a un enclave remoto
- ¿Cómo sé que realmente está corriendo el código correcto?
- ¿Cómo sé que es un enclave real y no una simulación?

Solución: Attestation

```
1. Enclave genera ''quote'' firmado por CPU
   quote = {
       MRENCLAVE: hash del c\'odigo del enclave,
       MRSIGNER: identidad del desarrollador,
       user_data: datos adicionales (ej: clave p\'ublica),
       signature: firmado por clave secreta del CPU
   }

2. Quote se env\'ia a Intel Attestation Service (IAS)
   o se verifica localmente con DCAP

3. IAS verifica:
   - Firma es de CPU Intel genuino
   - CPU no est\'a en lista de revocados
   - Retorna certificado de validez

4. Cliente verifica:
   - MRENCLAVE coincide con c\'odigo esperado
   - Certificado de IAS es v\'alido
   - $\to$ Puede confiar y enviar datos sensibles
```

Vulnerabilidades de SGX

Observación

Ataques conocidos contra SGX:
1. Spectre/Meltdown (2018):

- Ataques de ejecución especulativa
- Pueden filtrar datos del enclave
- Mitigaciones en microcódigo, pero con costo de performance

2. Foreshadow/L1TF (2018):

- Ataque específico contra SGX
- Podía leer toda la memoria del enclave
- Requirió actualizaciones de hardware y software

3. **Plundervolt (2019):**
   - Manipular voltaje del CPU
   - Causar errores de cálculo en el enclave
   - Mitigado deshabilitando control de voltaje
4. **SGAxe (2020):**
   - Ataque al sistema de attestation
   - Podía extraer claves de attestation
   - Intel revocó CPUs afectados

Implicación: SGX no es "perfect security". La confianza está en el hardware de Intel y en que no haya vulnerabilidades desconocidas.

3.7.3 ARM TrustZone

- Mundo "seguro" y "normal" separados
- Usado en móviles
- Menos features que SGX

Nota Matemática

Arquitectura de ARM TrustZone:

```
+-------------------+-------------------+
|   Normal World    |   Secure World    |
+-------------------+-------------------+
|  Android/Linux    |   Trusted OS      |
|                   |   (OP-TEE, etc.)  |
|  Apps normales    |   Trusted Apps    |
|                   |   (TAs)           |
+-------------------+-------------------+
|        Monitor Mode (switches)        |
+---------------------------------------+
|              ARM CPU                  |
+---------------------------------------+
```

Diferencias con SGX:

Aspecto	SGX	TrustZone
Modelo	Múltiples enclaves	Dos mundos
Granularidad	Por aplicación	Por sistema
Memoria	Encriptada	Particionada
Attestation	Remoto estándar	Depende de OEM
Plataforma	Servers/PCs	Móviles/IoT

Usos comunes:

- DRM (protección de contenido multimedia)

- Pagos móviles (Apple Pay, Google Pay)
- Biometría (huellas, FaceID)
- Almacenamiento de claves

3.7.4 Otros TEEs

Observación

Alternativas a SGX:
AMD SEV (Secure Encrypted Virtualization):

- Encriptación a nivel de VM
- Protege VMs de hipervisor malicioso
- Usado en cloud computing
- SEV-SNP añade integridad

AWS Nitro Enclaves:

- TEE específico de AWS
- VMs aisladas sin acceso de AWS
- Integrado con servicios AWS
- Attestation propio

Apple Secure Enclave:

- Coprocesador separado en iPhones/Macs
- Maneja biometría y claves
- No programable por desarrolladores externos

RISC-V Keystone:

- TEE open-source para RISC-V
- Alternativa abierta a SGX
- En desarrollo activo

3.7.5 TEE vs ZK

Aspecto	TEE	ZK
Confianza	Hardware	Matemáticas
Velocidad	Rápido	Lento (proving)
Verificación	Attestation	Prueba verificable
Vulnerabilidades	Side-channels	Bugs en circuitos

Idea Clave

Comparación detallada:
Modelo de confianza:

```
TEE:
    Conf\'ias en: Intel/AMD/ARM + que no hay 0-days
    Problema: si se descubre vulnerabilidad, todo expuesto
    Ventaja: performance casi nativo

ZK:
    Conf\'ias en: Matem\'aticas (asumiendo hardness de problema)
    Problema: circuitos complejos pueden tener bugs
    Ventaja: seguridad basada en problemas computacionales
```

Verificabilidad:

```
TEE Attestation:
    - Demuestra qu\'e c\'odigo se ejecuta
    - NO demuestra que el output es correcto
    - Si hay bug en el c\'odigo, output puede ser incorrecto
    - Attestation no detecta esto

ZK Proof:
    - Demuestra que el output es correcto
    - Verifier puede estar 100% seguro del resultado
    - No depende de confiar en la ejecuci\'on
    - Matem\'aticamente verificable
```

Cuándo usar cada uno:

```
TEE mejor para:
    - Procesamiento de grandes vol\'umenes de datos
    - Latencia baja cr\'itica
    - Datos que no salen del enclave
    - Ejemplo: procesamiento de datos de salud

ZK mejor para:
    - Pruebas p\'ublicamente verificables
    - Datos peque\~nos con alto valor
    - Casos donde la verificaci\'on es cr\'itica
    - Ejemplo: transacciones financieras, voting
```

3.7.6 TEE en Blockchain

Ejemplo: Proyectos blockchain con TEE

1. Secret Network:

```
Arquitectura:
    - Smart contracts ejecutados en SGX
    - Datos encriptados en la blockchain
    - Solo enclaves pueden desencriptar
```

```
Ventajas:
    - Privacy-by-default para contratos
    - Performance cerca de EVM normal
    - Privacidad sin ZK overhead

Riesgos:
    - Dependencia de Intel SGX
    - Si SGX se rompe, toda la privacidad comprometida
    - Attestation centralizado (v\'ia Intel)
```

2. Oasis Network:

```
Arquitectura:
    - ParaTimes: ejecuci\'on paralela en TEEs
    - Confidential ParaTimes para privacidad
    - Separation of consensus y execution

Diferencia con Secret:
    - Modular (no todo requiere TEE)
    - M\'ultiples runtimes posibles
    - M\'as flexible pero m\'as complejo
```

3. Phala Network:

```
Arquitectura:
    - Off-chain TEE workers
    - Resultados publicados on-chain
    - Staking de workers

Modelo:
    - Workers compiten por jobs
    - Incentivos econ\'omicos para honestidad
    - Fat Contracts (computo off-chain, estado on-chain)
```

3.7.7 TEE + ZK: Combinación

Observación

Usando TEE y ZK juntos:
1. TEE para proving acelerado:

```
Problema: ZK proving es lento
Soluci\'on: Usar TEE para proving

Flujo:
    1. Usuario env\'ia witness a enclave
    2. Enclave genera ZK proof (r\'apido, hardware dedicado)
    3. Prueba verificable p\'ublicamente

Ventaja: Proving r\'apido + verificaci\'on trustless
Ejemplo: Algunos zkEVMs consideran esto
```

2. ZK para verificar TEE execution:

```
Problema: ?‘El TEE realmente ejecut\’o correctamente?
Soluci\’on: TEE genera ZK proof de su ejecuci\’on

Flujo:
    1. TEE ejecuta c\’omputo
    2. TEE genera ZK proof del resultado
    3. Cualquiera puede verificar la prueba

Ventaja: No dependes solo de attestation
```

3. Fallback de uno a otro:

```
Design pattern:
    - Primario: TEE (r\’apido, pero trust en hardware)
    - Fallback: ZK (lento, pero trustless)

Si se descubre vulnerabilidad en TEE:
    - Migrar a ZK autom\’aticamente
    - Sin cambio para usuarios
```

🏛 Aplicación en Banco ZK

TEE en la arquitectura de Banco ZK:
Decisión: Preferimos ZK sobre TEE
Razones:

- Verificación matemática vs confianza en hardware
- No dependemos de Intel/AMD/ningún fabricante
- Vulnerabilidades de TEE son catastróficas (todo expuesto)
- ZK proofs son verificables por cualquiera

Sin embargo, consideramos TEE para:
1. Proving acceleration:

```
Posible futuro: Prover service con TEE
    - Usuarios env\’ian witnesses encriptados
    - TEE genera proofs r\’apidamente
    - Resultado: proof verificable p\’ublicamente
    - Witness nunca sale del enclave
```

2. Key management:

```
Hot wallet del relayer:
    - Clave privada en TEE
    - Firma transacciones autom\’aticamente
    - Reglas de firma enforced por enclave
    - Si servidor comprometido, clave protegida
```

3. Fallback/complemento:

```
Defense in depth:
    - Primario: ZK proofs (trustless)
    - Adicional: TEE para datos en tr\'ansito
    - M\'ultiples capas de protecci\'on
```

3.8 Resumen del Capítulo

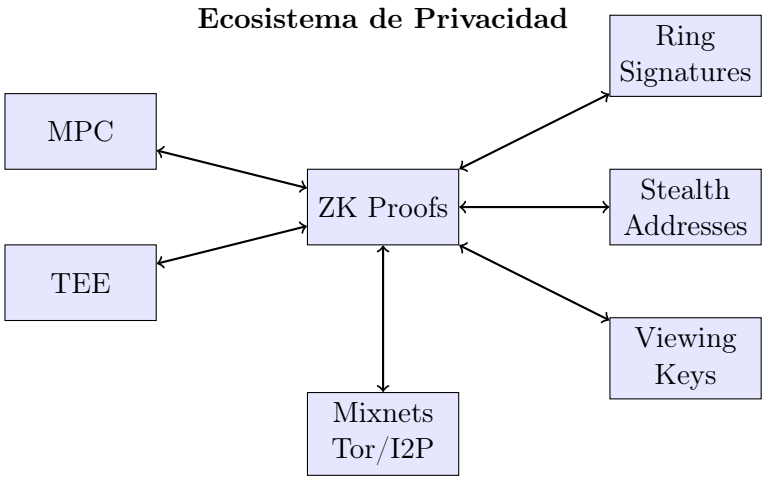
Idea Clave

Puntos clave de Privacidad Avanzada:

1. **Ring Signatures:**
   - Firma de 1-de-n sin revelar quién
   - Linkable: key images para doble gasto
   - Tamaño $O(n)$ - trade-off con anonimato
   - Usado en Monero (CLSAG)
2. **Stealth Addresses:**
   - Dirección única por transacción
   - Sender genera, receiver escanea
   - Meta-address pública, stealth privada
   - Proyectos: Umbra, EIP-5564
3. **Viewing Keys:**
   - Ver sin gastar
   - Granularidades: full, incoming, per-tx
   - Compliance, impuestos, auditoría
   - ZK disclosure como alternativa
4. **Mixnets (Nym):**
   - Encriptación en capas
   - Delays y reordenamiento
   - Cover traffic vs timing attacks
   - Incentivos con tokens
5. **Tor/I2P:**
   - Tor: baja latencia, onion services
   - I2P: garlic routing, red interna
   - Integración con crypto wallets

- Protección de capa de red

6. **MPC:**
   - Cómputo sin revelar inputs
   - Secret sharing, garbled circuits, OT
   - Aplicaciones: ceremonies, threshold sigs, auctions
   - Complementario a ZK
7. **TEE:**
   - Hardware trusted execution
   - SGX, TrustZone, SEV
   - Attestation para verificar código
   - Vulnerabilidades conocidas (side-channels)
   - Trade-off: velocidad vs confianza matemática



Chapter 4

DeFi y Aplicaciones

Introducción

Las finanzas descentralizadas (DeFi) representan uno de los casos de uso más exitosos de blockchain, permitiendo servicios financieros sin intermediarios tradicionales. Este capítulo explora los protocolos fundamentales de DeFi y su intersección con tecnologías de privacidad, especialmente Zero-Knowledge Proofs.

Idea Clave

DeFi replica servicios financieros tradicionales (intercambios, préstamos, derivados) usando smart contracts, eliminando intermediarios y permitiendo composabilidad entre protocolos. La integración con ZK permite añadir privacidad sin sacrificar auditabilidad.

4.1 AMMs (Automated Market Makers)

4.1.1 Historia y Contexto

Nota Matemática

Timeline de AMMs:

- **2017:** Bancor introduce el primer AMM on-chain
- **2018:** Uniswap v1 simplifica con constant product formula
- **2020:** Uniswap v2 añade pares ERC20/ERC20, flash swaps
- **2020:** Curve optimiza para stablecoins con StableSwap
- **2020:** Balancer generaliza a pools multi-asset
- **2021:** Uniswap v3 introduce concentrated liquidity
- **2023:** Uniswap v4 añade hooks para personalización
- **2024+:** AMMs privados con ZK empiezan a emerger

Intuición

Imagina un mercado tradicional donde compradores y vendedores negocian precios. Un AMM reemplaza esto con una **fórmula matemática** que determina precios automáticamente basándose en las reservas disponibles.
Es como una máquina expendedora infinita: siempre puedes comprar, pero el precio sube cuanto más compras (porque quedan menos). Y siempre puedes vender, pero el precio baja cuanto más vendes (porque hay más disponibles).

Definición: Automated Market Maker

Un AMM es un protocolo de intercambio descentralizado que:

1. Mantiene **reservas** de dos o más tokens en un smart contract
2. Usa una **función invariante** $f(x, y, ...) = k$ para determinar precios
3. Permite intercambios (*swaps*) que modifican reservas manteniendo el invariante
4. Permite a **Liquidity Providers (LPs)** depositar/retirar liquidez

4.1.2 Constant Product Formula (Uniswap v2)

Definición: Constant Product Market Maker (CPMM)

El invariante más simple y popular:

$$x \cdot y = k$$

donde:

- x = reserva del token X
- y = reserva del token Y
- k = constante (producto de las reservas)

Derivación Matemática del Precio

Nota Matemática

Precio Marginal:
El precio marginal de X en términos de Y se obtiene de la derivada implícita del invariante:

$$\frac{d(xy)}{dx} = 0 \implies y + x\frac{dy}{dx} = 0 \implies \frac{dy}{dx} = -\frac{y}{x}$$

Por tanto, el **precio spot** de X en términos de Y es:

$$P_X = \frac{y}{x}$$

Ejemplo: Cálculo de Precio Spot

Pool ETH/USDC con reservas:

- $x = 100$ ETH
- $y = 300,000$ USDC
- $k = 100 \times 300,000 = 30,000,000$

Precio spot de ETH:

$$P_{ETH} = \frac{300,000}{100} = 3,000 \text{ USDC/ETH}$$

Fórmula de Swap

Definición: Swap Output

Para un swap de Δx tokens X, la cantidad de tokens Y recibida es:

$$\Delta y = \frac{y \cdot \Delta x}{x + \Delta x}$$

Derivación:

$$x \cdot y = k \quad \text{(antes del swap)} \tag{4.1}$$
$$(x + \Delta x)(y - \Delta y) = k \quad \text{(después del swap)} \tag{4.2}$$
$$xy - x\Delta y + y\Delta x - \Delta x\Delta y = xy \tag{4.3}$$
$$y\Delta x = \Delta y(x + \Delta x) \tag{4.4}$$
$$\Delta y = \frac{y \cdot \Delta x}{x + \Delta x} \tag{4.5}$$

Ejemplo: Swap Numérico Completo

Usando el pool anterior (100 ETH, 300,000 USDC), queremos comprar ETH con 3,000 USDC:

Paso 1: Identificar variables

- $x = 300,000$ USDC (token que damos)
- $y = 100$ ETH (token que recibimos)
- $\Delta x = 3,000$ USDC (cantidad que damos)

Paso 2: Aplicar fórmula

$$\Delta y = \frac{100 \times 3,000}{300,000 + 3,000} = \frac{300,000}{303,000} \approx 0.9901 \text{ ETH}$$

Paso 3: Verificar invariante

- Antes: $100 \times 300,000 = 30,000,000$
- Después: $(100 - 0.9901) \times (300,000 + 3,000) = 99.0099 \times 303,000 \approx k$

Paso 4: Calcular precio efectivo

$$P_{efectivo} = \frac{3,000}{0.9901} \approx 3,030 \text{ USDC/ETH}$$

Pagamos \$3,030 por ETH cuando el precio spot era \$3,000 — la diferencia es el **slippage**.

Slippage y Price Impact

Definición: Slippage

La diferencia entre el precio esperado y el precio ejecutado:

$$\text{Slippage} = \frac{P_{ejecutado} - P_{spot}}{P_{spot}} \times 100\%$$

Definición: Price Impact

El cambio en el precio spot causado por un trade:

$$\text{Price Impact} = \frac{\Delta x}{x + \Delta x} \times 100\%$$

Para el ejemplo anterior:

$$\text{Price Impact} = \frac{3,000}{303,000} \approx 0.99\%$$

Observación

El slippage aumenta **cuadráticamente** con el tamaño del trade relativo al pool. Un trade del 10% de las reservas tiene ∼10% de price impact. Esto hace que trades grandes en pools pequeños sean muy costosos.

Nota Matemática

Fórmula General de Price Impact:
Para un trade de fracción α de las reservas ($\Delta x = \alpha \cdot x$):

$$\text{Price Impact} \approx \frac{\alpha}{1 + \alpha} \approx \alpha \quad \text{(para } \alpha \text{ pequeño)}$$

Trade Size (α)	Price Impact
0.1%	0.1%
1%	0.99%
5%	4.76%
10%	9.09%
50%	33.3%

Fees en AMMs

Definición: Fee Structure (Uniswap v2)

Uniswap v2 cobra 0.3% por swap, distribuido a LPs:

$$\Delta y_{neto} = \frac{y \cdot \Delta x \cdot (1 - \phi)}{x + \Delta x \cdot (1 - \phi)}$$

donde $\phi = 0.003$ (0.3%).
Alternativamente, el fee se puede aplicar al output:

$$\Delta y_{neto} = \Delta y_{bruto} \times (1 - \phi)$$

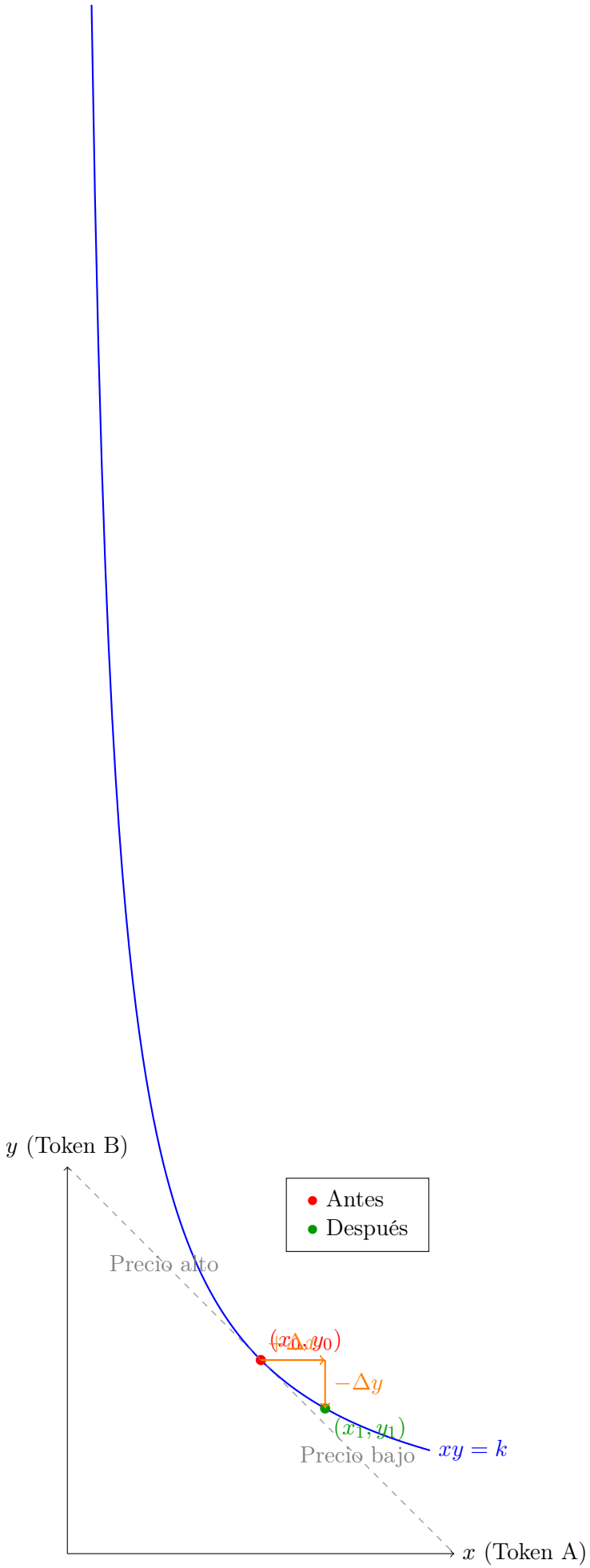
Ejemplo: Swap con Fees

Mismo ejemplo (3,000 USDC → ETH):
Sin fee: $\Delta y = 0.9901$ ETH
Con fee 0.3% (aplicado al input):

$$\Delta x_{efectivo} = 3,000 \times 0.997 = 2,991 \text{ USDC}$$

$$\Delta y = \frac{100 \times 2,991}{300,000 + 2,991} = 0.9871 \text{ ETH}$$

El LP gana $3,000 - 2,991 = 9$ USDC en fees.


y (Token B)
Antes
Después
Precio alto
(x_0, y_0)
Δx
$-\Delta y$
(x_1, y_1)
Precio bajo
$xy = k$
x (Token A)

4.1.3 Impermanent Loss

Definición: Impermanent Loss (IL)

La pérdida que sufre un LP comparado con simplemente holdear los tokens, causada por cambios en el precio relativo de los tokens.

Intuición

Como LP, siempre estás "vendiendo" el token que sube y "comprando" el que baja para mantener el balance. Si ETH sube 2x mientras eres LP, terminas con menos ETH del que tendrías si solo holdearas.
Se llama "impermanent" porque si el precio vuelve al original, la pérdida desaparece. Pero si retiras cuando el precio cambió, la pérdida se vuelve permanente.

Derivación Matemática de IL

Nota Matemática

Fórmula de Impermanent Loss:
Sea $r = \frac{P_1}{P_0}$ el ratio de cambio de precio. La IL es:

$$\text{IL}(r) = \frac{2\sqrt{r}}{1+r} - 1$$

Derivación:
Posición inicial: x_0 tokens X, y_0 tokens Y, precio $P_0 = y_0/x_0$.
Valor inicial: $V_0 = 2 \cdot x_0 \cdot P_0 = 2y_0$ (en términos de Y).
Después de cambio de precio a $P_1 = r \cdot P_0$:

- Invariante: $x \cdot y = x_0 \cdot y_0 = k$
- Nuevo precio: $\frac{y}{x} = r \cdot P_0$

Resolviendo:

$$\frac{y}{x} = r \cdot \frac{y_0}{x_0} \tag{4.6}$$

$$y = r \cdot \frac{y_0}{x_0} \cdot x \tag{4.7}$$

$$x \cdot r \cdot \frac{y_0}{x_0} \cdot x = x_0 \cdot y_0 \tag{4.8}$$

$$x^2 = \frac{x_0^2}{r} \tag{4.9}$$

$$x_1 = \frac{x_0}{\sqrt{r}}, \quad y_1 = y_0 \cdot \sqrt{r} \tag{4.10}$$

Valor LP después: $V_{LP} = x_1 \cdot P_1 + y_1 = \frac{x_0}{\sqrt{r}} \cdot rP_0 + y_0\sqrt{r} = 2y_0\sqrt{r}$
Valor HODL: $V_{HODL} = x_0 \cdot P_1 + y_0 = x_0 \cdot rP_0 + y_0 = y_0(r+1)$

$$\text{IL} = \frac{V_{LP}}{V_{HODL}} - 1 = \frac{2\sqrt{r}}{1+r} - 1$$

Cambio de Precio	Ratio r	Impermanent Loss
-50%	0.5	-5.72%
-25%	0.75	-1.03%
0%	1.0	0%
+25%	1.25	-0.62%
+50%	1.5	-2.02%
+100% (2x)	2.0	-5.72%
+200% (3x)	3.0	-13.4%
+400% (5x)	5.0	-25.5%

Observación

La IL es **simétrica**: un precio que se duplica (2x) o se reduce a la mitad (0.5x) produce la misma IL de ∼5.7%. Esto significa que la volatilidad siempre perjudica al LP, independientemente de la dirección.

IL vs Fees: Cuándo es Rentable ser LP

Idea Clave

Un LP es rentable si:

$$\text{Fees Acumulados} > |\text{Impermanent Loss}|$$

Para un pool con fee ϕ y volumen diario V:

$$\text{APR}_{fees} \approx \frac{\phi \cdot V \cdot 365}{\text{TVL}}$$

Ejemplo: Pool con TVL = \$10M, Volumen diario = \$5M, Fee = 0.3%

$$\text{APR}_{fees} = \frac{0.003 \times 5,000,000 \times 365}{10,000,000} = 54.75\%$$

4.1.4 Código Solidity: AMM Simplificado

Ejemplo: Implementación básica de CPMM

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract SimpleAMM {
    uint256 public reserveA;
    uint256 public reserveB;
    uint256 public totalSupply;

    uint256 public constant FEE_NUMERATOR = 997;
    uint256 public constant FEE_DENOMINATOR = 1000;

    function addLiquidity(uint256 amountA, uint256 amountB)
        external returns (uint256 lpTokens)
    {
```

```
        if (totalSupply == 0) {
            lpTokens = sqrt(amountA * amountB);
        } else {
            uint256 lpFromA = (amountA * totalSupply) / reserveA;
            uint256 lpFromB = (amountB * totalSupply) / reserveB;
            lpTokens = lpFromA < lpFromB ? lpFromA : lpFromB;
        }
        reserveA += amountA;
        reserveB += amountB;
        totalSupply += lpTokens;
    }

    function swapAforB(uint256 amountAIn)
        external returns (uint256 amountBOut)
    {
        uint256 amountAInWithFee = amountAIn * FEE_NUMERATOR;
        amountBOut = (reserveB * amountAInWithFee) /
                     (reserveA * FEE_DENOMINATOR + amountAInWithFee);
        reserveA += amountAIn;
        reserveB -= amountBOut;
    }

    function sqrt(uint256 x) internal pure returns (uint256 y) {
        if (x == 0) return 0;
        uint256 z = (x + 1) / 2;
        y = x;
        while (z < y) { y = z; z = (x / z + z) / 2; }
    }
}
```

🏛 Aplicación en Banco ZK

AMM en Banco ZK:
El exchange privado de Banco ZK implementa un AMM con privacidad:

- **Montos ocultos:** Los swaps no revelan cantidades exactas
- **Range proofs:** Prueban que el swap es válido sin revelar valores
- **Batching:** Múltiples swaps se agregan para aumentar anonymity set
- **Slippage privado:** El usuario especifica slippage máximo en la prueba ZK

Circuito de swap privado (∼8,000 constraints):

```
template PrivateSwap() {
    signal input amountIn;
    signal input minAmountOut;
    signal input reserveIn;
    signal input reserveOut;
    signal output amountOut;
    signal output nullifier;
```

```
    // Calcular output
    var amountInWithFee = amountIn * 997;
    amountOut <-- (reserveOut * amountInWithFee) /
                  (reserveIn * 1000 + amountInWithFee);

    // Verificar slippage
    signal slippageOk;
    slippageOk <== GreaterEqThan(64)([amountOut, minAmountOut]);
    slippageOk === 1;
}
```

4.1.5 Concentrated Liquidity (Uniswap v3)

Definición: Concentrated Liquidity

En lugar de distribuir liquidez uniformemente en todo el rango de precios $[0, \infty)$, los LPs concentran su liquidez en un **rango específico** $[p_a, p_b]$.

Intuición

Imagina que eres LP en un pool ETH/USDC. Con Uniswap v2, tu liquidez está distribuida desde precio \$0 hasta ∞. Pero ETH probablemente no va a costar \$1 ni \$1,000,000.
Con Uniswap v3, puedes decir: "Solo quiero proveer liquidez entre \$2,000 y \$4,000". Esto hace que tu capital sea mucho más eficiente, pero solo ganas fees cuando el precio está en tu rango.

Matemáticas de Concentrated Liquidity

Nota Matemática

Liquidez Virtual:
Uniswap v3 usa el concepto de **liquidez virtual** L:

$$L = \sqrt{xy}$$

La fórmula constant product se reescribe como:

$$(x + \frac{L}{\sqrt{p_b}})(y + L\sqrt{p_a}) = L^2$$

Donde:

- p_a = precio inferior del rango
- p_b = precio superior del rango
- L = liquidez concentrada

Para un rango dado, la liquidez real necesaria es:

$$x_{real} = L \cdot (\frac{1}{\sqrt{p}} - \frac{1}{\sqrt{p_b}})$$

$$y_{real} = L \cdot (\sqrt{p} - \sqrt{p_a})$$

Ticks en Uniswap v3

Definición: Tick

Un tick es un punto de precio discreto. El precio en tick i es:

$$p(i) = 1.0001^i$$

Esto significa que cada tick representa un cambio de precio de 0.01% (1 basis point).

Eficiencia de Capital

Idea Clave

La eficiencia de capital de concentrated liquidity vs full-range es:

$$\text{Eficiencia} = \frac{\sqrt{p_b/p_a}}{\sqrt{p_b/p_a} - 1} \cdot \frac{2}{1 + \sqrt{p_a/p_b}}$$

Para un rango de $\pm 10\%$ alrededor del precio actual:

$$\text{Eficiencia} \approx 10x$$

Esto significa que \$10,000 en un rango estrecho gana los mismos fees que \$100,000 en full-range.

Rango	Eficiencia vs Full-Range	Riesgo de salir del rango
±50%	2.5x	Bajo
±20%	5x	Medio
±10%	10x	Medio-Alto
±5%	20x	Alto
±1%	100x	Muy Alto

Observación

Riesgo de rangos estrechos:

- Si el precio sale de tu rango, dejas de ganar fees
- Tu posición se convierte 100% en el token depreciado
- IL es mayor en rangos estrechos
- Requiere gestión activa (rebalanceo)

4.1.6 StableSwap (Curve Finance)

Definición: StableSwap Invariant

Curve combina constant product con constant sum para assets de precio similar:

$$A \cdot n^n \sum x_i + D = A \cdot D \cdot n^n + \frac{D^{n+1}}{n^n \prod x_i}$$

Donde:

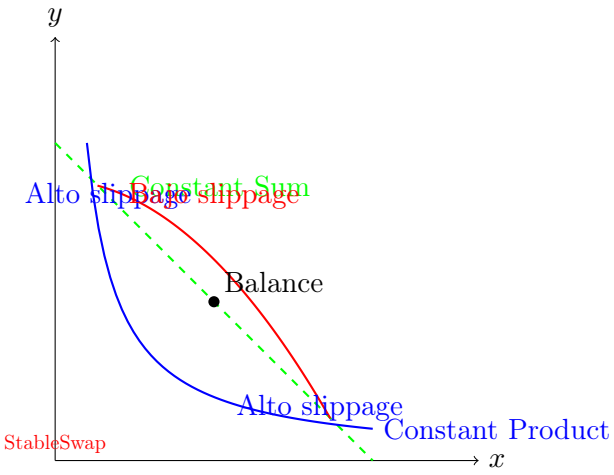
- x_i = reserva del token i
- n = número de tokens en el pool
- D = invariante (total value depositado)
- A = amplification coefficient (controla "planitud" de la curva)

Intuición

Constant Sum: $x + y = k$ — Precio siempre 1:1, pero se agota un token
Constant Product: $xy = k$ — Nunca se agota, pero alto slippage
StableSwap interpola entre ambos:

- Cuando reservas están balanceadas → se comporta como constant sum (bajo slippage)
- Cuando una reserva se agota → se comporta como constant product (protege liquidez)

El parámetro A controla cuánto se parece a constant sum. A alto = más plano = menos slippage para stablecoins.



4.1.7 Comparativa de AMMs

Protocolo	Fórmula	Capital Eff.	IL	Mejor para
Uniswap v2	$xy = k$	Baja	Media	General
Uniswap v3	Concentrated	Alta	Alta	Pares activos
Curve v1	StableSwap	Alta (stables)	Muy baja	Stablecoins
Curve v2	Crypto	Alta	Baja	Pares volátiles
Balancer	Weighted	Media	Variable	Portfolios

4.2 Lending Protocols

4.2.1 Historia y Contexto

Nota Matemática

Timeline de Lending DeFi:

- **2017:** MakerDAO lanza DAI (CDP model)
- **2018:** Compound introduce cTokens (pool model)
- **2020:** Aave v1 lanza con flash loans
- **2020:** Compound lanza COMP token (liquidity mining)
- **2021:** Aave v2 con credit delegation
- **2022:** Aave v3 con efficiency mode, portals
- **2023:** Morpho (peer-to-peer matching)
- **2024:** Lending privados con ZK empiezan a emerger

Definición: Lending Protocol

Un protocolo de préstamos DeFi permite:

1. **Depositar** activos y ganar interés
2. **Pedir prestado** contra colateral depositado

3. **Liquidación automática** si el colateral cae

Intuición

Funciona como un banco, pero sin intermediarios:

- **Depositantes** = ahorristas del banco (ganan interés)
- **Prestatarios** = tomadores de crédito (pagan interés)
- **Liquidadores** = cobradores de deuda (ganan bonificación)

La diferencia clave: todo es **overcollateralized**. No puedes pedir más de lo que depositas. Esto elimina el riesgo de crédito tradicional.

4.2.2 Modelo de Tasas de Interés

Definición: Utilization Rate

Proporción del pool que está prestada:

$$U = \frac{\text{Total Borrowed}}{\text{Total Supplied}}$$

Ejemplo: Pool de 100M USDC, 70M prestados $\rightarrow U = 0.70$ (70%)

Definición: Interest Rate Model (Compound-style)

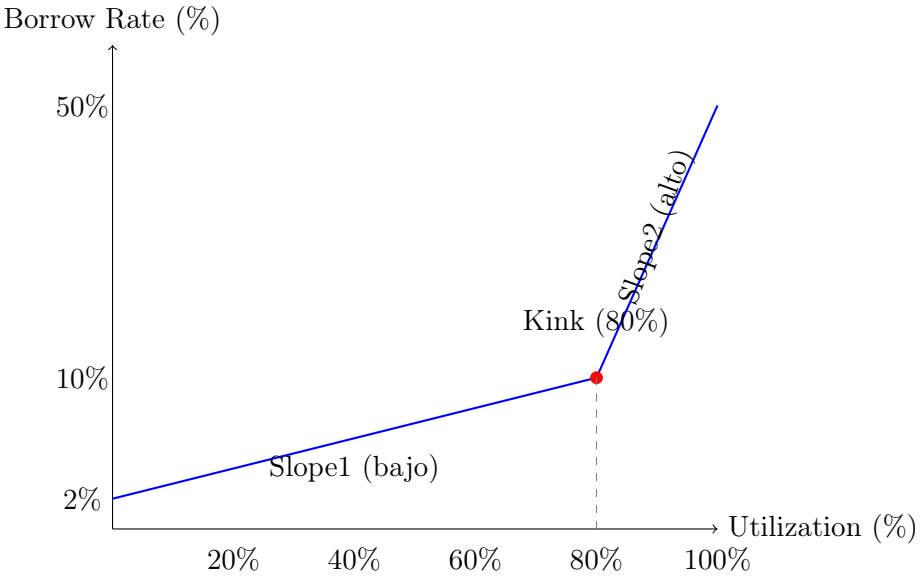
La tasa de interés es función de la utilización:
Modelo básico (lineal):

$$R_{borrow} = R_{base} + U \cdot R_{slope}$$

Modelo con "kink" (Compound/Aave):

$$R_{borrow} = \begin{cases} R_{base} + U \cdot R_{slope1} & \text{si } U \leq U_{optimal} \\ R_{base} + U_{optimal} \cdot R_{slope1} + (U - U_{optimal}) \cdot R_{slope2} & \text{si } U > U_{optimal} \end{cases}$$

Donde $R_{slope2} \gg R_{slope1}$ para desincentivar alta utilización.



4.2.3 Colateralización y Liquidaciones

Definición: Loan-to-Value (LTV)

Ratio máximo que puedes pedir prestado contra tu colateral:

$$\text{Max Borrow} = \text{Collateral Value} \times \text{LTV}$$

Ejemplo: LTV de ETH = 80%. Con \$10,000 en ETH, puedes pedir hasta \$8,000.

Definición: Health Factor

Medida de "salud" de una posición:

$$HF = \frac{\sum(\text{Collateral}_i \times \text{LiqThreshold}_i)}{\text{Total Borrowed}}$$

- $HF > 1$: Posición segura
- $HF = 1$: En el límite de liquidación
- $HF < 1$: Puede ser liquidada

Ejemplo: Escenario de Liquidación

Posición inicial:

- Colateral: 10 ETH @ \$3,000 = \$30,000
- Prestado: \$20,000 USDC
- Liquidation Threshold ETH: 82.5%

$$HF = \frac{30,000 \times 0.825}{20,000} = 1.2375$$

ETH cae a $2,400:

$$HF = \frac{24,000 \times 0.825}{20,000} = 0.99$$

La posición puede ser liquidada. Un liquidador:

1. Paga hasta $10,000 USDC (50% de la deuda, límite típico)
2. Recibe ETH equivalente + bonificación (5%)
3. Recibe: $\frac{10,000 \times 1.05}{2,400} = 4.375$ ETH

Observación

Riesgos de Lending:

- **Smart contract risk:** Bugs en el código
- **Oracle manipulation:** Precios incorrectos → liquidaciones incorrectas
- **Liquidation cascade:** Liquidaciones masivas causan más liquidaciones
- **Bad debt:** Si el precio cae muy rápido, la liquidación puede no cubrir la deuda
- **Governance attacks:** Cambios maliciosos en parámetros

4.2.4 Flash Loans

Definición: Flash Loan

Préstamo **sin colateral** que debe ser devuelto en la **misma transacción**. Si no se devuelve, toda la transacción revierte.

Intuición

Un flash loan es como pedir prestado $10 millones por 13 segundos (un bloque). Puedes hacer lo que quieras con ese dinero, pero *debes* devolverlo antes de que termine la transacción.
Es posible porque en blockchain todo es atómico: o la transacción completa sucede, o nada sucede. No hay estado intermedio donde el protocolo "pierde" el dinero.

Casos de Uso de Flash Loans

1. **Arbitraje:**
   - Pedir prestado USDC
   - Comprar ETH barato en DEX A
   - Vender ETH caro en DEX B
   - Devolver USDC + fee, quedarse con ganancia
2. **Liquidaciones:**
   - Pedir prestado USDC para pagar deuda de otro usuario
   - Recibir colateral con bonificación

- Vender colateral, devolver flash loan

3. **Collateral Swap:**

   - Pedir prestado para pagar tu propia deuda
   - Retirar colateral (ETH)
   - Cambiar ETH por otro activo (BTC)
   - Depositar nuevo colateral
   - Pedir prestado de nuevo para devolver flash loan

4. **Self-liquidation:**

   - Evitar pagar bonificación al liquidador
   - Usar flash loan para cerrar posición uno mismo

🏛 Aplicación en Banco ZK

Lending Privado en Banco ZK:
Banco ZK implementa lending con privacidad parcial:

- **Montos ocultos:** El público no ve cuánto depositas/pides prestado
- **Health Factor verificable:** ZK proof de que $HF > 1$ sin revelar valores exactos
- **Liquidaciones privadas:** Solo el liquidador sabe el monto exacto
- **Tasas públicas:** Las tasas de interés son públicas para transparencia

Circuito de verificación de Health Factor (∼5,000 constraints):

```
template HealthFactorCheck() {
    signal input collateralValue;
    signal input borrowedValue;
    signal input liqThreshold;  // e.g., 8250 para 82.5%
    signal output isHealthy;  // 1 si HF > 1

    signal numerator;
    numerator <== collateralValue * liqThreshold;

    signal denominator;
    denominator <== borrowedValue * 10000;

    isHealthy <== GreaterThan(128)([numerator, denominator]);
}
```

4.3 Oráculos

4.3.1 El Problema del Oráculo

Definición: Oracle Problem

Las blockchains son sistemas deterministas cerrados: no pueden acceder a datos externos de forma nativa. Un **oráculo** es un mecanismo para traer datos del mundo real a la blockchain de forma confiable.

Intuición

Imagina un smart contract de seguros agrícolas que paga si llueve menos de 50mm en un mes. El contrato necesita saber cuánto llovió, pero esa información está "fuera" de la blockchain.
El problema es: ¿cómo confiar en quien reporta la lluvia? Si una sola persona lo reporta, puede mentir. Si es un grupo, pueden coludirse. Los oráculos intentan resolver esto con incentivos económicos y descentralización.

Observación

"Garbage In, Garbage Out":
Un smart contract es tan seguro como sus oráculos. Un contrato perfectamente auditado puede ser explotado si el oráculo reporta precios incorrectos.
Ejemplos históricos:

- **Mango Markets (2022):** Manipulación de precio para pedir préstamos excesivos ($114M)
- **Cream Finance (2021):** Oracle manipulation, $130M robados
- **Harvest Finance (2020):** Flash loan + oracle manipulation, $34M

4.3.2 Tipos de Oráculos

Definición: Clasificación de Oráculos

Por dirección del flujo:

- **Inbound:** Traen datos externos → blockchain (precios, clima)
- **Outbound:** Envían datos blockchain → mundo externo (triggers)

Por arquitectura:

- **Centralizados:** Una entidad provee datos (simple pero single point of failure)
- **Descentralizados:** Red de nodos con consenso (Chainlink, Band)
- **On-chain:** Calculados de datos on-chain (TWAP de AMMs)

Por tipo de dato:

- **Price feeds:** Precios de activos
- **VRF:** Aleatoriedad verificable

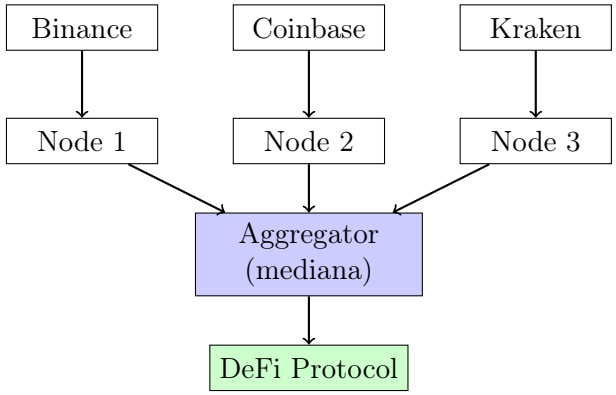
- **Proof of Reserve:** Verificación de reservas off-chain
- **Cross-chain:** Estado de otras blockchains

4.3.3 Chainlink

Definición: Chainlink

Red descentralizada de oráculos con:

- **Nodos operadores:** Proveen datos y staking
- **Aggregation:** Múltiples fuentes, mediana como resultado
- **Reputation:** Sistema de reputación basado en historial
- **LINK token:** Para pagos y staking



4.3.4 TWAP (Time-Weighted Average Price)

Definición: TWAP

Precio promedio ponderado por tiempo, calculado de datos on-chain (típicamente de un AMM):

$$\text{TWAP}_{t_1 \to t_2} = \frac{1}{t_2 - t_1} \int_{t_1}^{t_2} P(t)\, dt$$

En práctica, usando acumuladores:

$$\text{TWAP} = \frac{\text{cumulative}_{t_2} - \text{cumulative}_{t_1}}{t_2 - t_1}$$

Donde $\text{cumulative}_t = \sum_{i=0}^{t} P_i \cdot \Delta t_i$

Intuición

TWAP es más resistente a manipulación que el precio spot porque un atacante necesitaría mantener el precio manipulado durante todo el período TWAP, lo cual es costoso. Si el TWAP es de 30 minutos y un atacante manipula el precio por 1 minuto, solo afecta 1/30 del TWAP final.

4.3.5 Manipulación de Oráculos

Definición: Oracle Manipulation Attack

Ataque donde el adversario manipula el precio reportado por un oráculo para explotar un protocolo DeFi.

Vectores de Ataque

1. **Flash Loan + AMM Manipulation:**
   - Pedir flash loan masivo
   - Swap grande para mover precio del AMM
   - Protocolo víctima lee precio manipulado
   - Explotar (pedir préstamo excesivo, liquidar incorrectamente)
   - Revertir swap, devolver flash loan
2. **Sandwich de Oráculo:**
   - Detectar actualización de oráculo pendiente
   - Front-run con transacción que se beneficia del precio viejo
   - Dejar que el oráculo actualice
   - Back-run para cerrar posición con precio nuevo
3. **Stale Price:**
   - Explotar cuando el oráculo no se actualiza
   - Precio on-chain difiere significativamente del real
   - Arbitrar la diferencia

Defensas

Idea Clave

Best Practices para Oráculos:

1. **TWAP > Spot:** Usar precios promediados, no instantáneos
2. **Múltiples fuentes:** No depender de un solo oráculo
3. **Sanity checks:** Verificar que el precio está en rango razonable
4. **Freshness checks:** Rechazar precios muy viejos
5. **Circuit breakers:** Pausar protocolo si hay anomalías
6. **Delays:** Introducir delays para hacer ataques más costosos

4.3.6 Chainlink VRF (Verifiable Random Function)

Definición: VRF

Función que genera números aleatorios con prueba criptográfica de que fueron generados correctamente. Propiedades:

- **Unpredictable:** Nadie puede predecir el resultado antes de generarlo
- **Verifiable:** Cualquiera puede verificar que el resultado es correcto
- **Deterministic:** La misma entrada siempre produce la misma salida

Aplicación en Banco ZK

Oráculos en Banco ZK:
Banco ZK usa un diseño híbrido de oráculos:

- **Precios públicos:** Chainlink para ETH/USD (no hay razón para ocultar)
- **Verificación privada:** ZK proofs que verifican condiciones sin revelar valores
- **TWAP privado:** Acumuladores cifrados para TWAP que no revela trades individuales

Ejemplo: Verificar que un préstamo está healthy sin revelar el monto:

```
template OracleBasedHealthCheck() {
    signal input collateralAmount;     // Privado
    signal input oraclePrice;          // P\'ublico (de Chainlink)
    signal input borrowedAmount;       // Privado
    signal input liquidationThreshold; // P\'ublico
    signal output isHealthy;

    signal collateralValue;
    collateralValue <== collateralAmount * oraclePrice;

    signal threshold_x_collateral;
    threshold_x_collateral <== collateralValue * liquidationThreshold;

    signal threshold_x_borrowed;
    threshold_x_borrowed <== borrowedAmount * 10000;

    isHealthy <== GreaterThan(128)(
        [threshold_x_collateral, threshold_x_borrowed]
    );
}
```

Esto permite que Banco ZK use precios de mercado públicos mientras mantiene los montos de los usuarios privados.

4.4 Bridges ZK

4.4.1 El Problema de los Bridges

Definición: Blockchain Bridge

Protocolo que permite transferir activos o mensajes entre dos blockchains diferentes.

Intuición

Las blockchains son "islas" aisladas. Bitcoin no sabe nada de Ethereum, y viceversa. Un bridge es como un ferry que lleva activos de una isla a otra.
El problema es: ¿cómo confiar en el ferry? Si el operador del ferry dice "te doy 1 ETH en Ethereum porque bloqueaste 1 ETH en otra cadena", ¿cómo verificamos que realmente lo hizo?

Modelos de Bridges Tradicionales

Modelo	Trust	Seguridad	Ejemplos
Custodial	Centralizado	Baja	WBTC
Multisig	n-of-m	Media	Multichain
MPC	Threshold	Media-Alta	Thorchain
Optimistic	Fraud proofs	Alta	Nomad
ZK	**Math**	**Muy Alta**	**Succinct, zkBridge**

Observación

Historial de Hacks en Bridges:

- **Ronin (2022):** \$624M — Multisig comprometido
- **Wormhole (2022):** \$320M — Bug en verificación de firmas
- **Nomad (2022):** \$190M — Bug en verificación de merkle proofs
- **Multichain (2023):** \$126M — CEO detenido, claves comprometidas

Más de \$2B robados de bridges. Son el eslabón más débil del ecosistema multi-chain.

4.4.2 ZK Bridges: Verificación Criptográfica

Definición: ZK Bridge

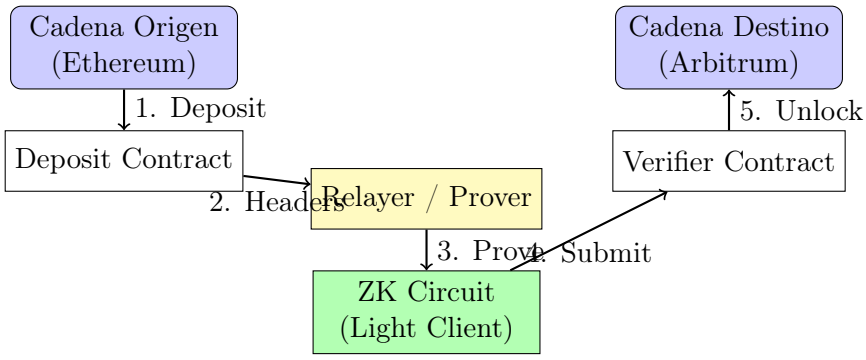
Bridge que verifica la validez de transacciones cross-chain usando **pruebas de conocimiento cero**, eliminando la necesidad de confiar en operadores.

Idea Clave

La idea central de un ZK bridge es:
"No confíes en que el operador dice la verdad. Verifica una prueba matemática de que la transacción realmente ocurrió en la cadena origen."
Esto convierte el problema de confianza en un problema de criptografía, que es mucho

más seguro.

Arquitectura de ZK Bridge



Componentes del ZK Bridge

Definición: Light Client en ZK

Un circuito ZK que verifica el consenso de una blockchain sin procesar todos los bloques:

1. **Verificar firmas de validadores:** Probar que $\geq 2/3$ de validadores firmaron el bloque
2. **Verificar transición de estado:** Block hash anterior $\rightarrow$ Block hash actual
3. **Verificar inclusión:** Merkle proof de que la transacción está en el bloque

Nota Matemática

Complejidad de Light Client ZK:
Para Ethereum PoS:

- **Sync Committee:** 512 validadores por período (256 epochs)
- **Firmas BLS:** Agregación permite verificar 512 firmas como una
- **Merkle Proofs:** ~20 hashes para verificar inclusión

Circuito típico: 1-10M constraints

- BLS signature verification: ~500K constraints per signature
- BLS aggregation check: ~2M constraints
- Merkle proof (SHA256, 20 levels): ~200K constraints
- State root verification: ~100K constraints

Tiempo de proving: 30 segundos - 5 minutos (dependiendo del hardware)

4.4.3 Proyectos de ZK Bridges

Proyecto	Tecnología	Cadenas	Tiempo Proof	Estado
Succinct	SP1 (zkVM)	Ethereum +	∼2 min	Mainnet
zkBridge	deVirgo	ETH, Cosmos	∼5 min	Testnet
Lagrange	Custom ZK	Multi-chain	∼1 min	Mainnet
Polyhedra	Expander	20+ chains	∼30 sec	Mainnet

4.4.4 Desafíos y Limitaciones

Observación

Desafíos actuales de ZK Bridges:

1. **Costo de proving:**
   - Generar pruebas requiere hardware potente
   - $0.10 - $10 por prueba dependiendo del circuito
   - Amortizado si hay volumen
2. **Latencia:**
   - 30 segundos - 5 minutos para generar prueba
   - + tiempo de inclusión en ambas cadenas
   - Total: 5-30 minutos vs segundos en bridges tradicionales
3. **Complejidad de consenso:**
   - Cada blockchain tiene consenso diferente
   - Cambios en el protocolo requieren actualizar circuitos
   - BLS signatures son costosas en ZK
4. **Ataques de reorganización:**
   - ¿Qué pasa si la cadena origen hace reorg después del bridge?
   - Necesita esperar finalidad (12+ minutos en Ethereum)

Aplicación en Banco ZK

ZK Bridge de Banco ZK:
Banco ZK implementa un bridge nativo a Ethereum L1 con las siguientes características:

- **Deposits (L1 → L2):**
  1. Usuario deposita en contrato L1
  2. Secuenciador de Banco ZK detecta el depósito
  3. Genera commitment a la nota privada
  4. Usuario puede gastar en L2 con prueba de inclusión
- **Withdrawals (L2 → L1):**

1. Usuario genera prueba ZK de posesión de nota
2. Prueba se incluye en batch de L2
3. Proof agregado se verifica en L1
4. Usuario reclama fondos en L1

- **Tiempos:**
  - Deposit: 12 minutos (finalidad ETH) + 1 minuto (procesamiento)
  - Withdrawal: Inmediato con prueba válida (no hay dispute period)
- **Privacidad del bridge:**
  - Deposits en L1 son públicos (necesario)
  - Actividad en L2 es privada
  - Withdrawals revelan monto pero no historial

4.5 Atomic Swaps

4.5.1 Fundamentos

Definición: Atomic Swap

Intercambio de activos entre dos blockchains sin intermediarios, con garantía de que:

- Ambas partes reciben sus fondos, o
- Ninguna parte pierde sus fondos (transacción revierte)

La "atomicidad" significa: todo o nada.

Intuición

Imagina que quieres cambiar 1 BTC por 30 ETH con un desconocido en internet:

- Si envías primero, pueden no enviarte ETH
- Si ellos envían primero, puedes no enviarles BTC

Un atomic swap usa criptografía para crear un "acuerdo" donde ambas transferencias están vinculadas por un secreto. Quien revela el secreto para cobrar en una cadena, automáticamente permite que la otra parte cobre en la otra cadena.

4.5.2 HTLC (Hash Time-Locked Contracts)

Definición: HTLC

Contrato que bloquea fondos con dos condiciones de desbloqueo:

1. **Hashlock:** Revelar preimagen s tal que $H(s) = h$
2. **Timelock:** Esperar hasta tiempo t (refund)

Protocolo de Atomic Swap con HTLC

Nota Matemática

Protocolo Detallado:
Setup:

- Alice quiere: 30 ETH (tiene 1 BTC)
- Bob quiere: 1 BTC (tiene 30 ETH)
- Acuerdan precio: 1 BTC = 30 ETH

Paso 1: Alice genera secreto

- $s \leftarrow \{0,1\}^{256}$ (256 bits aleatorios)
- $h = \text{SHA256}(s)$
- Alice envía h a Bob (NO el secreto s)

Paso 2: Alice bloquea en Bitcoin

- Crea HTLC: "1 BTC para quien revele s tal que $H(s) = h$, o devolver a Alice después de 48 horas"
- Publica transacción en Bitcoin

Paso 3: Bob verifica y bloquea en Ethereum

- Bob verifica que el HTLC de Alice es correcto
- Crea HTLC: "30 ETH para quien revele s tal que $H(s) = h$, o devolver a Bob después de 24 horas"
- El timelock de Bob (24h) es MENOR que el de Alice (48h)

Paso 4: Alice reclama ETH

- Alice revela s en Ethereum para reclamar 30 ETH
- Al hacerlo, s se vuelve público (visible en blockchain)

Paso 5: Bob reclama BTC

- Bob ve s en la transacción de Alice
- Bob usa s en Bitcoin para reclamar 1 BTC
- Tiene 24+ horas para hacerlo (48h - 24h = 24h de margen)

4.5.3 Limitaciones de HTLC

Observación

Problemas de Atomic Swaps con HTLC:

1. **Tiempos largos:**
   - Necesita esperar confirmaciones en ambas cadenas
   - Timelocks de 24-48 horas típicos
   - No es práctico para trading frecuente
2. **Problema de "Free Option":**
   - Alice puede esperar hasta el último momento
   - Si el precio se mueve a su favor, reclama
   - Si el precio se mueve en contra, deja expirar
   - Bob queda "atrapado" durante el timelock
3. **Requiere ambas partes online:**
   - Bob debe monitorear para ver cuando Alice revela s
   - Si Bob está offline, puede perder la ventana de reclamo
4. **No soporta todas las cadenas:**
   - Requiere scripting (hashlocks, timelocks)
   - Bitcoin sí, pero algunas cadenas no

4.6 Cross-chain Interoperability

4.6.1 Protocolos de Messaging

Definición: Cross-chain Messaging

Protocolos que permiten enviar mensajes arbitrarios entre blockchains, no solo transferencias de valor.

IBC (Inter-Blockchain Communication)

Definición: IBC

Protocolo de Cosmos para comunicación entre blockchains:

- **Light Clients:** Cada cadena mantiene light client de las otras
- **Relayers:** Transmiten paquetes entre cadenas
- **Canales:** Conexiones ordenadas entre módulos
- **Timeout:** Garantía de entrega o reembolso

LayerZero

Definición: LayerZero

Protocolo de messaging cross-chain con:

- **Ultra Light Nodes:** Light clients on-demand
- **Oracles:** Reportan block headers
- **Relayers:** Transmiten proofs de transacciones
- **Modular security:** Apps eligen oracles/relayers

Axelar y Wormhole

Definición: Axelar

Red de validadores para general message passing:

- **Proof-of-Stake:** Validadores con AXL staked
- **Threshold signatures:** Firma colectiva para aprobar mensajes
- **Gateway contracts:** En cada cadena conectada
- **GMP:** General Message Passing

Definición: Wormhole

Protocolo cross-chain con Guardians:

- **19 Guardians:** Nodos que observan todas las cadenas
- **VAAs:** Verified Action Approvals (mensajes firmados)
- **13/19 threshold:** Se requieren 13 firmas para aprobar
- **24+ cadenas:** Ethereum, Solana, BSC, etc.

4.6.2 Comparativa de Protocolos Cross-chain

Protocolo	Modelo	Seguridad	Cadenas	Latencia
IBC	Light clients	Muy alta	Cosmos	∼20 sec
LayerZero	Oracle + Relayer	Configurable	50+	∼2 min
Axelar	PoS validators	Alta	40+	∼2 min
Wormhole	Multisig (19)	Media-Alta	24+	∼15 min
Hyperlane	Modular	Configurable	30+	Variable
CCIP	DON network	Alta	10+	∼20 min

4.7 Privacy Pools

4.7.1 El Dilema Privacidad vs Compliance

Intuición

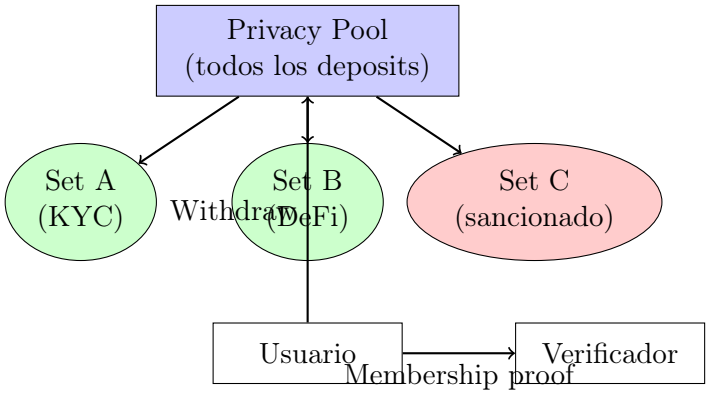
Tornado Cash demostró que la privacidad financiera es posible on-chain. Pero también fue usado para lavar fondos robados, lo que llevó a sanciones.
El dilema es: ¿Cómo tener privacidad legítima sin facilitar crímenes?
Privacy Pools es una propuesta para resolver esto: privacidad para usuarios legítimos, pero con la capacidad de demostrar que no estás asociado con actores maliciosos.

Definición: Privacy Pools (Propuesta de Vitalik et al., 2023)

Sistema de privacidad donde los usuarios pueden probar propiedades de sus fondos sin revelar su historial completo:

1. **Association Sets:** Conjuntos de direcciones/transacciones "limpias"
2. **Membership Proofs:** Pruebas ZK de que tus fondos vienen de un set "bueno"
3. **Exclusion Proofs:** Pruebas ZK de que tus fondos NO vienen de un set "malo"

4.7.2 Arquitectura de Privacy Pools



4.7.3 Tipos de Association Sets

Definición: Association Set Providers (ASPs)

Entidades que definen y mantienen association sets:

- **Exchanges:** Set de direcciones verificadas KYC
- **Chainalysis/Elliptic:** Sets de direcciones "limpias" o "sucias"
- **DAOs:** Sets curados por la comunidad
- **Gobiernos:** Sets de direcciones sancionadas

Nota Matemática

Pruebas de Membership/Exclusion:
Membership Proof:

- Input público: Raíz del Merkle tree del Association Set
- Input privado: Tu nota, Merkle proof de inclusión
- Output: "Mi nota está en este set"

Exclusion Proof:

- Input público: Raíz del Merkle tree del Bad Set
- Input privado: Tu nota, prueba de no-inclusión (más compleja)
- Output: "Mi nota NO está en este set"

La prueba de no-inclusión típicamente usa sparse Merkle trees o accumulators criptográficos.

4.8 zkML (Zero-Knowledge Machine Learning)

4.8.1 Motivación

Definición: zkML

Aplicación de pruebas de conocimiento cero a modelos de machine learning, permitiendo:

- Probar que un modelo produjo cierto output sin revelar el modelo
- Probar que un input produjo cierto output sin revelar el input
- Verificar que se usó un modelo específico (modelo auditable)

Intuición

Imagina que tienes un modelo de ML para scoring crediticio. Quieres:

- Demostrar al regulador que usas un modelo aprobado
- Sin revelar los parámetros exactos (propiedad intelectual)
- Sin revelar los datos del cliente (privacidad)

zkML permite esto: el regulador verifica una prueba de que "el modelo aprobado dice que este cliente tiene score > 700", sin ver el modelo ni los datos del cliente.

4.8.2 Casos de Uso de zkML

1. **On-chain AI:**
   - Ejecutar inferencia ML off-chain
   - Verificar resultado on-chain con ZK proof

- Ej: Oráculos de ML, NPCs en juegos, content moderation

2. **Private ML Inference:**
   - Usuario tiene datos privados
   - Proveedor tiene modelo privado
   - Ambos verifican resultado sin revelar sus secretos
3. **Model Auditing:**
   - Probar que el modelo no es discriminatorio
   - Probar que se entrenó con datos autorizados
   - Sin revelar el modelo completo
4. **Proof-of-Personhood:**
   - Probar que una imagen es de humano real
   - Sin revelar la imagen
   - Útil contra bots y Sybil attacks

4.8.3 Desafíos Técnicos

Observación

zkML es computacionalmente intensivo:

- **Modelos grandes = circuitos enormes:**
  - GPT-2 small (124M params) → ∼100B constraints
  - Inviable con tecnología actual
- **Operaciones no-ZK-friendly:**
  - Division, sqrt, exp → costosos en circuitos
  - Floating point → se usa fixed point
  - Non-linear activations (ReLU, sigmoid) → aproximaciones
- **Estado del arte:**
  - Modelos pequeños (<1M params) son viables
  - CNNs simples, decision trees, linear regression
  - Tiempo de proof: minutos a horas

4.8.4 Proyectos de zkML

Proyecto	Enfoque	Modelos Soportados
EZKL	PyTorch/ONNX → Halo2	CNNs pequeños, transformers mini
Modulus Labs	Circuitos optimizados	Custom models
Giza	Cairo/StarkNet	ONNX models
Worldcoin	Iris recognition	Neural network específico
Zkonduit	Halo2 backend	ONNX export

Aplicación en Banco ZK

zkML en Banco ZK:
Banco ZK explora zkML para varios casos de uso:

- **Compliance automático:**
  - Modelo de ML clasifica transacciones como suspicious/clean
  - Usuario genera prueba de que su transacción es "clean" según el modelo
  - Sin revelar detalles de la transacción
- **Credit scoring privado:**
  - Usuario tiene historial on-chain
  - Modelo de credit scoring aprobado por regulador
  - Usuario prueba "mi score > threshold" sin revelar score exacto ni historial
- **Fraud detection:**
  - Modelo detecta patrones de fraude
  - Se verifica que todas las transacciones pasaron el modelo
  - Sin revelar el modelo (evita gaming) ni las transacciones

Nota: Esto es experimental y los modelos deben ser pequeños (<100K params) para ser prácticos actualmente.

4.9 Resumen del Capítulo

Idea Clave

Puntos clave de DeFi y Aplicaciones:
1. **AMMs:**
   - Constant Product: $xy = k$, slippage proporcional al trade
   - Concentrated Liquidity: Mayor eficiencia, mayor riesgo de IL
   - StableSwap: Optimizado para stablecoins, bajo slippage
   - IL inevitable, compensada por fees
2. **Lending:**
   - Tasas basadas en utilización (modelo con "kink")
   - Overcollateralized: siempre depositas más de lo que pides
   - Health Factor $< 1 \rightarrow$ liquidación
   - Flash loans: préstamos atómicos sin colateral
3. **Oráculos:**
   - Traen datos externos a blockchain

- TWAP > spot price (resistencia a manipulación)
- Siempre verificar freshness y sanity bounds
- Chainlink VRF para aleatoriedad verificable

4. **ZK Bridges:**
   - Verificación criptográfica vs confianza en operadores
   - Light clients ZK verifican consenso de otras cadenas
   - Más seguro pero más lento que bridges tradicionales
   - Proyectos: Succinct, Lagrange, Polyhedra
5. **Atomic Swaps:**
   - HTLC: hashlock + timelock para intercambios trustless
   - Tiempos largos y problema de "free option"
   - ZK Atomic Swaps permiten montos privados
6. **Cross-chain:**
   - IBC: Light clients nativos (Cosmos)
   - LayerZero: Oracle + Relayer modular
   - Axelar/Wormhole: Validadores/Guardians
7. **Privacy Pools:**
   - Balance entre privacidad y compliance
   - Association Sets definen grupos "buenos"/"malos"
   - Membership proofs sin revelar transacción específica
8. **zkML:**
   - ML + ZK = inferencia verificable y/o privada
   - Solo modelos pequeños son prácticos actualmente
   - Casos: on-chain AI, compliance, fraud detection

🏛 Aplicación en Banco ZK

DeFi en Banco ZK — Visión General:
Banco ZK integra DeFi con privacidad en todos los niveles:

Componente	Público	Privado
AMM	Precios, liquidez total	Trades individuales
Lending	Tasas, utilización	Posiciones de usuarios
Oracle	Precios de mercado	Verificaciones individuales
Bridge	Raíz de estado	Transacciones específicas
Compliance	Sets aprobados	Membership individual

Principio guía: *"Máxima privacidad para usuarios, máxima transparencia para el sistema."*

Chapter 5

Avanzado e Investigación

Introducción

Este capítulo explora los temas más avanzados y las fronteras de investigación en Zero-Knowledge Proofs. Desde la verificación formal que garantiza la corrección de circuitos, hasta sistemas post-cuánticos que preparan la tecnología para el futuro, estos temas representan el estado del arte y las direcciones futuras del campo.

Idea Clave

Los temas de este capítulo son relevantes para:

- **Desarrolladores avanzados:** Optimización y verificación formal
- **Arquitectos de sistemas:** Agregación y recursión de pruebas
- **Investigadores:** Fronteras del conocimiento (zkML, post-quantum)
- **Planificadores:** Preparación para computación cuántica

5.1 zkML Avanzado

5.1.1 Fundamentos y Arquitectura

Definición: Zero-Knowledge Machine Learning

zkML es la aplicación de pruebas de conocimiento cero a modelos de machine learning, permitiendo:

1. **Inferencia verificable:** Probar que un modelo produjo cierto output
2. **Inferencia privada:** Ejecutar ML sin revelar inputs o modelo
3. **ML on-chain:** Verificar resultados de ML en blockchain
4. **Auditoría de modelos:** Probar propiedades del modelo sin revelarlo

Nota Matemática

Timeline de zkML:

- **2020:** Primeros papers sobre ML verificable con ZK

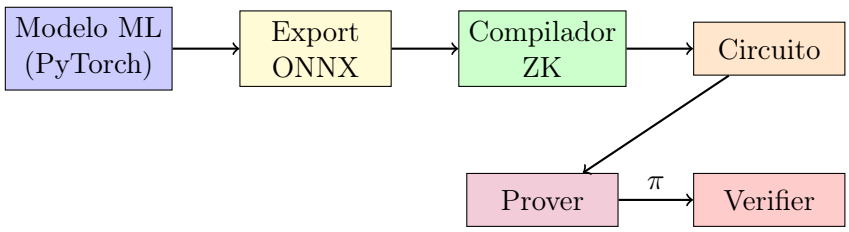
- **2021:** EZKL comienza desarrollo (PyTorch → ZK)
- **2022:** Modulus Labs prueba concepto con modelos pequeños
- **2023:** Worldcoin usa zkML para verificación de iris
- **2023:** EZKL alcanza producción, Giza en StarkNet
- **2024:** Mejoras en eficiencia, modelos más grandes viables
- **2025+:** Investigación en transformers ZK-friendly

Intuición

¿Por qué zkML?
Imagina que tienes un modelo de ML que detecta fraude financiero. Quieres:

- Demostrar a un auditor que usaste el modelo aprobado
- Sin revelar los parámetros exactos (tu ventaja competitiva)
- Sin revelar los datos del cliente (privacidad, GDPR)

Con zkML, generas una prueba que dice: "El modelo M con hash h_M, aplicado al input con hash h_x, produjo output y". El auditor puede verificar esto sin ver M ni x.



5.1.2 Desafíos Técnicos de zkML

Aritmética de Punto Flotante

Observación

El Problema: Los circuitos ZK operan sobre campos finitos $\mathbb{F}_p$, pero ML usa punto flotante IEEE 754.
No puedes hacer "3.14159 × 2.71828" en un campo finito directamente.

Definición: Fixed-Point Arithmetic

Representación de números reales usando enteros con punto decimal implícito:

$$x_{fixed} = \lfloor x_{float} \times 2^{scale} \rfloor$$

Donde *scale* es el número de bits fraccionales (típicamente 16-32).
Operaciones:

$$(a + b)_{fixed} = a_{fixed} + b_{fixed} \tag{5.1}$$

$$(a \times b)_{fixed} = \frac{a_{fixed} \times b_{fixed}}{2^{scale}} \tag{5.2}$$

Nota Matemática

Error de Cuantización:
El error máximo de fixed-point con *scale* bits es:

$$\epsilon_{max} = \frac{1}{2^{scale+1}}$$

Para $scale = 16$: $\epsilon_{max} \approx 7.6 \times 10^{-6}$
Este error se **acumula** a través de las capas del modelo. Un modelo con 100 capas puede tener error acumulado significativo.

Funciones de Activación No Lineales

Observación

Las funciones de activación comunes son **muy costosas** en circuitos:

- **ReLU:** $\max(0, x)$ requiere comparación → bits de descomposición
- **Sigmoid:** $\frac{1}{1+e^{-x}}$ requiere exponencial y división
- **Softmax:** Exponenciales + división + suma
- **GELU:** Aún más complejo (usado en transformers)

Definición: Aproximaciones ZK-Friendly

ReLU:

- Descomponer x en bits para determinar signo
- ~ 254 constraints para un número de 254 bits
- Optimización: usar lookup tables

Sigmoid (aproximación polinomial):

$$\sigma(x) \approx 0.5 + 0.25x - 0.0208x^3 \quad \text{para } x \in [-4, 4]$$

Costo de Operaciones

Operación	Constraints (aprox)	Notas
Suma	0	Gratis (lineal)
Multiplicación	1	Operación base
División	$\sim$500	Requiere inverse
ReLU (bit decomp)	$\sim$254	Por cada activación
ReLU (lookup)	$\sim$3	Con tabla pre-computada
Sigmoid (poly)	$\sim$5	Aproximación cúbica
Matrix Mul $(n \times m) \times (m \times k)$	$n \times m \times k$	Dominante en redes

Ejemplo: Costo de una Red Simple

Modelo: MLP con arquitectura $784 \to 128 \to 64 \to 10$ (MNIST classifier)
Capa 1: $784 \times 128 = 100,352$ multiplicaciones + 128 ReLU
Capa 2: $128 \times 64 = 8,192$ multiplicaciones + 64 ReLU
Capa 3: $64 \times 10 = 640$ multiplicaciones
Total (con ReLU lookup):

- Multiplicaciones: $100,352 + 8,192 + 640 = 109,184$ constraints
- ReLU: $(128 + 64) \times 3 = 576$ constraints (lookup)
- **Total:** ~110,000 constraints

Tiempo de proving (Halo2, CPU): ~30 segundos

5.1.3 Frameworks de zkML

Definición: EZKL

Framework que compila modelos ONNX a circuitos Halo2:

- **Input:** Modelo PyTorch/TensorFlow exportado a ONNX
- **Output:** Circuito Halo2 + prover + verifier Solidity
- **Features:** Calibración automática, lookups, optimizaciones

Definición: Giza (StarkNet)

Plataforma zkML para StarkNet usando STARKs:

- **Ventaja:** STARKs son post-quantum
- **Proceso:** ONNX → Cairo → STARK proof
- **Verificación:** Nativa en StarkNet

5.1.4 Aplicaciones Prácticas de zkML

Proof of Personhood (Worldcoin)

Definición: Worldcoin Iris Verification

Sistema que prueba unicidad de persona usando escaneo de iris:

1. **Captura:** Orb escanea iris del usuario
2. **Modelo ML:** Extrae embedding del iris
3. **ZK Proof:** Prueba que embedding es único sin revelarlo
4. **On-chain:** Verifica prueba y registra identidad

Private Credit Scoring

Ejemplo: Credit Scoring Privado

Un circuito que prueba "Mi score es ≥ threshold" sin revelar el score exacto:

```
template PrivateCreditScore(nFeatures) {
    signal input features[nFeatures];     // Datos financieros (privados)
    signal input modelCommitment;         // Hash de los pesos (p\'ublico)
    signal input threshold;               // Score m\'inimo requerido
    signal output meetsThreshold;         // 1 si score >= threshold

    // Ejecutar modelo y comparar con threshold
    // Solo revela: ''cumple'' o ''no cumple''
}
```

5.1.5 Limitaciones Actuales

Observación

Limitaciones de zkML (2025):

1. **Tamaño de modelos:**
   - Viable: < 1M parámetros
   - Posible con optimización: 1-10M parámetros
   - Imposible hoy: GPT-2 (124M), BERT (110M), LLaMA (7B+)
2. **Tiempo de proving:**
   - Modelo pequeño: segundos a minutos
   - Modelo mediano: minutos a horas
   - No es real-time para la mayoría de casos
3. **Precisión:**
   - Fixed-point introduce error de cuantización
   - Aproximaciones de activaciones reducen accuracy
   - Típicamente 1-5% pérdida de accuracy vs float32

Idea Clave

Direcciones de investigación para zkML:

1. **Modelos ZK-native:** Entrenar modelos optimizados para ZK desde el inicio
2. **Pruning y quantization:** Reducir tamaño del modelo manteniendo accuracy
3. **Hardware acceleration:** GPUs y ASICs para proving de ML
4. **Folding para ML:** Aplicar técnicas de folding (Nova) a capas

5.2 Verificación Formal de Circuitos

5.2.1 Motivación: El Problema de Soundness

Definición: Soundness en Circuitos ZK

Un circuito es **sound** si solo acepta witnesses válidos. Un circuito **unsound** acepta witnesses que no deberían ser válidos, permitiendo generar pruebas falsas.

Observación

Bugs de soundness son catastróficos:
Si un circuito tiene un bug que permite crear pruebas falsas, un atacante puede:

- Generar dinero de la nada en un L2
- Probar transacciones que nunca ocurrieron
- Bypassear toda la seguridad del sistema

A diferencia de bugs en smart contracts (donde al menos el estado es visible), bugs en circuitos ZK pueden ser explotados **sin dejar rastro**.

Nota Matemática

Historial de Bugs en Circuitos:

- **Zcash (2019):** Bug de inflación potencial en circuito Sprout
- **Tornado Cash (2022):** Under-constraint permitía doble gasto
- **Circom bugs varios:** Múltiples under-constraints en circomlib
- **zkSync (2023):** Bug en verificador que fue parcheado

5.2.2 Tipos de Bugs en Circuitos

Definición: Under-Constrained Circuit

Un circuito está **under-constrained** cuando existen múltiples witnesses válidos para el mismo set de inputs públicos, pero solo algunos representan computaciones legítimas.
Ejemplo:

```
// Bug: under-constrained
template BadSquare() {
    signal input x;
    signal output y;

    y <-- x * x;  // Solo asigna, no constra\~na
    // Falta: y === x * x;
}
```

En este caso, el prover puede asignar cualquier valor a y, no necesariamente x^2.

Definición: Over-Constrained Circuit

Un circuito está **over-constrained** cuando rechaza witnesses válidos, o es directamente insatisfacible.

Definición: Constraint Confusion

Confusión entre asignación (`<-`) y constraint (`===`):

```
// CORRECTO
y <== x * x;  // Asigna Y constra\~na

// INCORRECTO (bug com\'un)
y <-- x * x;  // Solo asigna, NO constra\~na
```

5.2.3 Verificación Formal: Conceptos

Definición: Verificación Formal

Proceso de probar matemáticamente que un circuito satisface su especificación usando:

- **Model checking:** Exploración exhaustiva del espacio de estados
- **Theorem proving:** Demostración matemática de propiedades
- **SMT solving:** Satisfiability Modulo Theories

Intuición

Testing prueba que "para estos inputs, el circuito funciona".
Verificación formal prueba que "para TODOS los inputs posibles, el circuito funciona".
Es la diferencia entre probar 1 millón de casos vs probar infinitos casos.

Propiedades a Verificar

Definición: Propiedades de Correctness

1. **Completeness:** Para todo input válido, existe un witness que satisface el circuito

$$\forall x \in \text{Valid} : \exists w : C(x, w) = 1$$

2. **Soundness:** Solo inputs válidos tienen witnesses que satisfacen el circuito

$$\forall x, w : C(x, w) = 1 \implies x \in \text{Valid}$$

3. **Determinism:** Para cada input público, hay exactamente un output válido

5.2.4 Herramientas de Verificación Formal

Definición: Ecne

Verificador formal para circuitos R1CS:

- **Input:** Circuito R1CS + especificación
- **Método:** SMT solving (Z3, CVC5)
- **Output:** Prueba de correctness o contraejemplo

Definición: Picus

Verificador que detecta under-constraints automáticamente:

- **Análisis estático:** Sin ejecutar el circuito
- **Detección de under-constraints:** Señales que pueden tomar múltiples valores
- **Visualización:** Muestra qué señales son problemáticas

Definición: Circomspect

Linter y analizador estático para Circom:

- **Detección de patrones:** Anti-patterns comunes
- **Under-constraint detection:** Señales sin suficientes constraints
- **Side-effect analysis:** Asignaciones sin constraints correspondientes

5.2.5 Best Practices para Circuitos Seguros

Idea Clave

Reglas de oro para circuitos seguros:

1. **Siempre usar <== en lugar de <-**
   - <- es solo para casos especiales donde el constraint viene después
   - Cuando uses <-, documenta POR QUÉ y dónde está el constraint
2. **Constrañir todos los inputs a sus rangos válidos**

```
// Malo: bit no constrained
signal input bit;
out <== x + bit * y;

// Bueno: verificar que es 0 o 1
signal input bit;
bit * (1 - bit) === 0;
out <== x + bit * y;
```

3. **Nunca confiar en el prover**

- El prover es potencialmente malicioso
- Toda señal que afecte el resultado debe ser constrained

4. **Usar templates auditados**
   - circomlib tiene bugs históricos — usar versiones actualizadas
   - Preferir templates ampliamente usados y auditados
5. **Testing no es suficiente**
   - Un test pasa con witness honesto
   - Un atacante usa witness deshonesto
   - Verificación formal detecta lo que testing no puede

5.3 Optimización de Circuitos

5.3.1 Métricas de Eficiencia

Definición: Métricas de Circuitos

- **Número de constraints:** Principal métrica para R1CS/Plonk
- **Número de multiplicaciones:** Solo multiplicaciones cuestan constraints
- **Profundidad del circuito:** Afecta paralelización
- **Número de señales:** Afecta memoria del prover
- **Tiempo de proving:** Métrica final

Nota Matemática

Complejidad de Proving:
Para un circuito con n constraints:

- **Groth16:** Proving $O(n \log n)$, Verificación $O(1)$
- **PLONK:** Proving $O(n \log n)$, Verificación $O(\log n)$
- **STARKs:** Proving $O(n \log^2 n)$, Verificación $O(\log^2 n)$

Reducir n a la mitad aproximadamente reduce tiempo de proving a la mitad.

5.3.2 Técnicas de Optimización

Reducción de Multiplicaciones

Idea Clave

En R1CS, solo las multiplicaciones generan constraints. Las sumas son "gratis".
R1CS: $A \cdot B = C$ donde A, B, C son combinaciones lineales.
Si puedes reescribir tu computación usando menos multiplicaciones, reduces constraints.

Ejemplo: Optimización de Polinomio

Calcular $y = x^3 + 2x^2 + 3x + 4$
Método naive (3 multiplicaciones):

```
x2 <== x * x;        // Mult 1
x3 <== x2 * x;       // Mult 2
y <== x3 + 2*x2 + 3*x + 4;  // Sin mult
```

Método Horner (2 multiplicaciones):

$$y = ((x + 2) \cdot x + 3) \cdot x + 4$$

```
t1 <== (x + 2) * x;      // Mult 1
t2 <== (t1 + 3) * x;     // Mult 2
y <== t2 + 4;            // Sin mult
```

Ahorro: 33% menos constraints

Lookup Arguments

Definición: Lookup Argument

Técnica que permite verificar que un valor pertenece a una tabla predefinida con costo constante (o logarítmico), en lugar de calcularlo explícitamente.

Intuición

En lugar de calcular $y = \sqrt{x}$ en el circuito (muy costoso), pre-computas una tabla con todos los valores posibles de x y $\sqrt{x}$.
Luego el circuito solo verifica: "(x, y) está en la tabla".
Es como buscar en un diccionario vs derivar la definición desde primeros principios.

Ejemplo: ReLU con Lookup

Sin lookup (254 constraints para ReLU):
Necesitas descomponer en bits para determinar signo.
Con lookup (3-5 constraints):
El circuito solo verifica membership en la tabla ReLU.
Ahorro: 50-100x menos constraints para operaciones no-lineales

Nota Matemática

Plookup:
Plookup es el lookup argument más usado:

- **Setup:** Tabla T de tamaño $|T|$
- **Proving:** $O(|T| \log |T|)$ para n lookups
- **Constraint overhead:** $\sim$3 constraints por lookup
- **Limitación:** Tabla debe ser conocida en compile time

Ideal para: XOR, AND, range checks, funciones no-lineales con dominio finito

Custom Gates (PLONKish)

Definición: Custom Gate

En sistemas PLONKish (PLONK, Halo2), puedes definir constraints personalizados más allá de la multiplicación simple:

$$q_L \cdot a + q_R \cdot b + q_O \cdot c + q_M \cdot ab + q_C = 0$$

Los q_i son "selectores" que activan/desactivan partes del gate.

Ejemplo: Custom Gate para Poseidon

Poseidon hash usa la operación x^5 (quintic) frecuentemente.
Con gates estándar: 3 gates, 3 rows
Con custom gate para x^5: 1 gate, 1 row
Ahorro: 66% menos rows para cada x^5

Selección de Campo

Definición: Campo Óptimo

El campo finito $\mathbb{F}_p$ afecta la eficiencia:

- **Tamaño de p:** Campos más pequeños = aritmética más rápida
- **Estructura de p:** $p = 2^k - c$ (Mersenne-like) es más eficiente
- **Compatibilidad:** Debe ser compatible con curvas elípticas usadas

Campo	Bits	Estructura	Uso
BN254 scalar	254	Genérico	Ethereum, Groth16
BLS12-381 scalar	255	Genérico	Zcash, many SNARKs
Pasta (Pallas/Vesta)	255	Genérico	Halo2, 2-cycles
Goldilocks	64	$2^{64} - 2^{32} + 1$	Plonky2 (muy rápido)
BabyBear	31	$2^{31} - 2^{27} + 1$	Risc Zero
M31 (Mersenne31)	31	$2^{31} - 1$	STARKs, Circle

Idea Clave

Trend actual: Campos más pequeños (64 bits o menos) son mucho más rápidos. Plonky2/Plonky3 usan Goldilocks (64 bits) y son ∼100x más rápidos que sistemas sobre BN254.
Trade-off: Campos pequeños tienen menor seguridad contra ataques algebraicos, requieren más rondas/repeticiones.

5.3.3 Hardware Acceleration

Definición: Aceleración por Hardware

- **GPU:** 10-100x speedup para MSM y FFT
- **FPGA:** Custom circuits para operaciones específicas
- **ASIC:** Máxima eficiencia, menor flexibilidad

Benchmarks típicos (2^{20} puntos MSM):

- CPU (Ryzen 9): ∼15 segundos
- GPU (RTX 3090): ∼0.3 segundos
- Speedup: ∼50x

5.4 Proof Aggregation

5.4.1 Motivación

Definición: Proof Aggregation

Técnica para combinar múltiples pruebas ZK en una sola prueba más pequeña, reduciendo el costo de verificación.

Intuición

Imagina que tienes 1000 pruebas, cada una cuesta \$0.50 verificar on-chain. Costo total: \$500.
Con agregación, combinas las 1000 pruebas en una sola. Costo total: \$0.50 (más el costo de agregar off-chain).
Es como enviar un camión lleno de paquetes en lugar de 1000 entregas individuales.

Idea Clave

Beneficios de la agregación:

- **Costo:** Verificación on-chain es costosa. Agregar reduce verificaciones.
- **Throughput:** Un rollup puede procesar más transacciones por batch.
- **Latencia:** Menor data on-chain = confirmación más rápida.

Trade-off:

- Agregar tiene costo computacional off-chain
- Latencia de esperar por batch
- Complejidad del sistema

5.4.2 Métodos de Agregación

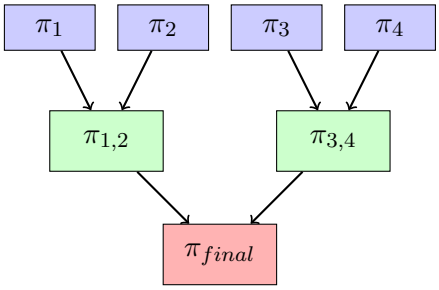
Recursive Proof Composition

Definición: Recursive Proof

Una prueba que verifica otra prueba dentro de su circuito:

$$\pi_{outer} = \text{Prove}(\text{“}V(\pi_{inner}) = 1\text{”})$$

El verificador del sistema interno se “embebe” en el circuito del sistema externo.



Nota Matemática

Complejidad de Agregación Recursiva:
Para n pruebas con árbol binario:

- Número de niveles: $\lceil \log_2 n \rceil$
- Número total de agregaciones: $n - 1$
- Tamaño final: constante (independiente de n)
- Tiempo de agregación: $O(n)$ trabajo total

Batching (sin recursión)

Definición: Proof Batching

Probar múltiples statements en un solo circuito, sin verificar pruebas existentes:

$$\pi_{batch} = \text{Prove}(x_1 \in L \land x_2 \in L \land ... \land x_n \in L)$$

SnarkPack

Definición: SnarkPack

Esquema de agregación para Groth16:

- Agrega n pruebas Groth16 en una sola
- Tamaño de prueba agregada: $O(\log n)$
- Verificación: $O(\log n)$
- No requiere recursión (más eficiente)

5.5 Sistemas Recursivos

5.5.1 Incrementally Verifiable Computation (IVC)

Definición: IVC

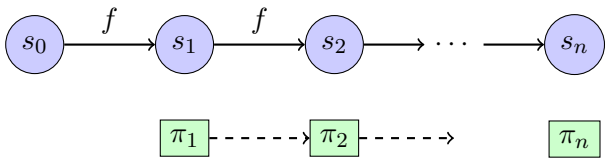
Sistema donde una computación de n pasos se puede verificar incrementalmente:

- Paso i: Prover genera π_i que prueba "los pasos $1..i$ son correctos"
- Paso $i+1$: Usando π_i, genera π_{i+1} con costo independiente de i

La verificación final solo necesita π_n, sin importar cuán grande sea n.

Intuición

Imagina una cadena de cálculos: $s_0 \to s_1 \to s_2 \to ... \to s_n$
Sin IVC: Verificar requiere re-ejecutar todos los pasos. Con IVC: La prueba final dice "confía que s_n es correcto", y verificarla es $O(1)$.
Es como un sello de calidad que se va pasando y acumulando confianza.



5.5.2 Proof-Carrying Data (PCD)

Definición: PCD

Generalización de IVC a grafos de computación (no solo cadenas):

- Múltiples "hilos" de computación
- Pueden unirse (merge) y dividirse (fork)
- Cada nodo verifica las pruebas de sus predecesores

5.5.3 Aplicaciones de IVC/PCD

Blockchains Sucintas (Mina Protocol)

Definición: Mina Protocol

Blockchain donde el estado completo es una sola prueba ZK:

- Cada bloque genera prueba IVC: "los bloques $0..n$ son válidos"
- Sincronizar = descargar una prueba de $\sim$22KB
- No necesitas descargar toda la historia

5.5.4 Folding Schemes

Definición: Folding

Técnica para "combinar" dos instancias de un problema en una sola, sin generar una prueba completa:

$$\text{Fold}(\text{instance}_1, \text{instance}_2) \to \text{instance}_{combined}$$

Más eficiente que recursión completa porque evita la verificación completa en cada paso.

Intuición

Recursión tradicional: En cada paso, generas una prueba completa que verifica la prueba anterior. Esto es costoso.
Folding: En cada paso, solo "combinas" las instancias. Al final, generas UNA prueba de la instancia combinada.
Es como acumular trabajo pendiente y hacerlo todo al final, en lugar de completar cada tarea individualmente.

Nova

Definición: Nova

Esquema de folding para R1CS:

- Fold: $O(n)$ operaciones de campo (sin FFT, sin MSM)
- Final proof: Una prueba Groth16/Spartan al final
- Muy eficiente para IVC

Nota Matemática

Cómo funciona Nova:
R1CS: $Az \circ Bz = Cz$
Relaxed R1CS: $Az \circ Bz = u \cdot Cz + E$
Donde:

- u es un escalar ("relaxation factor")
- E es un "error vector"

Folding: Dadas dos instancias (u_1, E_1, z_1) y (u_2, E_2, z_2):

$$u' = u_1 + r \cdot u_2, \quad E' = E_1 + r \cdot T + r^2 \cdot E_2, \quad z' = z_1 + r \cdot z_2$$

Donde r es un challenge aleatorio y T es un "cross-term".

SuperNova y HyperNova

Definición: SuperNova

Extensión de Nova para circuitos con múltiples "opcodes":

- Cada paso puede ejecutar uno de k circuitos diferentes
- Ideal para zkVMs (cada instrucción es un circuito)
- Solo paga por el circuito más grande, no la suma

Definición: HyperNova

Generalización de Nova a CCS (Customizable Constraint Systems):

- Soporta Plonkish, AIR, R1CS
- Folding más general
- Puede ser más eficiente para algunos circuitos

Esquema	Sistema Base	Costo por Fold	Aplicación
Nova	R1CS	$O(n)$	IVC general
SuperNova	R1CS (multi)	$O(n)$	zkVMs
HyperNova	CCS	$O(n)$	Plonkish IVC
Sangria	Plonk	$O(n \log n)$	Plonk native

5.6 Post-Quantum ZK

5.6.1 La Amenaza Cuántica

Definición: Computación Cuántica y Criptografía

Las computadoras cuánticas pueden ejecutar algoritmos que rompen la seguridad de muchos sistemas criptográficos actuales:

- **Algoritmo de Shor:** Factoriza enteros y calcula logaritmos discretos en tiempo polinomial
- **Algoritmo de Grover:** Búsqueda en $O(\sqrt{N})$ en lugar de $O(N)$

Observación

Impacto en sistemas ZK actuales:

- **Groth16, PLONK (con KZG):** Basados en curvas elípticas y pairings
  - Shor rompe el problema del logaritmo discreto
  - Completamente inseguros ante computadoras cuánticas
- **Bulletproofs:** Basados en logaritmo discreto
  - Completamente inseguros ante computadoras cuánticas

- **STARKs:** Basados solo en funciones hash
  - Grover reduce seguridad a la mitad (256-bit → 128-bit efectivo)
  - Seguro con parámetros adecuados

Nota Matemática

Timeline Estimado de Amenaza Cuántica:

- **2024:** Computadoras cuánticas con ∼1,000 qubits (ruidosos)
- **2030:** Posibles ∼10,000+ qubits con corrección de errores
- **2035-2040:** Potencialmente capaces de romper RSA-2048 / ECDSA

Incertidumbre alta, pero el consenso es prepararse **ahora**:

- "Harvest now, decrypt later" — Adversarios pueden guardar datos cifrados
- Migración de sistemas toma años
- NIST ya estandarizó algoritmos post-cuánticos (2024)

5.6.2 Análisis de Seguridad por Sistema

Sistema	Base Cripto	Post-Q	Proof Size	Notas
Groth16	Pairings (EC)	No	128 bytes	Más compacto
PLONK + KZG	Pairings (EC)	No	400-800 bytes	Universal
PLONK + IPA	Log discreto	No	1-2 KB	Sin trusted setup
Bulletproofs	Log discreto	No	0.7-1.5 KB	Range proofs
STARKs	Hash functions	Sí	50-200 KB	Transparente
FRI-based	Hash + Reed-Solomon	Sí	50-200 KB	Plonky2/3
Lattice SNARKs	Lattices (LWE)	Sí	10-50 KB	En desarrollo

5.6.3 STARKs: La Opción Post-Quantum Actual

Definición: STARK Security

La seguridad de STARKs se basa únicamente en:

1. **Collision resistance de hash:** Difícil encontrar $x \neq y$ con $H(x) = H(y)$
2. **Random oracle model:** Hash se comporta como función aleatoria

No depende de logaritmo discreto, factorización, ni pairings.

Nota Matemática

Impacto de Grover en STARKs:
Grover reduce la seguridad de búsqueda/colisión:

- Clásico: Encontrar colisión en hash de n bits requiere $O(2^{n/2})$
- Cuántico (Grover): Requiere $O(2^{n/3})$ (para colisiones)

Para mantener 128 bits de seguridad post-cuántica:

- Usar hash de al menos 256 bits
- STARKs típicamente usan 256+ bits de seguridad
- Pruebas más grandes, pero seguras

5.6.4 Lattice-Based SNARKs

Definición: Lattice Problems

Problemas computacionales en lattices (retículas) que se creen resistentes a ataques cuánticos:

- **LWE (Learning With Errors):** Distinguir $As + e$ de aleatorio
- **SIS (Short Integer Solution):** Encontrar vector corto x con $Ax = 0$
- **Ring-LWE:** Versión estructurada de LWE (más eficiente)

Nota Matemática

Por qué Lattices son Post-Quantum:

- No hay algoritmo cuántico conocido que resuelva LWE/SIS eficientemente
- Los mejores ataques (clásicos y cuánticos) son exponenciales
- Base de los estándares NIST post-cuánticos (Kyber, Dilithium)

Observación

Limitaciones actuales de Lattice SNARKs:

- Pruebas más grandes que SNARKs basados en curvas (∼10-100x)
- Proving más lento
- Menos maduros (menos implementaciones, menos auditorías)
- Parámetros de seguridad aún en debate

STARKs son la opción práctica hoy. Lattice SNARKs son para el futuro.

5.6.5 Estrategias de Migración

Idea Clave

Preparación para era post-cuántica:

1. **Híbrido:** Usar SNARKs + firma post-cuántica
   - SNARK para eficiencia (puede ser comprometido en el futuro)
   - Firma lattice/hash para seguridad a largo plazo

2. **Migración gradual a STARKs:**
   - STARKs para nuevos sistemas
   - Wrap SNARKs existentes en STARKs
3. **Crypto-agility:**
   - Diseñar sistemas para cambiar primitivas fácilmente
   - Evitar hard-coding de algoritmos específicos

5.7 Resumen del Capítulo

Idea Clave

Puntos clave de Avanzado e Investigación:

1. **zkML:**
   - ML verificable con pruebas ZK
   - Desafíos: fixed-point, activaciones no-lineales, tamaño de modelos
   - Frameworks: EZKL (PyTorch → Halo2), Giza (STARKs)
   - Aplicaciones: on-chain AI, proof of personhood, credit scoring privado
   - Limitación actual: solo modelos pequeños (<1M params)
2. **Verificación Formal:**
   - Crucial para seguridad — testing no es suficiente
   - Under-constraints: bug más común y peligroso
   - Herramientas: Circomspect (linting), Picus (análisis), Ecne (SMT)
   - Best practice: Siempre usar `<==`, constrañir rangos, auditoría múltiple
3. **Optimización de Circuitos:**
   - Reducir multiplicaciones (único costo en R1CS)
   - Lookups: 50-100x mejora para operaciones no-lineales
   - Custom gates: optimizaciones específicas por operación
   - Hardware: GPU 50x speedup para MSM
   - Campos pequeños (Goldilocks): 100x más rápido que BN254
4. **Proof Aggregation:**
   - Combinar múltiples pruebas en una
   - Recursive: verificar prueba dentro de circuito
   - Batching: probar múltiples statements juntos
   - SnarkPack: agregación específica para Groth16

5. **Sistemas Recursivos:**
   - IVC: computación verificable incremental
   - PCD: generalización a grafos de computación
   - Mina: blockchain sucinta (22KB para sincronizar)
   - Folding (Nova): más eficiente que recursión completa
6. **Post-Quantum:**
   - SNARKs basados en curvas: inseguros ante quantum
   - STARKs (hash-based): seguros, opción práctica hoy
   - Lattice SNARKs: seguros, en investigación
   - Estrategia: migrar gradualmente a STARKs, crypto-agility

Tema	Estado 2025	Perspectiva
zkML	Modelos pequeños viables	Mejorando rápidamente
Verificación formal	Herramientas maduras	Estándar de la industria
Optimización	Lookups, campos pequeños	Hardware especializado
Agregación	Producción (rollups)	Universal
Folding	Nova/SuperNova en uso	Estándar para IVC
Post-quantum	STARKs listos	Lattices en 5-10 años

🏛 Aplicación en Banco ZK

Banco ZK — Visión de Investigación:
Banco ZK invierte en investigación para mantenerse en la frontera:

1. **zkML para compliance:**
   - Modelos de detección de fraude en ZK
   - Credit scoring privado
   - Meta: compliance sin surveillance
2. **Verificación formal:**
   - Todos los circuitos críticos verificados formalmente
   - CI/CD con análisis estático obligatorio
   - Bug bounty permanente
3. **Optimización continua:**
   - Migración a Plonky3 para throughput
   - GPU proving para sequencer
   - Objetivo: 10,000+ TPS
4. **Preparación post-quantum:**
   - Roadmap de migración a STARKs

- Monitoreo de avances en lattices
- Crypto-agility en diseño

Presupuesto anual de R&D: 15-20% de treasury

Chapter 6

Protocolos de Anonimización

Introducción

La anonimización es el proceso de ocultar la identidad de los participantes en una comunicación o transacción. Este capítulo explora los protocolos fundamentales que permiten privacidad en sistemas digitales, desde Bitcoin mixing hasta redes de comunicación anónima.

Idea Clave

Niveles de anonimato:

- **Pseudonimato:** Identidad oculta tras pseudónimo (direcciones Bitcoin básicas)
- **Unlinkability:** No se pueden vincular diferentes acciones del mismo usuario
- **Untraceability:** No se puede rastrear el origen de una acción
- **Anonimato perfecto:** Información-teóricamente imposible de identificar

Nota Matemática

Timeline de Protocolos de Anonimización:

- **1981:** David Chaum propone DC-nets
- **1982:** Chaum inventa blind signatures y e-cash
- **1996:** Onion routing desarrollado en Naval Research Lab
- **2002:** Tor Project iniciado
- **2003:** I2P (Invisible Internet Project) lanzado
- **2013:** CoinJoin propuesto por Greg Maxwell
- **2014:** CoinShuffle publicado
- **2017:** Dandelion propuesto para Bitcoin
- **2022:** Fedimint y Cashu reviven e-cash sobre Bitcoin

6.1 CoinJoin (Bitcoin Mixing)

6.1.1 El Problema de Privacidad en Bitcoin

Definición: Bitcoin Transaction Graph

Bitcoin mantiene un grafo público de todas las transacciones:

- Cada transacción tiene inputs (UTXOs gastados) y outputs (nuevos UTXOs)
- Cada UTXO es rastreable hasta su creación (coinbase)
- Análisis de grafos puede vincular direcciones a identidades

Intuición

Bitcoin es **pseudónimo**, no anónimo. Es como escribir bajo un seudónimo: si alguien descubre que "Satoshi Nakamoto" eres tú, todo lo que escribiste bajo ese nombre queda vinculado a ti.
Las empresas de análisis de blockchain (Chainalysis, Elliptic) especializan en "de-anonimizar" direcciones Bitcoin vinculándolas a exchanges (KYC), patrones de gasto, y metadata de red.

Observación

Vectores de identificación en Bitcoin:

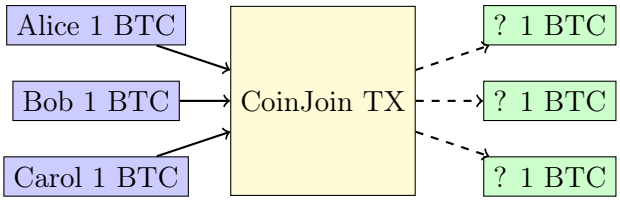
1. **Address reuse:** Usar la misma dirección múltiples veces
2. **Change outputs:** El "vuelto" revela información
3. **Input clustering:** Inputs en misma tx probablemente del mismo dueño
4. **Timing analysis:** Patrones temporales de transacciones
5. **Amount correlation:** Montos específicos que coinciden
6. **Network analysis:** IP del nodo que broadcast la tx

6.1.2 CoinJoin: Concepto Básico

Definición: CoinJoin

Técnica de privacidad donde múltiples usuarios combinan sus transacciones en una sola transacción con múltiples inputs y outputs:

- **Inputs:** UTXOs de diferentes usuarios
- **Outputs:** Direcciones de destino mezcladas
- **Firma:** Cada usuario firma solo sus inputs
- **Resultado:** Observador no puede determinar qué input corresponde a qué output



¿Quién recibe cada output?
Imposible determinar solo viendo la tx

6.1.3 Análisis de Anonimato

Definición: Anonymity Set

El conjunto de posibles orígenes/destinos de una transacción. Cuanto mayor el anonymity set, mayor la privacidad.

$$\text{Anonymity Set} = n \implies \text{Probabilidad de identificación} = \frac{1}{n}$$

Nota Matemática

Entropía de Anonimato:
La entropía mide la incertidumbre del observador:

$$H = -\sum_{i=1}^{n} p_i \log_2(p_i)$$

Para CoinJoin con n participantes equiprobables:

$$H = -n \cdot \frac{1}{n} \log_2\left(\frac{1}{n}\right) = \log_2(n) \text{ bits}$$

Participantes	Entropía
2	1 bit
4	2 bits
8	3 bits
16	4 bits
100	6.64 bits

Para 128 bits de seguridad, necesitarías 2^{128} participantes — impracticable. Por eso CoinJoin se hace múltiples veces (remix).

6.1.4 Requisitos para CoinJoin Efectivo

Idea Clave

Condiciones para anonimato real:

1. **Outputs uniformes:** Todos los outputs deben tener el mismo monto
   - Si Alice pone 1.23 BTC y hay un output de 1.23 BTC, es obvio que es de

Alice
- Solución: "Equal-output CoinJoin" con denominaciones fijas (0.1, 0.01 BTC)

2. **Suficientes participantes:** Más participantes = mayor anonymity set
3. **Sin coordinador malicioso:** El coordinador no debe poder vincular inputs con outputs
4. **Múltiples rondas:** Una ronda da anonimato limitado, múltiples rondas lo multiplican
5. **Timing uniforme:** Participantes deben entrar/salir en momentos similares

6.1.5 Implementaciones de CoinJoin

Definición: Wasabi Wallet

Wallet Bitcoin con CoinJoin integrado usando protocolo WabiSabi:

- **Coordinador centralizado:** Pero no puede vincular (Chaumian blind signatures)
- **Denominaciones variables:** WabiSabi permite outputs de diferentes tamaños
- **Fees:** 0.3% del monto mezclado
- **Participantes:** Típicamente 100-150 por ronda

Definición: JoinMarket

Mercado descentralizado para CoinJoin:

- **Makers:** Ofrecen liquidez para CoinJoin, ganan fees
- **Takers:** Pagan fees para mezclarse con makers
- **Sin coordinador central:** P2P entre participantes
- **Más privacidad:** Pero menos conveniente

Definición: Whirlpool (Samourai)

Implementación de CoinJoin con énfasis en simplicidad:

- **Pools fijos:** 0.5, 0.05, 0.01, 0.001 BTC
- **Zero-link:** Entrada y salida completamente desvinculadas
- **Tx0:** Transacción preparatoria que divide UTXOs
- **Remixing gratuito:** Una vez dentro, remix sin costo adicional

Nota: En 2024, los fundadores de Samourai fueron arrestados por cargos de lavado de dinero, afectando la disponibilidad del servicio.

6.1.6 Ataques contra CoinJoin

Observación

Vectores de ataque:

1. **Amount correlation:**
   - Si los outputs no son uniformes, se pueden vincular por monto
   - Solución: Equal-output CoinJoin
2. **Timing analysis:**
   - Observar cuándo entran/salen fondos del mixer
   - Vincular por proximidad temporal
   - Solución: Delays aleatorios, múltiples rondas
3. **Sybil attack:**
   - Adversario controla múltiples participantes
   - Reduce el anonymity set real
   - Solución: Fees que hacen Sybil costoso
4. **Intersection attack:**
   - Observar múltiples CoinJoins del mismo usuario
   - Intersección de anonymity sets reduce opciones
   - Solución: No reusar direcciones, mezclar en diferentes pools
5. **Dust attack:**
   - Enviar pequeñas cantidades a direcciones conocidas
   - Si el usuario las gasta junto con fondos mezclados, vincula
   - Solución: No gastar dust, UTXOs separados

6.1.7 WabiSabi: CoinJoin Avanzado

Definición: WabiSabi

Protocolo de Wasabi Wallet que permite CoinJoin con montos variables usando credenciales anónimas:

- **Keyed Verification Anonymous Credentials (KVAC):** Permite demostrar propiedades sin revelar valores
- **Range proofs:** Prueban que montos están en rango válido
- **Sin denominación fija:** Más flexibilidad, mismo anonimato

Nota Matemática

Cómo funciona WabiSabi:
Fase 1 - Registro de inputs:

1. Usuario registra UTXO y recibe credencial por el monto
2. Credencial está "cegada" (coordinador no ve monto específico)

Fase 2 - Registro de outputs:

1. Usuario presenta credenciales (sin vincular a input)
2. Prueba que suma de outputs $\leq$ suma de credenciales
3. Registra direcciones de output

Fase 3 - Firma:

1. Coordinador crea transacción
2. Cada usuario firma sus inputs
3. Broadcast a la red

La magia está en que el coordinador nunca ve la conexión input↔output.

6.2 CoinShuffle / CoinShuffle++

6.2.1 El Problema del Coordinador

Intuición

En CoinJoin básico, hay un problema: alguien tiene que recolectar las direcciones de output de todos los participantes. Si ese "coordinador" es malicioso, puede vincular inputs con outputs.
CoinShuffle resuelve esto: los participantes mezclan las direcciones entre ellos, de forma que **nadie** (ni siquiera otros participantes) sabe qué dirección pertenece a quién.

6.2.2 Protocolo CoinShuffle

Definición: CoinShuffle

Protocolo de mezcla descentralizado usando encriptación por capas:

1. Cada participante tiene un par de claves de encriptación (pk_i, sk_i)
2. Los mensajes (direcciones de output) se encriptan en capas
3. Cada participante remueve una capa y reordena aleatoriamente
4. Al final, las direcciones están mezcladas y nadie sabe de quién es cuál

Nota Matemática

Protocolo Detallado:
Setup:

- n participantes $P_1, ..., P_n$
- Cada uno genera par de claves: (pk_i, sk_i)
- Orden de participantes acordado: $P_1 \to P_2 \to ... \to P_n$

Fase 1 - Encriptación:
Cada participante P_i encripta su dirección $addr_i$ con las claves públicas de todos los participantes después de él:

$$m_i = E_{pk_n}(E_{pk_{n-1}}(...E_{pk_{i+1}}(addr_i)...))$$

P_i envía m_i a P_{i+1}.
Fase 2 - Shuffle:
Cada participante P_j (para $j = 2, ..., n$):

1. Recibe lista de mensajes cifrados del anterior
2. Descifra una capa de cada mensaje usando sk_j
3. Permuta aleatoriamente la lista
4. Envía al siguiente participante

Fase 3 - Output:
P_n termina con lista de direcciones en claro, pero permutadas aleatoriamente. Nadie sabe qué dirección es de quién.

6.2.3 CoinShuffle++

Definición: CoinShuffle++

Versión mejorada de CoinShuffle con:

- **DiceMix:** Protocolo de mezcla más eficiente usando DC-nets
- **Blame protocol mejorado:** Identificar participantes maliciosos
- **Menor latencia:** Menos rondas de comunicación
- **Resistencia a DoS:** Mejor manejo de participantes que abandonan

6.3 Chaumian e-cash

6.3.1 Historia y Concepto

Definición: e-cash (David Chaum, 1982)

Sistema de dinero digital anónimo donde:

- Un **banco/mint** emite tokens firmados
- Los tokens son **bearer instruments** (quien tiene el token, tiene el valor)
- El banco **no puede vincular** emisión con gasto gracias a blind signatures

Nota Matemática

Impacto Histórico:
David Chaum fundó DigiCash en 1990, implementando e-cash. El sistema fue técnicamente exitoso pero falló comercialmente. DigiCash quebró en 1998.
Sin embargo, las ideas de Chaum influyeron directamente en:

- Bitcoin (anonimato parcial)
- Monero (ring signatures derivadas de trabajos relacionados)
- Zcash (zero-knowledge para privacidad)
- Y ahora: Fedimint, Cashu (e-cash sobre Bitcoin)

6.3.2 Blind Signatures

Definición: Blind Signature

Esquema de firma digital donde el firmante no ve el mensaje que firma:

1. **Blind:** Usuario "ciega" el mensaje m para obtener m'
2. **Sign:** Firmante firma m' para obtener σ'
3. **Unblind:** Usuario "desciega" σ' para obtener firma válida σ sobre m

El firmante nunca ve m, pero σ es una firma válida de su clave sobre m.

Nota Matemática

RSA Blind Signatures:
Usando RSA con clave pública (e, n) y privada d:
Blind:

1. Usuario elige r aleatorio coprimo con n
2. Calcula mensaje cegado: $m' = m \cdot r^e \mod n$

Sign:

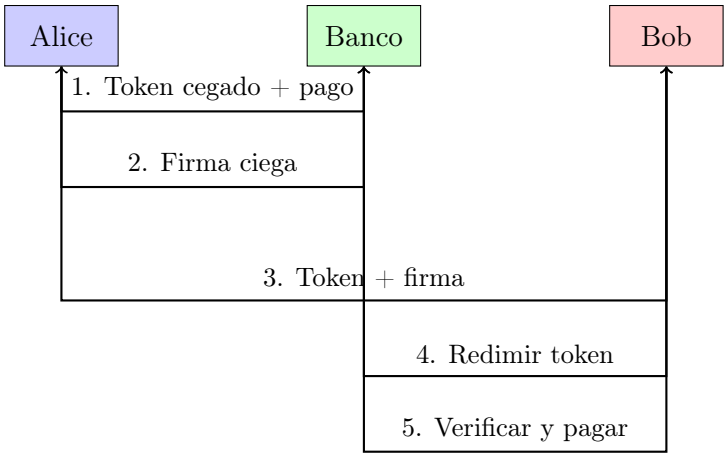
1. Banco firma: $\sigma' = (m')^d = (m \cdot r^e)^d = m^d \cdot r \mod n$

Unblind:

1. Usuario calcula: $\sigma = \sigma' \cdot r^{-1} = m^d \cdot r \cdot r^{-1} = m^d \mod n$

$\sigma = m^d$ es una firma RSA válida sobre m, pero el banco nunca vio m.

6.3.3 Protocolo e-cash Completo



Propiedad: Banco no puede vincular emisión (paso 2) con redención (paso 4)

6.3.4 Prevención de Doble Gasto

Observación

El problema: Un token digital puede ser copiado. ¿Cómo prevenir que Alice gaste el mismo token dos veces?
Solución centralizada: El banco mantiene lista de tokens gastados.

- Al recibir token, banco verifica que no está en la lista
- Si es nuevo, lo añade a la lista y paga
- Si está en la lista, rechaza

Limitación: Requiere banco online para cada transacción.

6.3.5 Resurgimiento: Fedimint y Cashu

Definición: Fedimint

E-cash federado construido sobre Bitcoin:

- **Federación:** Múltiples "guardianes" controlan los fondos (multisig)
- **Chaumian e-cash:** Tokens ciegos para privacidad dentro de la mint
- **Respaldo Bitcoin:** 1:1 respaldado por BTC en custodia
- **Lightning integration:** Puede recibir/enviar via Lightning

Definición: Cashu

Implementación simple de e-cash sobre Lightning Network:

- **Mint centralizada:** Un operador (más simple que Fedimint)
- **Tokens sobre Lightning:** Depósito/retiro vía Lightning
- **Privacidad local:** Transacciones entre usuarios de la misma mint son privadas
- **Open source:** Múltiples implementaciones

Característica	Fedimint	Cashu
Custodia	Federada (multisig)	Centralizada
Complejidad	Alta	Baja
Privacidad	Alta	Alta
Resistencia censura	Alta (federación)	Baja (un operador)
Madurez	Beta	Producción
Interoperabilidad	Lightning, on-chain	Lightning

6.4 Dandelion / Dandelion++

6.4.1 El Problema de la Capa de Red

Intuición

Aunque uses CoinJoin o mixing, hay un problema: cuando tu nodo envía una transacción a la red Bitcoin, otros nodos pueden observar **de dónde vino** la transacción.
Si un adversario controla muchos nodos de la red, puede triangular el origen de una transacción basándose en qué nodo la vio primero y cuándo.
Dandelion resuelve esto oscureciendo el origen de la transacción a nivel de red.

Definición: Network-level Deanonymization

Técnicas para identificar el origen de una transacción observando la red P2P:

- **First-spy estimator:** El nodo que ve la tx primero está cerca del origen
- **Timing analysis:** Patrones temporales de propagación
- **Spy nodes:** Adversario opera muchos nodos para observar propagación

6.4.2 Protocolo Dandelion

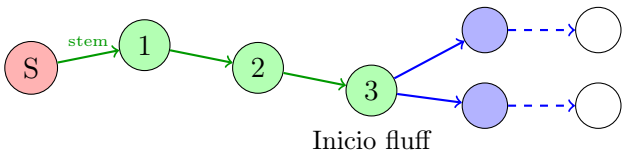
Definición: Dandelion

Protocolo de propagación en dos fases para ocultar el origen:
Fase 1 - Stem (Tallo):

- La tx se propaga en línea, un nodo a la vez
- Cada nodo pasa la tx a **un solo** peer
- Duración: ∼10 hops (configurable)

Fase 2 - Fluff (Pelusa):

- Después del stem, se hace difusión normal
- La tx ya está lejos del origen real
- Observadores piensan que el nodo de difusión es el origen



Verde: Fase Stem (línea)
Azul: Fase Fluff (difusión)

6.4.3 Dandelion++

Definición: Dandelion++

Versión mejorada de Dandelion que resiste ataques más sofisticados:
Mejoras:

1. **4-regular graph para stem:** Cada nodo tiene exactamente 2 "outbound stem peers"
2. **Decisión per-nodo, no per-tx:** El mismo nodo siempre envía a los mismos stem peers
3. **Embargo timer:** Si la tx no se ve en la red después de un tiempo, hacer fluff
4. **Failsafe diffusion:** Evitar que tx se pierda en nodos maliciosos

6.5 Dining Cryptographers Problem (DC-nets)

6.5.1 El Problema Original

Definición: Dining Cryptographers Problem (David Chaum, 1988)

Tres criptógrafos cenan juntos. Al final, la cuenta está pagada. Quieren saber: ¿pagó uno de ellos, o pagó la NSA (un tercero)?
Requisitos:

- Si uno de ellos pagó, los otros no deben saber quién
- Si ninguno pagó, todos deben saber que fue la NSA

Intuición

Es un problema de **anonimato perfecto**: el que pagó quiere decir "yo pagué" sin que nadie sepa quién es "yo".
La solución usa XOR y secretos compartidos para crear un canal de comunicación donde el mensaje aparece "de la nada" — sin revelar quién lo envió.

6.5.2 Protocolo DC-net

Nota Matemática

Protocolo para 3 participantes:
Setup:

- Cada par de criptógrafos comparte un bit secreto
- s_{AB} entre Alice y Bob
- s_{BC} entre Bob y Carol
- s_{AC} entre Alice y Carol

Protocolo:

1. Cada participante XORea sus secretos compartidos
2. Si quiere enviar "1" (yo pagué), también XORea con 1
3. Publica el resultado

Alice calcula: $a = s_{AB} \oplus s_{AC} \oplus m_A$
Bob calcula: $b = s_{AB} \oplus s_{BC} \oplus m_B$
Carol calcula: $c = s_{AC} \oplus s_{BC} \oplus m_C$
Verificación: $a \oplus b \oplus c = m_A \oplus m_B \oplus m_C$
Cada secreto aparece dos veces y se cancela. Solo quedan los mensajes.

Observación

Limitaciones de DC-nets:

1. **Escalabilidad:** $O(n^2)$ secretos compartidos para n participantes

2. **Bandwidth:** Cada participante envía $O(n)$ bits por mensaje de 1 bit
3. **Disruptors:** Un participante malicioso puede enviar ruido, bloqueando comunicación
4. **Sincronización:** Todos deben participar en cada round
5. **Setup:** Requiere establecer secretos compartidos de antemano

DC-nets proveen anonimato **información-teórico** (perfecto), pero son impráctricos a escala. Se usan como base teórica para protocolos más prácticos.

6.6 Mix Zones

6.6.1 Contexto: Privacidad de Ubicación

Definición: Location Privacy

Protección contra el rastreo de la ubicación física de un usuario a través del tiempo. Amenazas:

- **Tracking:** Seguir movimientos de un individuo
- **Profiling:** Inferir hábitos, trabajo, relaciones
- **Correlation:** Vincular identidades en diferentes contextos

Intuición

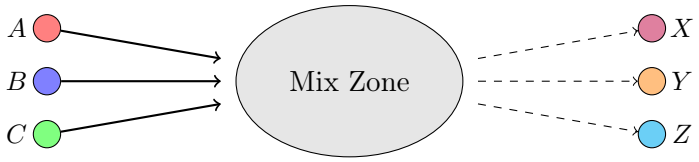
Imagina que llevas un dispositivo GPS que reporta tu ubicación. Aunque uses pseudónimos diferentes, un observador puede vincularlos viendo que "usuario A" termina donde "usuario B" comienza.
Mix Zones son como "zonas de cambio de disfraz": entras como Alice, sales como Bob, y nadie puede vincular Alice con Bob.

6.6.2 Concepto de Mix Zone

Definición: Mix Zone

Área geográfica donde:

1. No hay tracking activo (o está encriptado)
2. Múltiples usuarios entran y salen
3. Al salir, cada usuario tiene un nuevo pseudónimo
4. Observador no puede vincular entrada con salida



$¿A \to X?$ $¿A \to Y?$ $¿A \to Z?$
Observador no puede determinar

6.6.3 Aplicaciones

1. **Vehículos conectados:**
   - Coches autónomos intercambian datos constantemente
   - Mix zones en intersecciones, estacionamientos
   - Cambio de identificadores al salir
2. **IoT y smart cities:**
   - Sensores que rastrean dispositivos (WiFi, Bluetooth)
   - Zonas designadas donde se cambian identificadores
3. **Transporte público:**
   - Estaciones como mix zones naturales
   - Tarjetas de transporte con pseudónimos rotativos

6.7 Onion Routing

6.7.1 Concepto Fundamental

Definición: Onion Routing

Técnica de comunicación anónima donde los mensajes se encriptan en múltiples capas:

1. El mensaje se encripta con la clave del último nodo (exit)
2. Se envuelve con la clave del penúltimo nodo
3. Se envuelve con la clave del primer nodo (entry/guard)
4. Cada nodo "pela" una capa y reenvía al siguiente
5. Solo el exit node ve el destino final

Intuición

Es como enviar una carta dentro de muchos sobres:

- Sobre exterior dice: "Para Nodo 1"
- Nodo 1 abre y ve otro sobre: "Para Nodo 2"
- Nodo 2 abre y ve otro sobre: "Para Nodo 3"
- Nodo 3 abre y ve la carta real con el destino final

Ningún nodo sabe todo:

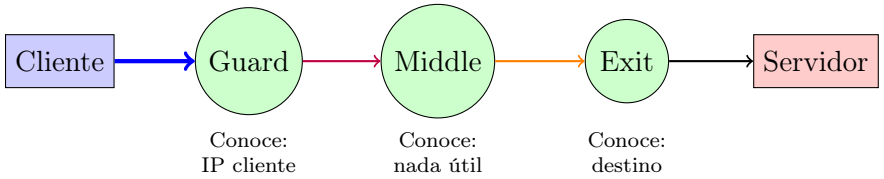
- Nodo 1 sabe quién envió, pero no el destino
- Nodo 3 sabe el destino, pero no quién envió
- Nodo 2 no sabe ni origen ni destino

6.7.2 Arquitectura de Tor

Definición: Tor (The Onion Router)

Red de anonimato más usada, implementando onion routing:

- **Guard nodes:** Primer nodo, conoce IP del usuario
- **Middle nodes:** Nodos intermedios, solo relay
- **Exit nodes:** Último nodo, conecta a Internet
- **Directory authorities:** Servidores que listan nodos disponibles



6.7.3 Ataques contra Tor

Observación

Ataques conocidos:

1. **Traffic correlation:**
   - Adversario observa entrada y salida del circuito
   - Correlaciona por timing y volumen
   - Efectivo si adversario controla ISPs en ambos extremos
2. **Sybil attack:**
   - Adversario opera muchos nodos
   - Probabilidad de controlar circuito completo: $(f)^3$ donde f = fracción de nodos
   - Mitigación: Guard nodes fijos por usuario
3. **Website fingerprinting:**
   - Patrones de tráfico revelan qué sitio se visita
   - ML puede clasificar con alta precisión
   - Mitigación: padding, traffic shaping
4. **Exit node malicioso:**

- Exit ve tráfico no encriptado (HTTP)
- Puede modificar contenido, robar credenciales
- Mitigación: Siempre usar HTTPS

6.8 Garlic Routing (I2P)

6.8.1 Diferencias con Onion Routing

Definición: Garlic Routing

Variante de onion routing usada en I2P (Invisible Internet Project):

- **Garlic cloves:** Múltiples mensajes empaquetados juntos
- **Tunnels unidireccionales:** Separados para entrada y salida
- **Packet-based:** En lugar de stream-based como Tor
- **Red interna:** Optimizado para servicios dentro de I2P

Intuición

Si onion routing es una cebolla (capas concéntricas), garlic routing es un bulbo de ajo: múltiples "dientes" (cloves) empaquetados juntos.
Cada clove puede ir a un destino diferente, lo que dificulta el análisis de tráfico.

6.8.2 Comparación Tor vs I2P

Característica	Tor	I2P
Modelo de routing	Onion (circuitos)	Garlic (tunnels)
Direccionalidad	Bidireccional	Unidireccional
Transporte	Stream (TCP-like)	Packet (UDP-like)
Uso principal	Acceso a Internet	Servicios internos
Exit nodes	Sí (a Internet)	Opcional (outproxies)
Latencia	Baja-Media	Media-Alta
Madurez	Alta (20+ años)	Media (18+ años)
Usuarios	∼2-3 millones	∼50,000
Servicios ocultos	.onion	.i2p (eepsites)

6.9 Comparativa de Protocolos

Protocolo	Tipo	Anonimato	Escala	Latencia	Uso
CoinJoin	Tx mixing	Computacional	Media	Minutos	Bitcoin
CoinShuffle	Tx mixing	Computacional	Media	Minutos	Bitcoin
E-cash	Token	Perfecto*	Alta	Segundos	Pagos
Dandelion	Red P2P	Heurístico	Alta	Segundos	Bitcoin
DC-nets	Broadcast	Info-teórico	Baja	Alta	Teórico
Mix Zones	Ubicación	Computacional	Media	Variable	IoT
Tor	Red	Computacional	Alta	Media	Internet
I2P	Red	Computacional	Media	Alta	Servicios

*E-cash es perfecto dentro de la mint.

6.10 Resumen del Capítulo

Idea Clave

Puntos clave de Protocolos de Anonimización:

1. **CoinJoin:**
   - Mezcla transacciones de múltiples usuarios en una
   - Requiere outputs uniformes para efectividad
   - Implementaciones: Wasabi (WabiSabi), JoinMarket, Whirlpool
   - Anonymity set limitado por participantes por ronda
2. **CoinShuffle:**
   - Coordinación descentralizada para CoinJoin
   - Usa encriptación por capas para ocultar direcciones
   - Nadie (ni otros participantes) sabe qué dirección es de quién
3. **Chaumian E-cash:**
   - Blind signatures: firmante no ve lo que firma
   - Tokens bearer: quien tiene el token, tiene el valor
   - Fedimint: e-cash federado sobre Bitcoin
   - Cashu: e-cash simple sobre Lightning
4. **Dandelion:**
   - Oculta origen de transacciones a nivel de red
   - Fase stem (línea) $\rightarrow$ fase fluff (difusión)
   - Complementa mixing a nivel de transacción
5. **DC-nets:**
   - Anonimato información-teórico (perfecto)
   - XOR de secretos compartidos
   - Impracticable a escala, base teórica
6. **Mix Zones:**
   - Privacidad de ubicación
   - Cambio de pseudónimo al atravesar zona
   - Aplicaciones: vehículos, IoT, transporte
7. **Onion Routing (Tor):**
   - Encriptación en capas, cada nodo pela una

- 3 hops: Guard, Middle, Exit
- Guard conoce IP, Exit conoce destino, ninguno sabe todo

8. **Garlic Routing (I2P):**
   - Múltiples mensajes empaquetados (cloves)
   - Tunnels unidireccionales
   - Optimizado para servicios internos (.i2p)

🏛 Aplicación en Banco ZK

Anonimización en Banco ZK — Enfoque Integral:
Banco ZK combina múltiples técnicas para privacidad completa:

Capa de Red: Tor/I2P integration	IP oculta
Capa de Relay: Relayers anónimos	Gas payer oculto
Capa de Transacción: ZK Proofs	Detalles ocultos
Capa de Estado: Merkle trees privados	Balances ocultos

Métricas de privacidad objetivo:

- Anonymity set: Todos los usuarios de Banco ZK (>10,000)
- Unlinkability: Transacciones no vinculables entre sí
- Untraceability: Origen de fondos indeterminable
- Plausible deniability: No se puede probar participación

🧮 Nota Matemática

Métricas de Anonimato:
Anonymity Set Size:

k-anonymity: cada usuario es indistinguible de al menos $k-1$ otros

Entropía de Anonimato:

$$H = -\sum_{i=1}^{n} p_i \log_2(p_i)$$

Para distribución uniforme: $H = \log_2(n)$ bits
Anonymity Degree (Diaz et al.):

$$d = \frac{H}{H_{max}} = \frac{H}{\log_2(n)}$$

Donde $d = 1$ es anonimato perfecto, $d = 0$ es identificación completa.

Idea Clave

Principios de Diseño para Sistemas Anónimos:

1. **Defense in depth:** Múltiples capas de protección
2. **Minimize trust:** Reducir puntos de confianza
3. **Uniform behavior:** Todos los usuarios se ven iguales
4. **Plausible deniability:** No se puede probar participación
5. **Forward secrecy:** Compromiso futuro no afecta pasado
6. **Incentive alignment:** Participar debe ser atractivo
7. **Graceful degradation:** Fallos parciales no revelan todo

Chapter 7

Monedas Privadas (Privacy Coins)

Introducción

Las privacy coins representan la aplicación más directa de la criptografía avanzada para lograr privacidad financiera en blockchain. Este capítulo analiza en profundidad los sistemas más importantes: desde Zcash con sus ZK-SNARKs hasta Monero con ring signatures, y los protocolos de privacidad sobre Ethereum como Tornado Cash, Aztec y Railgun.

Idea Clave

Evolución de la privacidad en criptomonedas:

- **2009 - Bitcoin:** Pseudonimato (direcciones públicas, transacciones trazables)
- **2014 - Monero:** Ring signatures para ocultar remitente
- **2016 - Zcash:** ZK-SNARKs para privacidad total
- **2019 - Tornado Cash:** Mixing con ZK sobre Ethereum
- **2022 - Aztec/Railgun:** DeFi privado
- **2023+:** Compliance-friendly privacy (Privacy Pools)

Nota Matemática

Taxonomía de Privacy Coins:

1. **Privacidad nativa (Layer 1):**
   - Zcash: ZK-SNARKs
   - Monero: Ring signatures + CT
   - Firo: Lelantus protocol
2. **Privacidad sobre Ethereum (Layer 2/Contracts):**
   - Tornado Cash: Mixer con SNARKs
   - Aztec: ZK rollup privado
   - Railgun: Shielded pool para ERC-20
3. **Privacidad en otros ecosistemas:**

- Penumbra: Cosmos/IBC
- Namada: Interchain shielded transfers

7.1 Zcash

7.1.1 Historia y Fundamentos

Definición: Zcash

Criptomoneda lanzada en 2016 que implementa transacciones completamente privadas usando ZK-SNARKs:

- **Fundadores:** Zooko Wilcox, basado en protocolo Zerocash
- **Base:** Fork de Bitcoin con adición de transacciones "shielded"
- **Modelo:** UTXO con "notes" encriptadas
- **Dual:** Soporta transacciones transparentes (t-addresses) y privadas (z-addresses)

Nota Matemática

Timeline de Zcash:

- **2014:** Zerocash paper publicado
- **2016:** Lanzamiento de Zcash (Sprout)
- **2018:** Upgrade Sapling (mejor performance)
- **2020:** Canopy (eliminación de founders reward)
- **2022:** NU5 - Orchard (sin trusted setup)
- **2024:** Zcash Shielded Assets (ZSA) en desarrollo

7.1.2 Arquitectura de Transacciones

Definición: Zcash Note

Una "note" en Zcash es el equivalente privado de un UTXO:

$$\text{note} = (d, pk_d, v, rcm)$$

Donde:

- d: Diversifier (permite múltiples direcciones de una clave)
- pk_d: Clave pública derivada del diversifier
- v: Valor (cantidad de ZEC)
- rcm: Randomness para el commitment

Definición: Note Commitment

El commitment a una note se calcula como:

$$cm = \text{COMMIT}(d, pk_d, v, rcm)$$

En Sapling: $cm = \text{PedersenHash}(d \| pk_d \| v \| rcm)$
En Orchard: $cm = \text{SinsemillaCommit}(d \| pk_d \| v \| rcm)$
El commitment se añade a un Merkle tree público. La note en sí permanece encriptada.

Definición: Nullifier

Identificador único que se revela al gastar una note:

$$nf = \text{PRF}_{nk}(cm)$$

Donde nk es la nullifier key derivada de la spending key.
Propiedades:

- Previene doble gasto: mismo nullifier = misma note gastada
- No revela qué note: no se puede vincular nf con cm sin nk

7.1.3 Sapling (2018)

Definición: Sapling

Segunda generación de protocolo shielded de Zcash:

- **Sistema de prueba:** Groth16
- **Curva:** BLS12-381
- **Trusted setup:** Powers of Tau + Sapling MPC
- **Performance:** ~40 segundos proving (vs minutos en Sprout)
- **Memoria:** ~40 MB (vs GB en Sprout)

Nota Matemática

Circuito Sapling Spend:
El circuito de gasto en Sapling verifica:

1. **Merkle proof:** cm está en el árbol con root rt

$$\text{MerkleProof}(cm, \text{path}, rt) = 1$$

2. **Nullifier correcto:**

$$nf = \text{PRF}^{nf}_{nk}(\rho)$$

3. **Autorización de gasto:**

$$\text{SpendAuthSig.Verify}(ak, m, \sigma) = 1$$

4. **Value commitment:**

$$cv = \text{ValueCommit}(v, rcv)$$

Inputs públicos: rt, nf, cv, rk (randomized key)
Inputs privados: note, path, spending key components

7.1.4 Orchard (2022)

Definición: Orchard

Tercera generación del protocolo shielded, eliminando el trusted setup:

- **Sistema de prueba:** Halo2 (basado en PLONK)
- **Curvas:** Pasta (Pallas y Vesta)
- **Sin trusted setup:** Recursion-friendly
- **Actions:** Unifica spend y output en una sola estructura
- **Performance:** Similar a Sapling pero sin setup

Nota Matemática

Curvas Pasta:
Pallas y Vesta son un "2-cycle" de curvas elípticas:

- E_p (Pallas): $y^2 = x^3 + 5$ sobre $\mathbb{F}_p$
- E_q (Vesta): $y^2 = x^3 + 5$ sobre $\mathbb{F}_q$
- Propiedad: $|E_p(\mathbb{F}_p)| = q$ y $|E_q(\mathbb{F}_q)| = p$

Esta propiedad permite recursión eficiente: una prueba sobre Pallas puede verificarse en un circuito sobre Vesta, y viceversa.

Definición: Orchard Action

Un Action combina un spend y un output:

$$\text{Action} = (\text{spend} \parallel \text{output})$$

- Cada action gasta una note y crea una nueva
- Para solo enviar: input es note "dummy" de valor 0
- Para solo recibir: output es note dummy
- Mejora privacidad: no se distingue envío de recepción

7.1.5 Value Balance y Binding Signature

Definición: Value Commitment

Para ocultar valores pero permitir verificar balance:

$$cv = v \cdot G_v + rcv \cdot G_{rcv}$$

Donde:

- v es el valor
- rcv es randomness
- G_v, G_{rcv} son generadores independientes

Es un Pedersen commitment que permite verificación homomórfica.

Nota Matemática

Verificación de Balance:
Para una transacción con inputs $\{v_i\}$ y outputs $\{v_j\}$:
Commitments:

$$cv_i = v_i \cdot G_v + rcv_i \cdot G_{rcv}$$

$$cv_j = v_j \cdot G_v + rcv_j \cdot G_{rcv}$$

Balance check:

$$\sum_i cv_i - \sum_j cv_j = \left(\sum_i v_i - \sum_j v_j\right) \cdot G_v + \left(\sum_i rcv_i - \sum_j rcv_j\right) \cdot G_{rcv}$$

Si los valores balancean ($\sum v_i = \sum v_j + \text{fee}$):

$$\sum cv_i - \sum cv_j = \text{fee} \cdot G_v + R \cdot G_{rcv}$$

La binding signature prueba conocimiento de $R = \sum rcv_i - \sum rcv_j$.

7.1.6 Claves en Zcash

Idea Clave

Capacidades por nivel de clave:

1. **Spending Key:**
   - Control total: gastar, ver incoming/outgoing
   - Derivar todas las otras claves
2. **Full Viewing Key:**
   - Ver todas las transacciones (incoming y outgoing)
   - No puede gastar
   - Útil para auditoría, contabilidad

3. **Incoming Viewing Key:**
   - Ver solo transacciones recibidas
   - No ve a quién enviaste
4. **Address:**
   - Solo permite recibir fondos
   - Diversifiers permiten múltiples direcciones de una clave

7.2 Monero

7.2.1 Filosofía y Diseño

Definición: Monero

Criptomoneda lanzada en 2014 con privacidad obligatoria por defecto:

- **Base:** Protocolo CryptoNote (fork de Bytecoin)
- **Privacidad:** Ring signatures + Stealth addresses + Confidential Transactions
- **Moneda:** XMR
- **Filosofía:** "Financial privacy is a human right"

Aspecto	Monero	Zcash
Criptografía	Ring signatures	ZK-SNARKs
Privacidad	Obligatoria	Opcional
Anonymity set	Ring size (16)	Todo el pool shielded
Prueba tamaño	∼1.5 KB	∼200 bytes
Setup	No	Sí (Sapling) / No (Orchard)
Escalabilidad	Peor	Mejor

Idea Clave

Componentes de privacidad en Monero:

1. **Ring Signatures:** Ocultan *quién* gasta
   - El gastador se "esconde" entre múltiples outputs públicos
2. **Stealth Addresses:** Ocultan *quién* recibe
   - One-time addresses por cada transacción
3. **Confidential Transactions:** Ocultan *cuánto*
   - Pedersen commitments para montos
   - Range proofs para garantizar positividad

7.2.2 Ring Signatures

Definición: Ring Signature

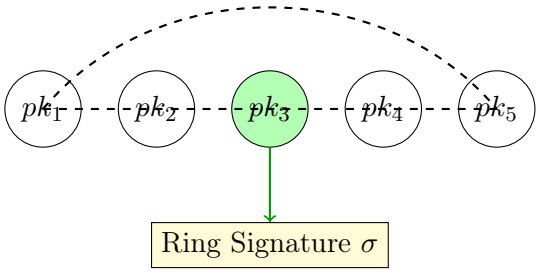
Firma que puede ser verificada como proveniente de "alguno de un grupo", sin revelar cuál miembro específico firmó:

$$\text{RingSign}(m, \{pk_1, ..., pk_n\}, sk_i) \to \sigma$$

$$\text{RingVerify}(m, \{pk_1, ..., pk_n\}, \sigma) \to \{0, 1\}$$

Propiedades:

- **Correctness:** Firma válida del firmante real
- **Anonymity:** Verificador no puede distinguir quién firmó
- **Unforgeability:** No se puede crear firma sin conocer algún sk_i



Verificador sabe que uno de los 5 firmó
pero no puede determinar cuál

Definición: Key Image

Para prevenir doble gasto sin revelar qué output se gasta:

$$I = x \cdot H_p(P)$$

Donde:

- x es la clave privada
- $P = x \cdot G$ es la clave pública
- H_p es hash-to-point

Cada output tiene un único key image. Si el mismo aparece dos veces, es doble gasto.

7.2.3 RingCT (Ring Confidential Transactions)

Definición: RingCT

Combina ring signatures con confidential transactions para ocultar tanto remitente como monto:

- **Introducido:** Enero 2017 (hardfork)

- **Mandatory:** Septiembre 2017
- **Componentes:**
  1. MLSAG para firmas de anillo multi-input
  2. Pedersen commitments para montos
  3. Range proofs para probar positividad

Nota Matemática

Pedersen Commitment en Monero:

$$C = a \cdot G + b \cdot H$$

Donde:

- a es el monto
- b es el blinding factor
- G, H son generadores con relación desconocida

Propiedad homomórfica:

$$C_1 + C_2 = (a_1 + a_2) \cdot G + (b_1 + b_2) \cdot H$$

Para verificar balance: $\sum C_{inputs} - \sum C_{outputs} = 0 \cdot G + \Delta b \cdot H$

7.2.4 Bulletproofs

Definición: Bulletproofs

Sistema de range proofs con tamaño logarítmico:

- **Paper:** B$\acute{}$unz et al., 2018
- **Tamaño:** $O(\log n)$ vs $O(n)$ anterior
- **Agregación:** Múltiples range proofs en una
- **Sin trusted setup:** Solo discrete log assumption

Sistema	Tamaño (1 output)	Tamaño (n outputs)	Setup
Borromean (original)	6.2 KB	$n \times 6.2$ KB	No
Bulletproofs	0.7 KB	$0.7 + 0.064 \cdot \log_2 n$ KB	No
Bulletproofs+	0.6 KB	Similar	No

Para una transacción típica de 2 outputs: Borromean requiere 12.4 KB, Bulletproofs solo 0.8 KB — un ahorro del 93%.

7.2.5 Stealth Addresses

Definición: Stealth Address

Sistema para generar direcciones únicas por transacción:

1. Receptor tiene par de claves $(a, A = aG)$ y $(b, B = bG)$
2. Emisor genera r aleatorio
3. Calcula: $P = H(rA)G + B$ (one-time key)
4. Receptor calcula: $P = H(aR)G + B$ donde $R = rG$
5. Solo receptor puede gastar (conoce $H(aR) + b$)

7.3 Aztec Protocol

7.3.1 Visión y Arquitectura

Definición: Aztec

ZK-rollup con privacidad nativa sobre Ethereum:

- **Versión 1 (2020):** Aztec Connect - bridge privado a DeFi
- **Versión 2 (2023+):** Aztec Network - L2 completo con contratos privados
- **Lenguaje:** Noir (DSL para circuitos ZK)
- **Modelo:** UTXO con notes encriptadas

Idea Clave

Propuesta de valor de Aztec:

1. **Privacidad para DeFi existente:**
   - Usar Uniswap, Aave, etc. de forma privada
   - Sin necesidad de que protocolos cambien
2. **Smart contracts privados:**
   - Estado encriptado
   - Lógica verificable sin revelar inputs
   - Composabilidad entre contratos privados
3. **Rollup efficiency:**
   - Batch proofs para múltiples transacciones
   - Costos amortizados entre usuarios

7.3.2 Noir: Lenguaje para Circuitos Privados

Definición: Noir

Domain-Specific Language (DSL) para escribir programas ZK:

- **Sintaxis:** Similar a Rust
- **Compilación:** Noir → ACIR → Backend proof system
- **Backend-agnostic:** Puede usar diferentes provers
- **Abstracciones:** Alto nivel, oculta complejidad de circuitos

7.4 Railgun

7.4.1 Concepto: Privacidad para Tokens Existentes

Definición: Railgun

Protocolo de privacidad para cualquier token ERC-20 existente:

- **Sin cambios al token:** Funciona con cualquier ERC-20
- **Shielded pool:** Depositar tokens, transferir privadamente, retirar
- **On Ethereum:** Smart contracts, no L2 separado
- **ZK-SNARKs:** Groth16 para pruebas

Intuición

Railgun es como un "Tornado Cash generalizado": mientras Tornado Cash solo soporta ETH y algunos tokens específicos con denominaciones fijas, Railgun soporta cualquier cantidad de cualquier token ERC-20.
Depositas tokens → entran al pool privado → transferencias internas son privadas → retiras a cualquier dirección.

7.5 Tornado Cash

7.5.1 Historia y Diseño

Definición: Tornado Cash

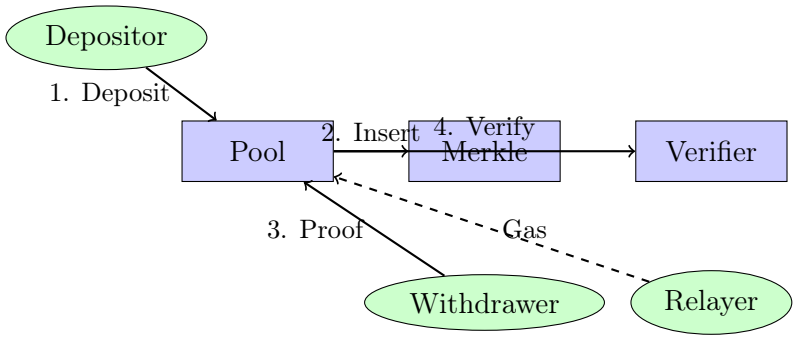
Mixer descentralizado para ETH y tokens ERC-20:

- **Lanzamiento:** 2019
- **Mecanismo:** Shielded pool con denominaciones fijas
- **Tecnología:** Groth16 ZK-SNARKs, Circom
- **Estado actual:** Sancionado por OFAC (Agosto 2022)

Observación

Contexto Legal:
En agosto 2022, el Departamento del Tesoro de EE.UU. (OFAC) sancionó las direcciones de Tornado Cash, alegando que fue usado para lavar más de $7 billones, incluyendo fondos robados por hackers norcoreanos (Lazarus Group).
Esto marcó la primera vez que el gobierno sancionó código (smart contracts) en lugar de personas o empresas. Las implicaciones legales siguen en debate.
Nota educativa: Analizamos Tornado Cash como caso de estudio técnico, no como recomendación de uso.

7.5.2 Arquitectura Técnica



Denominaciones: 0.1, 1, 10, 100 ETH

7.5.3 Relayer Network

Definición: Relayer

Servicio que envía la transacción de withdraw en nombre del usuario:

- **Problema:** Withdraw requiere gas, pero la nueva dirección no tiene ETH
- **Solución:** Relayer paga gas, cobra fee del monto retirado
- **Privacidad:** Usuario nunca conecta su wallet a la nueva dirección

7.6 Umbra (Stealth Payments)

7.6.1 Concepto: Privacidad sin ZK

Definición: Umbra Protocol

Sistema de pagos sigilosos (stealth) para Ethereum que no requiere ZK proofs:

- **Mecanismo:** Stealth addresses usando criptografía de curvas elípticas
- **Ventaja:** Simple, gas-efficient, no requiere proving
- **Limitación:** Solo oculta receptor, no oculta monto
- **Lanzamiento:** 2021

Intuición

El problema: Si Alice quiere recibir pagos, publica su dirección. Cualquiera puede ver todos los pagos que recibe viendo la blockchain.
Solución de Umbra: Alice publica una "meta-address" especial. Cada vez que alguien le paga, genera una dirección única *one-time* que solo Alice puede controlar. Nadie puede vincular estas direcciones a Alice (excepto Alice y el sender).

7.6.2 Protocolo de Stealth Addresses

Nota Matemática

Stealth Address Generation (ECDH):
Setup:

- Receptor genera dos pares de claves:
  - Spending key: $(k_s, K_s = k_s \cdot G)$
  - Viewing key: $(k_v, K_v = k_v \cdot G)$
- Meta-address pública: (K_s, K_v)

Envío (Sender):

1. Genera ephemeral key pair: $(r, R = r \cdot G)$
2. Calcula shared secret: $S = r \cdot K_v$
3. Calcula stealth address: $P = K_s + H(S) \cdot G$
4. Envía fondos a P y publica R on-chain

Recepción (Receiver):

1. Escanea blockchain buscando ephemeral keys R
2. Calcula: $S' = k_v \cdot R$ $(= S$ por ECDH$)$
3. Calcula stealth address: $P' = K_s + H(S') \cdot G$
4. Si $P' = P$: el pago es para mí
5. Private key de P: $p = k_s + H(S')$

7.7 Penumbra

Definición: Penumbra

Blockchain privada en el ecosistema Cosmos con DeFi nativo:

- **Arquitectura:** Zona Cosmos con IBC
- **Privacidad:** Shielded por defecto (como Zcash)
- **DeFi:** DEX privado, staking delegado
- **Viewing keys:** Transparencia selectiva

- **Estado:** Mainnet 2024

Nota Matemática

Componentes de Penumbra:

1. **Shielded Pool:**
   - Multi-asset (cualquier token IBC)
   - Notes con commitments (similar a Zcash)
   - Nullifiers para doble gasto
2. **DEX (ZSwap):**
   - Swaps privados
   - Concentrated liquidity
   - Batch auctions para MEV resistance
3. **Staking:**
   - Delegación privada
   - Voting shielded
   - Rewards distribuidos privadamente
4. **IBC Integration:**
   - Transferencias cross-chain
   - Shielding de tokens de otras cadenas
   - "Privacy hub" del ecosistema

7.8 Comparativa de Privacy Coins

Sistema	Tipo	Crypto	Setup	Proof	Privacy
Zcash Sapling	L1	Groth16	Sí	200B	Opcional
Zcash Orchard	L1	Halo2	No	5KB	Opcional
Monero	L1	Ring+BP	No	1.5KB	Obligatoria
Tornado Cash	Contract	Groth16	Sí	200B	Solo mixing
Aztec	L2	Custom	En dev	Variable	Nativa
Railgun	Contract	Groth16	Sí	200B	Shielded pool
Umbra	Contract	ECDH	No	N/A	Solo receptor
Penumbra	L1 (Cosmos)	Groth16	Sí	200B	Obligatoria

Sistema	Multi-Asset	DeFi	Compliance	Estado
Zcash	Solo ZEC	No	Viewing keys	Producción
Monero	Solo XMR	No	Limitado	Producción
Tornado Cash	ETH + tokens	No	No	Sancionado
Aztec	Cualquier	Sí	En desarrollo	Testnet
Railgun	ERC-20	Limitado	Privacy Pools	Producción
Umbra	ETH + ERC-20	No	Viewing keys	Producción
Penumbra	IBC tokens	DEX, Staking	Viewing keys	Mainnet

7.9 Resumen del Capítulo

Idea Clave

Puntos clave de Privacy Coins:

1. **Zcash:**
   - Pionero en ZK-SNARKs para privacidad
   - Sapling (2018): Groth16, trusted setup
   - Orchard (2022): Halo2, sin trusted setup
   - Notes, commitments, nullifiers = modelo canónico
2. **Monero:**
   - Ring signatures ocultan remitente
   - Stealth addresses ocultan receptor
   - RingCT oculta montos
   - Bulletproofs para range proofs eficientes
   - Privacidad obligatoria, mayor anonymity set
3. **Aztec:**
   - ZK rollup con privacidad nativa
   - Noir: lenguaje para contratos privados
   - Permite DeFi privado sobre Ethereum
4. **Railgun:**
   - Privacy para cualquier ERC-20
   - Shielded pool con ZK-SNARKs
   - No requiere cambios a tokens existentes
5. **Tornado Cash:**
   - Mixer descentralizado para ETH
   - Denominaciones fijas, simple pero limitado
   - Sancionado — lecciones sobre compliance
6. **Umbra:**
   - Stealth addresses sin ZK
   - Simple, eficiente en gas
   - Solo oculta receptor, no monto
7. **Penumbra:**
   - DeFi privado en Cosmos
   - IBC para interoperabilidad
   - Shielded swaps y staking

🏛 Aplicación en Banco ZK

Diseño de Banco ZK — Síntesis de Privacy Coins:
Banco ZK incorpora las mejores características de cada sistema:

Feature	Inspiración
Modelo UTXO con notes	Zcash
Nullifiers	Zcash, Tornado
Sin trusted setup	Orchard (Halo2)
Multi-asset	Railgun, Penumbra
DeFi composable	Aztec
Relayer network	Tornado Cash
Viewing keys	Zcash, Penumbra
Privacy Pools (compliance)	Vitalik et al. 2023

Arquitectura resultante:

1. **Base:** L2 sobre Ethereum (seguridad heredada)
2. **Privacidad:** Obligatoria por defecto
3. **Tokens:** Cualquier ERC-20
4. **Proofs:** Halo2 (sin trusted setup)
5. **Compliance:** Privacy Pools opcionales
6. **DeFi:** AMM privado, lending privado
7. **UX:** Relayers para gas, stealth addresses para recepción

Chapter 8

Técnicas de Unlinkability

Introducción

La unlinkability es una propiedad fundamental de los sistemas de privacidad: la imposibilidad de vincular diferentes acciones, transacciones o identidades de un mismo usuario. Este capítulo explora las técnicas para lograr unlinkability, los ataques que intentan romperla, y las defensas disponibles.

Idea Clave

Jerarquía de privacidad en transacciones:

1. **Confidencialidad:** Ocultar *qué* se transfiere (montos)
2. **Anonimato:** Ocultar *quién* participa (identidades)
3. **Unlinkability:** Ocultar *relaciones* entre acciones
4. **Unobservability:** Ocultar que *algo ocurrió*

Un sistema puede tener anonimato sin unlinkability (pseudónimos fijos) o unlinkability sin anonimato (acciones vinculadas a identidad conocida pero no entre sí).

Definición: Unlinkability (Pfitzmann & Hansen)

Dos items de interés (IOI) son unlinkable si, desde la perspectiva del atacante, la probabilidad de que estén relacionados es igual a la probabilidad a priori (sin información adicional).
Formalmente, para items i_1, i_2:

$$P(i_1 \sim i_2 \mid \text{observación}) = P(i_1 \sim i_2 \mid \text{sin observación})$$

Donde $\sim$ denota "están relacionados".

8.1 Input/Output Unlinkability

8.1.1 Definición Formal

Definición: Input/Output Unlinkability

En una transacción con múltiples inputs $\{I_1, ..., I_n\}$ y outputs $\{O_1, ..., O_m\}$, input/output unlinkability significa que un observador no puede determinar qué input "pagó" a qué output con probabilidad mejor que random:

$$P(I_i \to O_j \mid \text{tx}) = \frac{1}{n \cdot m} \quad \forall i, j$$

Nota Matemática

Niveles de Unlinkability:
Nivel 1 — Intra-transacción:

- Dentro de una sola transacción
- No se puede vincular input específico con output específico
- Ejemplo: CoinJoin con outputs uniformes

Nivel 2 — Inter-transacción:

- Entre diferentes transacciones
- No se puede rastrear el "flujo" de fondos
- Ejemplo: Mixer que rompe la cadena de custody

Nivel 3 — Con el mundo exterior:

- Entre transacciones on-chain e identidades off-chain
- No se puede vincular tx con persona real
- Ejemplo: Uso de Tor + direcciones nuevas + timing aleatorio

8.1.2 Métricas de Unlinkability

Definición: Anonymity Set

Conjunto de posibles orígenes o destinos de una acción. El tamaño del anonymity set determina la fortaleza de la unlinkability:

$$\text{Anonymity Set Size} = |\{x : P(x \text{ es el origen}) > 0\}|$$

Definición: Entropía de Anonimato

Medida más precisa que considera las probabilidades:

$$H = -\sum_{i=1}^{n} p_i \log_2(p_i)$$

Interpretación:

- $H = 0$: Identificación completa (un $p_i = 1$)
- $H = \log_2(n)$: Anonimato perfecto (todos $p_i = 1/n$)
- $0 < H < \log_2(n)$: Anonimato parcial

8.2 Transaction Graph Analysis

8.2.1 El Grafo de Transacciones

Definición: Transaction Graph

Representación de la blockchain como grafo dirigido:

- **Nodos:** Transacciones o direcciones
- **Aristas:** Flujo de fondos
- **Peso:** Monto transferido

Tipos de grafos:

1. **Transaction graph:** Nodos = txs, aristas = output→input
2. **Address graph:** Nodos = direcciones, aristas = pagos
3. **Entity graph:** Nodos = entidades (clusters), aristas = relaciones

8.2.2 Heurísticas de Clustering

Definición: Address Clustering

Técnicas para agrupar direcciones que probablemente pertenecen a la misma entidad.

Nota Matemática

Common Input Ownership Heuristic:
Supuesto: Si múltiples inputs aparecen en la misma transacción, probablemente pertenecen al mismo dueño.
Razón: Para firmar una transacción con múltiples inputs, necesitas las claves privadas de todos. Es raro que múltiples personas coordinen para crear una tx.
Excepción: CoinJoin rompe esta heurística intencionalmente.
Formalización:

$$\text{cluster}(a_1) = \text{cluster}(a_2) \text{ si } \exists\ \text{tx} : a_1, a_2 \in \text{inputs}(\text{tx})$$

Nota Matemática

Change Detection Heuristics:
El "change" (vuelto) de una transacción suele tener características identificables:
H1 — Dirección nueva: El change suele ir a una dirección que no existía antes.

H2 — Número redondo: Pagos suelen ser números redondos (1.0, 0.5, 100). El output "raro" (0.0372819) probablemente es change.
H3 — Self-change: Algunos wallets reusan direcciones de input para change.
H4 — Script type matching: Change suele usar mismo tipo de script que inputs.

8.2.3 Ataques de Análisis de Grafos

Observación

Técnicas avanzadas de deanonimización:

1. **Taint analysis:** Rastrear "contaminación" de fondos desde origen conocido.
2. **Temporal analysis:** Patrones de timing revelan información.
3. **Value flow analysis:** Seguir montos específicos a través del grafo.
4. **Behavioral clustering:** Patrones de uso similares sugieren misma entidad.
5. **Cross-chain analysis:** Correlacionar actividad en múltiples blockchains.

Aplicación en Banco ZK

Defensa contra Graph Analysis en Banco ZK:
Banco ZK es inherentemente resistente a graph analysis porque:

1. **Sin grafo público:**
   - Las transacciones no revelan remitente ni receptor
   - Solo se publican nullifiers y commitments
   - No hay "aristas" visibles entre transacciones
2. **Modelo Note (no UTXO ni Account):**
   - Las notes son privadas
   - Nullifiers no revelan qué note se gasta
   - No se puede construir el grafo de flujo
3. **Anonymity set = todo el pool:**
   - Cualquier note podría ser cualquier output anterior
   - No hay clustering posible

Puntos débiles a mitigar:

- Depósitos/retiros son públicos (boundary del shielded pool)
- Timing de deposits→withdrawals puede correlacionarse
- Solución: Delays aleatorios, batching, cover traffic

8.3 Timing Attacks y Mitigaciones

8.3.1 Anatomía de los Timing Attacks

Definición: Timing Attack

Ataque que explota correlaciones temporales para deanonimizar usuarios:

- **Correlación temporal:** Eventos cercanos en tiempo probablemente relacionados
- **Patrones regulares:** Actividad predecible revela identidad
- **Timing side-channels:** Información filtrada por tiempos de respuesta

8.3.2 Tipos de Timing Attacks

Nota Matemática

Categorías de Timing Attacks:

1. **Deposit-Withdraw Correlation:**
   - Usuario deposita en mixer, retira poco después
   - Adversario correlaciona por proximidad temporal
   - $P(\text{link}) \propto e^{-\lambda \cdot \Delta t}$ donde Δt = tiempo entre eventos
2. **Behavioral Patterns:**
   - Actividad a las mismas horas cada día
   - Patrones semanales (ej: siempre los lunes)
   - Fingerprint basado en distribución temporal
3. **External Correlation:**
   - Withdraw seguido de depósito en exchange
   - Compra online correlacionada con transacción
   - Eventos de red (IP logging) correlacionados con txs
4. **Network-Level Timing:**
   - Observar cuándo un nodo propaga una tx
   - First-seen heuristic para identificar origen
   - Latencia de respuesta revela ubicación

8.3.3 Mitigaciones

Idea Clave

Técnicas de mitigación de timing attacks:

1. **Random delays:** Añadir retardos aleatorios antes de operaciones.
2. **Batching:** Agrupar transacciones de múltiples usuarios.
3. **Cover traffic:** Generar transacciones falsas para confundir.
4. **Constant-time operations:** Operaciones toman mismo tiempo independiente de datos.

8.4 Amount Hiding

8.4.1 Por Qué Ocultar Montos

Intuición

Los montos son altamente identificables. Si transfieres exactamente 1.23456789 BTC, y luego alguien gasta exactamente esa cantidad, probablemente eres tú.
Además, los montos revelan información sensible:

- Salarios (pagos regulares de montos específicos)
- Transacciones comerciales (facturas, precios)
- Patrimonio (balances totales)

8.4.2 Técnicas de Amount Hiding

Definición: Pedersen Commitment

Commitment criptográfico que oculta el valor pero permite verificación:

$$C = g^v \cdot h^r$$

Donde:

- v = valor a comprometer
- r = factor de cegado (randomness)
- g, h = generadores del grupo con relación discreta desconocida

Propiedades:

1. **Hiding:** Sin conocer r, no se puede determinar v
2. **Binding:** Conociendo (v, r), no se puede encontrar (v', r') diferente
3. **Homomórfico:** $C_1 \cdot C_2 = g^{v_1+v_2} \cdot h^{r_1+r_2}$

Nota Matemática

Verificación de Balance con Pedersen:
Para transacción con inputs $\{v_i\}$ y outputs $\{v_j\}$:
Balance check:

$$\prod_i C_i = \prod_j C_j \cdot g^{\text{fee}}$$

Expandiendo:

$$g^{\sum v_i} h^{\sum r_i} = g^{\sum v_j + \text{fee}} h^{\sum r_j}$$

Esto requiere $\sum v_i = \sum v_j + \text{fee}$ **y** $\sum r_i = \sum r_j$.

Definición: Range Proof

Prueba de conocimiento cero de que un valor comprometido está en un rango:

$$\text{RangeProof}(C, v, r) \implies v \in [0, 2^n)$$

Sin range proofs, alguien podría crear valor negativo oculto.

Sistema	Tamaño	Verificación	Agregación
Borromean	$O(n)$ bits	$O(n)$	No
Bulletproofs	$O(\log n)$	$O(n)$	Sí
Bulletproofs+	$O(\log n)$	$O(n)$	Sí, mejor
SNARK-based	$O(1)$	$O(1)$	Nativo

8.5 Decoy Selection

8.5.1 El Problema de los Decoys

Definición: Decoy (Señuelo)

En sistemas de ring signatures, un decoy es un output público incluido en el "ring" para confundir a observadores sobre cuál es el output realmente gastado.

- **Ring size:** Número total de miembros (real + decoys)
- **Monero actual:** Ring size = 16 (1 real + 15 decoys)
- **Anonymity set efectivo:** Puede ser menor que ring size

Intuición

La idea es simple: si gastas el output #5, incluyes también los outputs #2, #8, #12, ... en tu ring. El verificador sabe que uno de ellos se gastó, pero no cuál.
El problema: si los decoys son "obviamente falsos" (muy viejos, ya gastados en otra tx visible, etc.), el anonymity set se reduce.

8.5.2 Distribución Gamma para Decoys

Definición: Gamma Distribution para Decoys

Monero usa una distribución gamma para seleccionar decoys basándose en su "edad":

$$f(x;\alpha,\beta) = \frac{\beta^\alpha x^{\alpha-1} e^{-\beta x}}{\Gamma(\alpha)}$$

Parámetros calibrados para imitar el comportamiento real de gasto:

- α (shape): Controla la forma de la curva
- β (rate): Controla qué tan rápido decae

8.5.3 Ataques contra Decoy Selection

Observación

Ataques conocidos:

1. **Temporal analysis:** Si los decoys son de hace años y el real es de hoy, obvio. Papers han mostrado 80%+ de precisión.
2. **Spent output detection:** Si un decoy ya fue gastado (key image conocido), eliminarlo reduce el anonymity set.
3. **Chain reaction:** Si identifico el real en tx A, puedo usarlo para analizar tx B que lo usó como decoy.
4. **Unusual denominations (pre-RingCT):** Si el monto es único, los decoys con otros montos son obvios.

8.6 Modelos de Datos: Note vs UTXO vs Account

8.6.1 Comparación de Modelos

Definición: UTXO Model (Bitcoin)

Transacciones consumen outputs existentes y crean nuevos:

- **Estado:** Set de Unspent Transaction Outputs
- **Transacción:** inputs (referencias a UTXOs) → outputs (nuevos UTXOs)
- **Privacidad:** Pseudónima (graph analysis posible)

Definición: Account Model (Ethereum)

Estado global con balances por cuenta:

- **Estado:** Mapping address → balance
- **Transacción:** Actualiza balances directamente

- **Privacidad:** Baja (todo público)

Definición: Note Model (Zcash, Tornado)

Variante privada del modelo UTXO:

- **Estado:** Merkle tree de note commitments + set de nullifiers
- **Note:** $(value, owner, randomness)$ encriptada
- **Privacidad:** Alta (valores y owners ocultos)

Aspecto	UTXO	Account	Note
Paralelismo	Alto	Bajo	Alto
Privacidad inherente	Baja	Muy baja	Alta
Complejidad tx	Media	Baja	Alta
Verificación	Simple	Simple	ZK proof
Graph analysis	Posible	Fácil	Imposible
Doble gasto	Check UTXO set	Check nonce	Check nullifier set

8.7 Shielded Pools

8.7.1 Arquitectura de Shielded Pool

Definición: Shielded Pool

Conjunto de fondos donde todas las transacciones internas son privadas:

- **Boundary:** Interfaz entre mundo público y privado
- **Interior:** Todas las operaciones son anónimas
- **Anonymity Set:** Todos los fondos en el pool

8.7.2 Operaciones del Pool

Definición: Shield (Deposit)

Mover fondos del mundo público al privado:

1. Usuario envía tokens al contrato (público)
2. Genera note con commitment
3. Commitment se añade al Merkle tree
4. Fondos ahora son "anónimos"

Definición: Unshield (Withdraw)

Mover fondos del pool al mundo público:

1. Usuario genera ZK proof de que posee note válida

2. Revela nullifier (marca note como gastada)
3. Contrato envía tokens a dirección especificada

Definición: Transfer (Shielded)

Mover fondos dentro del pool:

1. Usuario genera ZK proof
2. Revela nullifiers de inputs
3. Añade commitments de outputs
4. **Ningún valor ni dirección es visible**

Aplicación en Banco ZK

Shielded Pool de Banco ZK:
El pool de Banco ZK es la implementación central del sistema:
Características:

- Multi-token: Soporta cualquier ERC-20
- Unified anonymity set: Todos los tokens comparten anonimato
- Efficient proofs: Halo2 sin trusted setup
- Compliance-ready: Privacy Pools opcionales

Anonymity set:

$$\text{AS}_{\text{Banco ZK}} = \sum_{\text{tokens}} \text{deposits} - \text{withdrawals}$$

Con target de 10,000+ notes activas para entropía > 13 bits.

8.8 Splitting y Merging de Notas

8.8.1 Motivación

Definición: Split (División)

Dividir una note en múltiples notes más pequeñas:

$$\text{Note}(v) \to \text{Note}(v_1) + \text{Note}(v_2) + ... + \text{Note}(v_n)$$

donde $v = v_1 + v_2 + ... + v_n$

Definición: Merge (Consolidación)

Combinar múltiples notes en una sola:

$$\text{Note}(v_1) + \text{Note}(v_2) + ... + \text{Note}(v_n) \to \text{Note}(v)$$

donde $v = v_1 + v_2 + ... + v_n$

8.8.2 Riesgos de Privacidad

Observación

Problemas de privacidad en split/merge:

1. **Temporal correlation:** Si depositas 100 ETH y haces split inmediato, obvio.
2. **Amount fingerprinting:** Split 100 → (37, 63): montos específicos identificables.
3. **Merge reveals common ownership:** Si mergeas A + B → C, sabes que A y B tenían mismo dueño.
4. **Graph analysis of splits:** Árbol de splits puede rastrearse ("peeling attack").

8.8.3 Estrategias para Preservar Privacidad

Idea Clave

Mejores prácticas para split/merge:

1. **Usar denominaciones estándar:** Split en potencias de 2 o 10: 100 → 50 + 50, no 100 → 37 + 63.
2. **Añadir delays:** No hacer split inmediatamente después de deposit.
3. **Evitar merge innecesario:** Mejor usar múltiples inputs en transacción que merge previo.
4. **Batch con otros:** Si el sistema soporta, hacer split/merge en misma tx que otros usuarios.
5. **Dummy outputs:** Añadir outputs de valor 0 para confundir.

8.9 Resumen del Capítulo

Idea Clave

Puntos clave de Técnicas de Unlinkability:
1. Fundamentos:

- Unlinkability = imposibilidad de vincular acciones del mismo usuario
- Jerarquía: confidencialidad → anonimato → unlinkability → unobservability
- Métricas: anonymity set size, entropía de anonimato

2. Graph Analysis:

- Heurísticas de clustering: common input ownership, change detection
- Ataques: taint analysis, temporal analysis, value flow analysis

- Defensa ZK: sin grafo público, nullifiers no vinculables

3. **Timing Attacks:**
   - Correlación deposit-withdraw, patrones comportamentales
   - Mitigaciones: random delays, batching, cover traffic
4. **Amount Hiding:**
   - Pedersen commitments con propiedades homomórficas
   - Range proofs para prevenir inflación oculta
   - Alternativa: denominaciones fijas (Tornado Cash)
5. **Decoy Selection:**
   - Distribución gamma para imitar comportamiento real
   - Ataques: temporal analysis, chain reaction
   - ZK approach elimina necesidad de decoys
6. **Modelos de Datos:**
   - UTXO: paralelizable pero graph analysis posible
   - Account: simple pero baja privacidad
   - Note: alta privacidad, requiere ZK proofs
7. **Shielded Pools:**
   - Shield/Unshield como boundary público-privado
   - Transfers internos completamente privados
   - Anonymity set = todo el pool
8. **Split/Merge:**
   - Necesarios pero riesgosos para privacidad
   - Mejores prácticas: denominaciones estándar, delays, batching

🏛 Aplicación en Banco ZK

Unlinkability en Banco ZK — Síntesis:
Banco ZK implementa unlinkability en múltiples capas:

Capa	Técnica
Red	Tor/I2P para conexiones
Transacción	Notes + nullifiers + ZK proofs
Montos	Pedersen commitments con range proofs
Timing	Random delays en operaciones
Identidad	Anonymous credentials para compliance

Resultado: Sistema donde un observador externo no puede:

- Vincular depósitos con retiros
- Rastrear flujo de fondos dentro del pool
- Identificar partes de una transacción
- Correlacionar actividad por timing o montos

Solo el usuario (con su viewing key) puede ver su propio historial.

Conclusión del Volumen 2

Este segundo volumen de la serie **Banco ZK** ha completado el recorrido por los fundamentos teóricos y prácticos necesarios para comprender y construir sistemas de privacidad en blockchain.

Resumen por Capítulo

1. **Compromisos y Estructuras:** Pedersen, KZG, Merkle trees, estructuras de datos para ZK.

2. **Infraestructura ZK:** Oracles, relayers, circuitos eficientes, optimizaciones.

3. **Privacidad Avanzada:** Técnicas de ZK para transacciones confidenciales.

4. **DeFi y Aplicaciones:** AMMs privados, lending, bridges cross-chain.

5. **Avanzado e Investigación:** FHE, MPC, técnicas de frontera.

6. **Protocolos de Anonimización:** CoinJoin, Tor, mixers, DC-nets.

7. **Monedas Privadas:** Zcash, Monero, Aztec, Railgun, Tornado Cash.

8. **Técnicas de Unlinkability:** Graph analysis, timing attacks, shielded pools.

El Sistema Banco ZK Completo

Combinando ambos volúmenes, Banco ZK implementa:

Componente	**Implementación**
Modelo de datos	Notes con commitments Pedersen
Privacidad de montos	Value commitments + range proofs
Prevención doble gasto	Nullifiers verificados con ZK
Prueba de validez	Halo2 (sin trusted setup)
Estructura de estado	Sparse Merkle Tree
Multi-asset	Cualquier token ERC-20
DeFi privado	AMM, lending con ZK
Compliance	Privacy Pools opcionales
Red	Soporte Tor/I2P
Identidad	Anonymous credentials + DIDs

Métricas del Volumen 2

Métrica	**Cantidad**
Capítulos	8
Definiciones formales	$\sim$100
Tablas comparativas	$\sim$25
Páginas estimadas	$\sim$200
Referencias bibliográficas	95

"Privacy is not about having something to hide.
Privacy is about having the right to choose
what to reveal about yourself."

Bibliography

[1] Pedersen, T. P. (1991). *Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing*. CRYPTO.

[2] Kate, A., Zaverucha, G., & Goldberg, I. (2010). *Constant-Size Commitments to Polynomials and Their Applications*. ASIACRYPT.

[3] Gabizon, A., Williamson, Z., & Ciobotaru, O. (2019). *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*. IACR ePrint.

[4] Gabizon, A., & Williamson, Z. (2020). *plookup: A simplified polynomial protocol for lookup tables*. IACR ePrint.

[5] Bowe, S., Grigg, J., & Hopwood, D. (2019). *Halo: Recursive Proof Composition without a Trusted Setup*. IACR ePrint.

[6] Kothapalli, A., Setty, S., & Tzialla, I. (2022). *Nova: Recursive Zero-Knowledge Arguments from Folding Schemes*. CRYPTO.

[7] Kothapalli, A., & Setty, S. (2022). *SuperNova: Proving universal machine executions without universal circuits*. IACR ePrint.

[8] B́unz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., & Maxwell, G. (2018). *Bulletproofs: Short Proofs for Confidential Transactions and More*. S&P.

[9] Groth, J. (2016). *On the Size of Pairing-based Non-interactive Arguments*. EUROCRYPT.

[10] Bowe, S., Grigg, J., & Hopwood, D. (2020). *The Pasta Curves for Halo 2 and Beyond*. ZCash Documentation.

[11] Pertsev, A., Semenov, R., & Storm, R. (2019). *Tornado Cash Privacy Solution*. Whitepaper.

[12] Buterin, V., et al. (2023). *Blockchain Privacy and Regulatory Compliance: Towards a Practical Equilibrium*. SSRN.

[13] Buterin, V. (2022). *The different types of ZK-EVMs*. Blog post.

[14] RISC Zero Team. (2023). *RISC Zero zkVM: Technical Documentation*. https://dev.risczero.com

[15] Succinct Labs. (2024). *SP1: A performant, 100% open-source, contributor-friendly zkVM*. Documentation.

[16] Scroll Team. (2023). *Scroll: Native zkEVM Layer 2 for Ethereum*. Technical Documentation.

[17] Polygon Team. (2023). *Polygon zkEVM: Technical Documentation*. https://docs.polygon.technology

[18] Rivest, R., Shamir, A., & Tauman, Y. (2001). *How to Leak a Secret*. ASIACRYPT.

[19] Liu, J. K., Wei, V. K., & Wong, D. S. (2004). *Linkable Spontaneous Anonymous Group Signature for Ad Hoc Groups*. ACISP.

[20] van Saberhagen, N. (2012). *CryptoNote v2.0*. Whitepaper.

[21] Goodell, B., Noether, S., & RandomRun. (2019). *Concise Linkable Ring Signatures and Forgery Against Adversarial Keys*. IACR ePrint.

[22] Noether, S. (2020). *Triptych: Logarithmic-sized Linkable Ring Signatures with Applications*. IACR ePrint.

[23] Todd, P. (2014). *Stealth Addresses*. Bitcoin Wiki.

[24] Wahrst?tter, A., et al. (2022). *EIP-5564: Stealth Address Standard*. Ethereum Improvement Proposals.

[25] ScopeLift. (2021). *Umbra Protocol: Privacy-preserving stealth payments for Ethereum*. Documentation.

[26] Electric Coin Company. (2020). *Zcash Protocol Specification: Key Components*. https://zips.z.cash

[27] Chaum, D. (1981). *Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms*. Communications of the ACM.

[28] Nym Technologies. (2021). *Nym Whitepaper*. https://nymtech.net

[29] Dingledine, R., Mathewson, N., & Syverson, P. (2004). *Tor: The Second-Generation Onion Router*. USENIX Security.

[30] I2P Project. (2003). *I2P: The Invisible Internet Project*. Technical Documentation.

[31] Yao, A. C. (1982). *Protocols for Secure Computations*. FOCS.

[32] Shamir, A. (1979). *How to Share a Secret*. Communications of the ACM.

[33] Yao, A. C. (1986). *How to Generate and Exchange Secrets*. FOCS.

[34] Intel Corporation. (2016). *Intel Software Guard Extensions Programming Reference*. Intel Developer Zone.

[35] Secret Network. (2020). *Secret Network: Privacy-Preserving Smart Contracts*. Documentation.

[36] Oasis Labs. (2020). *The Oasis Network: A Privacy-First, Decentralized Computing Platform*. Whitepaper.

[37] Komlo, C., & Goldberg, I. (2020). *FROST: Flexible Round-Optimized Schnorr Threshold Signatures*. SAC.

[38] Adams, H., Zinsmeister, N., & Robinson, D. (2020). *Uniswap v2 Core*. Whitepaper.

[39] Adams, H., et al. (2021). *Uniswap v3 Core*. Whitepaper.

[40] Egorov, M. (2019). *StableSwap - Efficient Mechanism for Stablecoin Liquidity*. Curve Finance Whitepaper.

[41] Martinelli, F., & Mushegian, N. (2019). *Balancer: A Non-Custodial Portfolio Manager*. Whitepaper.

[42] Leshner, R., & Hayes, G. (2019). *Compound: The Money Market Protocol*. Whitepaper.

[43] Aave Team. (2020). *Aave Protocol Whitepaper*. https://aave.com

[44] Ellis, S., Juels, A., & Nazarov, S. (2017). *ChainLink: A Decentralized Oracle Network*. Whitepaper.

[45] Qin, K., et al. (2021). *Attacking the DeFi Ecosystem with Flash Loans*. FC 2021.

[46] Pintail. (2019). *Uniswap: A Good Deal for Liquidity Providers?*. Medium.

[47] Succinct Labs. (2023). *SP1: A zkVM for Ethereum*. Documentation.

[48] LayerZero Labs. (2021). *LayerZero: Trustless Omnichain Interoperability Protocol*. Whitepaper.

[49] Kwon, J., & Buchman, E. (2019). *The Inter-Blockchain Communication Protocol*. Cosmos Documentation.

[50] Herlihy, M. (2018). *Atomic Cross-Chain Swaps*. PODC.

[51] EZKL Team. (2023). *EZKL: Easy Zero-Knowledge Machine Learning*. Documentation.

[52] Worldcoin Team. (2023). *Worldcoin: Privacy-Preserving Proof of Personhood*. Whitepaper.

[53] Giza Team. (2023). *Giza: Verifiable Machine Learning on StarkNet*. Documentation.

[54] Trail of Bits. (2022). *Circomspect: A Static Analyzer for Circom*. GitHub.

[55] Picus Team. (2023). *Picus: Automated Under-Constraint Detection for ZK Circuits*. IACR ePrint.

[56] Ecne Team. (2022). *Ecne: Automated Verification of ZK Circuits*. Documentation.

[57] Polygon Zero Team. (2022). *Plonky2: Fast Recursive Arguments with PLONK and FRI*. Documentation.

[58] Polygon Zero Team. (2023). *Plonky3: Next Generation ZK Proving System*. Documentation.

[59] B́unz, B., et al. (2021). *SnarkPack: Practical SNARK Aggregation*. FC 2022.

[60] Mina Foundation. (2021). *Mina Protocol: A Succinct Blockchain*. Whitepaper.

[61] Kothapalli, A., Setty, S., & Tzialla, I. (2022). *Nova: Recursive Zero-Knowledge Arguments from Folding Schemes*. CRYPTO.

[62] Kothapalli, A., & Setty, S. (2022). *SuperNova: Proving Universal Machine Executions*. IACR ePrint.

[63] Kothapalli, A., & Setty, S. (2023). *HyperNova: Recursive Arguments from Multilinear IOP*. IACR ePrint.

[64] Shor, P. (1994). *Algorithms for Quantum Computation*. FOCS.

[65] Grover, L. (1996). *A Fast Quantum Mechanical Algorithm for Database Search*. STOC.

[66] NIST. (2024). *Post-Quantum Cryptography Standardization*. NIST.

[67] Albrecht, M., et al. (2023). *LaBRADOR: Lattice-Based SNARKs*. IACR ePrint.

[68] Maxwell, G. (2013). *CoinJoin: Bitcoin Privacy for the Real World*. Bitcoin Forum.

[69] zkSNACKs. (2018). *Wasabi Wallet: Privacy-Focused Bitcoin Wallet*. Documentation.

[70] Ficsor, A., et al. (2021). *WabiSabi: Centrally Coordinated CoinJoins with Variable Amounts*. IACR ePrint.

[71] JoinMarket Team. (2015). *JoinMarket: Decentralized Bitcoin CoinJoin Implementation*. GitHub.

[72] Ruffing, T., et al. (2014). *CoinShuffle: Practical Decentralized Coin Mixing for Bitcoin*. ESORICS.

[73] Ruffing, T., et al. (2017). *CoinShuffle++: Efficient Decentralized Mixing*. NDSS.

[74] Chaum, D. (1982). *Blind Signatures for Untraceable Payments*. CRYPTO.

[75] Fedimint Team. (2022). *Fedimint: Federated E-Cash on Bitcoin*. Documentation.

[76] Cashu Team. (2022). *Cashu: Chaumian E-Cash System for Bitcoin*. Documentation.

[77] Venkatakrishnan, S., et al. (2017). *Dandelion: Redesigning the Bitcoin Network for Anonymity*. SIGMETRICS.

[78] Fanti, G., et al. (2018). *Dandelion++: Lightweight Cryptocurrency Networking*. SIGMETRICS.

[79] Chaum, D. (1988). *The Dining Cryptographers Problem*. Journal of Cryptology.

[80] Beresford, A., & Stajano, F. (2003). *Location Privacy in Pervasive Computing*. IEEE Pervasive Computing.

[81] Dingledine, R., Mathewson, N., & Syverson, P. (2004). *Tor: The Second-Generation Onion Router*. USENIX Security.

[82] I2P Team. (2003). *I2P: The Invisible Internet Project*. Documentation.

[83] Hopwood, D., et al. (2016). *Zcash Protocol Specification*. Electric Coin Company.

[84] Zcash Team. (2018). *Sapling Protocol Specification*. Electric Coin Company.

[85] Zcash Team. (2022). *Orchard Protocol Specification*. Electric Coin Company.

[86] van Saberhagen, N. (2013). *CryptoNote v 2.0*. CryptoNote Whitepaper.

[87] Noether, S., et al. (2016). *Ring Confidential Transactions*. Monero Research Lab.

[88] B́unz, B., et al. (2018). *Bulletproofs: Short Proofs for Confidential Transactions*. IEEE S&P.

[89] Noether, S., & Goodell, B. (2020). *CLSAG: A Compact Linkable Spontaneous Anonymous Group Signature*. Monero Research Lab.

[90] Aztec Team. (2023). *Aztec Protocol Specification*. Aztec Documentation.

[91] Aztec Team. (2023). *Noir: Domain-Specific Language for Zero-Knowledge Proofs*. Documentation.

[92] Railgun Team. (2021). *Railgun Protocol: Privacy for DeFi*. Whitepaper.

[93] Tornado Cash Team. (2019). *Tornado Cash: Decentralized Mixer for Ethereum*. Documentation.

[94] ScopeLift. (2021). *Umbra Protocol: Stealth Addresses for Ethereum*. Documentation.

[95] Penumbra Labs. (2023). *Penumbra Protocol Specification*. Documentation.

[96] Pfitzmann, A., & Hansen, M. (2010). *A Terminology for Talking about Privacy by Data Minimization*. TU Dresden.

[97] Meiklejohn, S., et al. (2013). *A Fistful of Bitcoins: Characterizing Payments Among Men with No Names*. IMC.

[98] Reid, F., & Harrigan, M. (2013). *An Analysis of Anonymity in the Bitcoin System*. Security and Privacy in Social Networks.

[99] M?ser, M., et al. (2018). *An Empirical Analysis of Traceability in the Monero Blockchain*. PoPETs.

[100] Yu, Z., et al. (2019). *New Empirical Traceability Analysis of CryptoNote-Style Blockchains*. Financial Cryptography.

[101] Wijaya, D., et al. (2021). *Monero Ring Attack: Recreating Zero Mixin Transaction Effect*. Trust, Security and Privacy in Computing.

[102] Kappos, G., et al. (2018). *An Empirical Analysis of Anonymity in Zcash*. USENIX Security.

[103] Biryukov, A., & Tikhomirov, S. (2019). *Deanonymization and Linkability of Cryptocurrency Transactions*. IEEE EuroS&P.